

spintent

SPANISH PARSE INTENTS

v0.99 2026-08-27^{*}

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua $\text{\LaTeX}$.

Índice

1 Descripción	1	3.3 El comando <code>\spmoney</code>	6
1.1 Carga del paquete	1	3.3.1 Llaves para <code>\spmoney</code>	7
1.2 El paquete <code>babel</code>	2	3.3.2 Monedas y alias para <code>\spmoney</code>	7
1.3 Fuentes tipográficas	2	3.4 El comando <code>\spshort</code>	7
1.4 El paquete <code>hyperref</code>	2	3.4.1 Llaves para <code>\spshort</code>	9
2 El comando <code>\setspintent</code>	3	3.4.2 Argumentos para <code>\spshort</code>	9
3 The big four	3	4 Utilidades generales	11
3.1 El comando <code>\spnum</code>	3	5 Utilidades para símbolos	11
3.1.1 Llaves para <code>\spnum</code>	3	6 Utilidades para números	13
3.2 El comando <code>\spunit</code>	4	7 Utilidades para geometría plana	17
3.2.1 Llaves para <code>\spunit</code>	4	8 Referencias	24
3.2.2 Argumentos para <code>\spunit</code>	4	9 Change history	25
3.2.3 El comando <code>\spnewunit</code>	6	10 Índice de la Documentación	26
3.2.4 El comando <code>\spunitalias</code>	6	11 Implementation	28
		12 Índice de la Implementación	110

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua $\text{\LaTeX}$ ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

^{*}Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-08-27.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «A Work in Progress»¹.

- El requerimiento mínimo es $\LaTeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/lualatex/spintent/
spintent.lua  ⇒ TDS:tex/lualatex/spintent/
spintent.pdf  ⇒ TDS:doc/lualatex/spintent/
README.md    ⇒ TDS:doc/lualatex/spintent/
spintent.dtx  ⇒ TDS:source/lualatex/spintent/
spintent.ins  ⇒ TDS:source/lualatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves (*keys*) aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma ';'.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}`

default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '100000' en la salida PDF.

`read-sign = {⟨terse | clear⟩}`

default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}`

default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘,’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘,’ periódica.

`notation-sym = {(times | cdot)}`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘x’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit</code>	<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad)}</code>
	<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>
	<code>\spunit[key = val]{(número unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad*unidad)}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^{2}}` se imprimirá ‘23ms²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra*» ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^{2}}` se imprimirá ‘23m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra*» ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {true | false}`

default: `false`

Establece si se *imprime* `\cdot` ‘.’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en `true`, se imprimirá ‘.’ entre las `{(unidades)}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {true | false}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {por | partido-por}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra*» ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\langle nueva unidad \rangle}{\langle verbalización NVDA \rangle}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\langle \text{nueva unidad} \rangle$ que establece lo que se imprimirá en la salida PDF y $\langle \text{verbalización NVDA} \rangle$ que establece como NVDA/MathCAT verbalizará la $\langle \text{nueva unidad} \rangle$.

La $\{\langle \textit{nueva unidad} \rangle\}$ no debe estar registrada como $\{\langle \textit{unidad} \rangle\}$ o $\{\langle \textit{alias} \rangle\}$ para `\spunit` descritos en §3.2.2. Si la $\{\langle \textit{nueva unidad} \rangle\}$ ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\langle\text{*nueva unidad*}\rangle$ es «global» y se almacena en el módulo `spintent.lua` como una $\langle\text{*unidad*}\rangle$ válida para pasar como $\langle\text{*argumento*}\rangle$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\langle\text{*nueva unidad*}\rangle$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spunitalias` `\spunitalias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: `{\nuevo alias}` que establece es el nuevo `{\alias}` de `{\argumento}` para `\spunit` y `{\expresión de unidades}` que puede contener multiplicación o división de `{\unidades}` ya registradas que se imprimirán en la salida PDF.

El `{\langle nuevo alias \rangle}` no debe estar registrado como `{\langle unidad \rangle}` o `{\langle alias \rangle}` para `\spunit` descritos en §3.2.2 o estar definido como `{\langle unidad \rangle}` por medio del comando `\spnewunit`. Si el `{\langle nuevo alias \rangle}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un $\{\langle \text{nuevo alias} \rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle \text{alias} \rangle\}$ valido para pasar como $\{\langle \text{argumento} \rangle\}$ a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^{\text{3}}/s}` creará el $\{\langle \text{alias} \rangle\}$ ‘caudal’ y cuando se utilice $\$ \backslash \text{spunit}\{50 \text{ caudal}\} \$$ se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<code>\spmoney</code>	<code>\spmoney{<i>cifra</i>}</code>
	<code>\spmoney{<i>cifra moneda</i>}</code>
	<code>\spmoney[<i>key = val</i>]{<i>cifra moneda</i>}</code>

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{«cifra»}` que contiene la «*cifra numérica*» positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del `{«argumento»}` acompañada de un «*símbolo de moneda*» o «*alias de moneda*» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `$\spmoney{500}$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\$ \spmoney{500} dolares` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos dolares» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«símbolo de moneda» correspondiente. Por ejemplo: `\$ \spmoney{500 clp}` imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «símbolo de moneda» como ‘\$’, ‘€’, ‘£’ o un «alias de moneda» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «símbolo de moneda» a la izquierda o a la derecha de la «cifra numérica» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «símbolo de moneda» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «símbolo de moneda» poco habitual que esté soportada por `spintent`. El `{\argumento}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{\#1}` o `\mathbf{\#1}` por ejemplo. La coma ‘,’ o el punto ‘.’ como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.3.1 Llaves para `\spmoney`

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {\moneda | alias}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «cifra numérica» como `{\argumento}` al comando `\spmoney`. El «símbolo de moneda» o el «alias de moneda» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {\true | false}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá ‘\$100,5’ en la salida PDF y NVDA/MathCAT verbalizará «cien dólares con cincuenta centavos».

`dolar-math = {\true | false}` default: true

Establece si el *símbolo* ‘\$’ se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir ‘\$’ en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir ‘\$’ en la salida PDF.

3.3.2 Monedas y alias para `\spmoney`

Resumen de los `{\símbolos de moneda}` y `{\alias de moneda}` soportados como `{\argumento}` para `\spmoney`.

3.4 El comando `\spshort`

`\spshort` `\spshort{\abreviaturas | acrónimos | números ordinales}`
`\spshort[key = val]{\abreviaturas | acrónimos | números ordinales}`
`\spshort[read-arg={\verbalización del argumento para NVDA}]{\comandos LATEX}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{\abreviaturas}`, `{\acrónimos}` o `{\números ordinales}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr . a}` se imprimirá ‘Dr.^a’ en la salida PDF y NVDA/MathCAT verbalizará «doctora»; si se ejecuta `\spshort{onu}` se imprimirá ‘ONU’ en la salida PDF y NVDA/MathCAT verbalizará «organización de las naciones unidas» y si se ejecuta `\spshort{mineduc}` se imprimirá ‘Mineduc’ en la salida PDF y NVDA/MathCAT verbalizará «ministerio de educación».

Para la escritura de `{\números ordinales}` se puede ejecutar `\spshort{1 . a}` que imprimirá ‘1.^a’ en la salida PDF y NVDA/MathCAT verbalizará «primera»; si se ejecuta `\spshort{1 . o}` imprimirá ‘1.^o’ en la salida PDF y NVDA/MathCAT verbalizará «primero».

Para crear una nueva `{\abreviatura}`, `{\acrónimo}` o `{\número ordinal}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{\comandos LATEX}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{\números ordinales}` se debe tener cuidado y no confundir la «letra o voladita» ‘^o’ con el símbolo de grado ‘°’. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «letra voladita», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1 . o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₾	grivna, grivnas	grivna	₾	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₾	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = { <verbalización NVDA> }` default: no usado

Establece la verbalización del argumento { <comandos $\LaTeX$ > } de la nueva { <abreviatura> }, { <acrónimo> } o { <número ordinal> } para NVDA. El valor de esta llave siempre debe pasarse entre ‘ { } ’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva { <abreviatura> } que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = { <true | false> }` default: false

Establece si se usarán versalitas para imprimir { <acrónimos> } y para la «letra volada» en { <abreviaturas> } y { <números ordinales> } respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán versalitas. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el argumento `{<siglo>}`. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el argumento `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <separador>
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «delimitadores» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «paréntesis izquierdo».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none | big | Big | bigg | Bigg>} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true | false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘÷’ en la salida PDF y NVDA/MathCAT verbalizará «dividido».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style | two-dots | slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido | dividido-por | dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «dividido»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «dividido por» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «dividido entre».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «forma» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmsym$` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «por».

Llaves para `\spmsym`

`prefix = {\langle prefijo \rangle}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`suffix = {\langle sufijo \rangle}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`set-sym = {\langle cross | dot | asterisk \rangle}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá `×`; si se establece en `dot`, imprimirá `·` y si se establece en `asterisk`, imprimirá `*`.

`read-sym = {\langle por | multiplicado-por | veces \rangle}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {\langle true | false \rangle}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

`\spdiv` `\spdiv{\langle dividendo \rangle}{\langle divisor \rangle}`
`\spdiv[\langle key = val \rangle]{\langle dividendo \rangle}{\langle divisor \rangle}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spdiv{20}{26} \$` imprimirá `20 : 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{\langle factor \rangle}{\langle factor \rangle}`
`\spmul[\langle key = val \rangle]{\langle factor \rangle}{\langle factor \rangle}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spmul{20}{26} \$` imprimirá `20 · 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}` pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \text{factor} \rangle \}$ y $\{ \langle \text{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnD$` imprimirá ‘D(n)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnD(6)$`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \text{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)$` imprimirá ‘D(6) = {1,2,3,6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \text{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)$` imprimirá ‘D(m)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de m»; si se establece en `false`, el $(\langle \text{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \text{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \text{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \text{argumento} \rangle)$, `\spnM$` imprimirá ‘M(n)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de n».

Si se usa con $(\langle \text{argumento} \rangle)$, por ejemplo, `\spnM(6)$`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \text{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)$` imprimirá ‘M(6) = {6,12,18,24,30,...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM(argumento)` cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc(⟨números separados por coma⟩)
\spmc[key = val](⟨números separados por coma⟩)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = { (true | false) }`

default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = { (terse | clear) }`

default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = { (prefijo) }`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { (true | false) }`

default: `false`

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2,3,6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = { (true | false) }`

default: `false`

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*,*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = { (true | false) }`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(argumento)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

```
\spmc \spmc
\spmc(⟨números separados por coma⟩)
\spmc[key = val](⟨números separados por coma⟩)
```

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2,3,6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {⟨true | false⟩}`

default: *false*

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en *true* se imprimirá ‘MCM’ en la salida PDF; si se establece en *false* se imprimirá ‘mcm’ en la salida PDF.

`read-cmd = {⟨terse | clear⟩}`

default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en *terse*, NVDA/MathCAT verbalizará «*m c m*»; si se establece en *clear*, NVDA/MathCAT verbalizará «*mínimo común múltiplo*».

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {⟨true | false⟩}`

default: *false*

Establece si se *imprime* el resultado de calcular el «*mínimo común múltiplo*» del (⟨*argumento*⟩) pasado a `\spmc`. Si se establece en *true*: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcm(2,3,6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre dos, tres y seis es igual a seis*»; si se establece en *false* no se imprimirá en la salida PDF el resultado de calcular el «*mínimo común múltiplo*».

`sample-arg = {⟨true | false⟩}`

default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a `\spmc`, permitiendo usar ejemplos «*no numéricos*» como entrada. Si se establece en *true*: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcm(*m*,*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «*mínimo común múltiplo entre m y n*»; si se establece en *false* el (⟨*argumento*⟩) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc(⟨argumento⟩)` cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (⟨*argumento*⟩). Si se establece en *true* NVDA/MathCAT NO lo verbalizará; si se establece en *false* NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnZ`

`\spnZ` `\spnZ{⟨número entero⟩}`

`\spnZ[⟨key = val⟩]{⟨número entero⟩}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨*número entero*⟩} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las ⟨*meta-keys*⟩ *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(…)’ alrededor del argumento {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, se imprimirán paréntesis redondos alrededor del {⟨*número entero*⟩}; si se establece en *false*, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *false*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos ‘(…)’ alrededor del *argumento* {⟨*número entero*⟩} pasado a `\spnZ`. Si se establece en *true*, NVDA/MathCAT verbalizará la lectura del paréntesis ‘(…)’; si se establece en *false*, NVDA/MathCAT no los verbalizará.

El comando `\spmQ`

`\spmQ` `\spmQ{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

`\spmQ[⟨key = val⟩]{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {⟨*número entero*⟩}, {⟨*numerador*⟩} y {⟨*denominador*⟩}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spmQ{2}{5}{7}` imprimirá ‘ $2\frac{5}{7}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `$\spmQ[join-word=entero]{1}{5}{7}$` imprimirá ‘ $1\frac{5}{7}$ ’, en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `$\spmQ[join-word=enteros]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’, en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `$\spmQ[join-word=y]{2}{5}{7}$` imprimirá ‘ $2\frac{5}{7}$ ’, en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: *true*

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `$\spmQ[print-dfrac=true]{2}{5}{7}$` imprimirá la fracción en tamaño *display* ‘ $2\frac{5}{7}$ ’, en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *true*

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría plana

El paquete `spintent` provee comandos para trabajar con los objetos y medidas de la *geometría plana* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y comparten el mismo módulo `spintent.lua` para construir la verbalización correcta.

- Todos los comandos de esta sección aceptan como puntos solo *letras mayúsculas* como base, con modificadores opcionales de subíndice ‘`_{\dots}`’ o prima ‘`’`’. La única excepción es `\sprecta` cuando el *argumento* es de la forma `{⟨letra⟩}`, que acepta *letras minúsculas*. Los subíndices siempre deben escribirse de la forma correcta usando `_{\dots}` como `A_{\dots}` y nunca como `A_n` sin llaves que funciona solo por coincidencia.

El comando `\spPoint`

`\spPoint` `\spPoint{⟨punto⟩}`
`\spPoint[⟨key = val⟩]{⟨punto⟩}`

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con un modificador opcional de subíndice ‘`_{\dots}`’ o prima ‘`’`’.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `$\spPoint{A}$` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `$\spPoint{A_{1}}$` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

- El comando `\spPoint` solo acepta *un punto*. Para pares o listas de puntos (rectas, rayos, lados, ángulos, arcos, polígonos), véanse los comandos correspondientes de esta sección.

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `$\spPoint{A}$` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`silent-cmd = {⟨true | false⟩}`

default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «punto». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sprecta`

```
\sprecta <{punto, punto}> <{letra}>
\sprecta[<key = val>]{<punto, punto>} <{letra}>
```

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<letra>}` o `{<punto, punto>}`. Si el *argumento* es de la forma `{<punto, punto>}` se aplica `\overleftrightharpoonrightarrow` en la impresión; si el *argumento* no contiene coma y es de la forma `{<letra>}` (mayúscula o minúscula), se imprimirá solo la `{<letra>}` sin aplicar `\overleftrightharpoonrightarrow`.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sprecta{A, B}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprecta{m}` imprimirá m en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para `\sprecta`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}` y el argumento `{<letra>}` cuando está en mayúscula. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{A, B}` se verbaliza «recta A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «recta». El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «recta». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sprayo`

```
\sprayo <{punto, punto}>
\sprayo[<key = val>]{<punto, punto>}
```

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *rayo* o *semirecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sprayo{A, B}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para `\sprayo`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{<punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{A, B}` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves `{ }`.

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

```
read-sty = {<rayo | semirrecta>} default: rayo
```

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{A, B}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{A, B}`, NVDA/MathCAT verbalizará «semirrecta A B».

- La impresión $\overrightarrow{AB}$ en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

```
\spside <{punto, punto}>
\spside[<key = val>]{<punto, punto>}
```

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto>}`. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spside{A, B}` imprimirá $\overline{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$(\spside{A, B})$` se verbaliza «*lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$(\spside[read-sty=lado]{A, B})$`, NVDA/MathCAT verbalizará «*lado A B*»; si se utiliza `$(\spside[read-sty=segmento]{A, B})$`, NVDA/MathCAT verbalizará «*segmento A B*» y si se utiliza `$(\spside[read-sty=trazo]{A, B})$`, NVDA/MathCAT verbalizará «*trazo A B*».

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$(\spside[read-arg={hipotenusa}]{A, B})$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*hipotenusa A B*».

El comando `\spmside`

`\spmside` `\spmside{<punto, punto>}`
 `\spmside[<key = val>]{<punto, punto>}`

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* { `<punto, punto>` }. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ' $\overline{AB}$ '; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* [`<key = val>`], `$(\spmside{A, B})$` imprimirá ' AB ' en la salida PDF y NVDA/MathCAT verbalizará «*medida del lado A B*».

Llaves para `\spmside`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `$(\spmside{A, B})$` se verbaliza «*medida del lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar «*medida del*» seguido por el término establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de «*medida del*» seguido por el término establecido por la llave `read-sty`. Si se establece en `true`, NVDA/MathCAT NO verbalizará «*medida del*» seguido por el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *término* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza `$(\spmside[read-sty=lado]{A, B})$`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `$(\spmside[read-sty=segmento]{A, B})$`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `$(\spmside[read-sty=trazo]{A, B})$`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *término* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `$(\spmside[read-arg={medida cateto}]{A, B})$` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*medida cateto A B*».

`print-m` = { `<true | false>` } default: `false`

Establece si se *imprime* la notación visual 'm' antes del argumento. Si se establece en `true`, `$(\spmside[print-m=true]{A, B})$` imprimirá ' $m\overline{AB}$ ' en la salida PDF, si se establece en `false` imprimirá ' AB ' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

El comando `\spangle`

<code>\spangle</code>	<code>\spangle{⟨punto letra griega número⟩}</code>	<code>\spangle{⟨punto, punto, punto⟩}</code>
	<code>\spangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\angle` o `\rightangle` en la impresión.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo en A»; `\spangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo alfa» y `\spangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo A O B».

Llaves para `\spangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{A, O, B}` se verbaliza «ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\rightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A», `\spangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en *false*, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

El comando `\spmangle`

<code>\spmangle</code>	<code>\spmangle{⟨punto letra griega número⟩}</code>	<code>\spmangle{⟨punto, punto, punto⟩}</code>
	<code>\spmangle[⟨key = val⟩]{⟨punto letra griega número⟩}</code>	<code>\spmangle[⟨key = val⟩]{⟨punto, punto, punto⟩}</code>

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{⟨punto | letra griega | número⟩}` o `{⟨punto, punto, punto⟩}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa» y `\spmangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

Llaves para `\spmangle`

`read-cmd = {⟨school | mathcat⟩}` default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{⟨punto⟩}` o `{⟨punto, punto, punto⟩}`. Si se establece en *school*, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{A, O, B}` se verbaliza «medida del ángulo A O B»; si se establece en *mathcat*, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`silent-cmd = {⟨true | false⟩}` default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del ángulo». Si se establece en *true*, NVDA/MathCAT NO verbalizará el término; si se establece en *false*, NVDA/MathCAT SÍ lo verbalizará.

`recto = {⟨true | false⟩}` default: *false*

Establece si se usará `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en *true*, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A», `\spmangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B».

imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto $A O B$ »; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

```
print-m = {\true | false}
```

default: `false`

Establece si se *imprime* la notación visual ‘m’ antes del argumento. Si se establece en `true`, `\spangle[print-m=true]{A}`, imprimirá ‘ $m\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo $A O B$ ».

El comando `\spang`

```
\spang \spang{\langle grados \rangle}
\spang \spang{\langle grados : minutos \rangle}
\spang \spang{\langle grados : minutos : segundos \rangle}
```

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `\{\langle grados \rangle\}`, `\{\langle grados : minutos \rangle\}` o `\{\langle grados : minutos : segundos \rangle\}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `\spang{45:30:15}` imprimirá ‘ $45^{\circ}30'15''$ ’ en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

```
\sparc \sparc{\langle punto, punto \rangle} \sparc{\langle punto, punto, punto \rangle}
\sparc \sparc[\langle key = val \rangle]{\langle punto, punto \rangle} \sparc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `\{\langle punto, punto \rangle\}` o `\{\langle punto, punto, punto \rangle\}`. Representa un arco determinado por dos o tres puntos aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\sparc{A, B}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «arco $A B$ ».

Llaves para `\sparc`

```
read-cmd = {\langle school | mathcat \rangle}
```

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `\{\langle punto, punto \rangle\}` o `\{\langle punto, punto, punto \rangle\}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{A, B}` se verbaliza «arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {\langle prefijo \rangle}
```

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
silent-cmd = {\langle true | false \rangle}
```

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmarc`

```
\spmarc \spmarc{\langle punto, punto \rangle} \spmarc{\langle punto, punto, punto \rangle}
\spmarc \spmarc[\langle key = val \rangle]{\langle punto, punto \rangle} \spmarc[\langle key = val \rangle]{\langle punto, punto, punto \rangle}
```

El comando `\spmarc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `\{\langle punto, punto \rangle\}` o `\{\langle punto, punto, punto \rangle\}`. Representa la *medida* del arco determinado por dos o tres puntos. La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* ‘ $\widehat{AB}$ ’; `\spmarc` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\spmarc{A, B}` imprimirá ‘ $m\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del arco $A B$ ».

Llaves para `\spmarc`

```
read-cmd = {\langle school | mathcat \rangle}
```

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `\{\langle punto, punto \rangle\}` o `\{\langle punto, punto, punto \rangle\}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{A, B}` se verbaliza «medida del arco $A B$ »; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {\langle prefijo \rangle}
```

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del arco». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
silent-cmd = {\langle true | false \rangle}
```

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spPoly`

```
\spPoly {<punto, punto, punto, ...>}
\spPoly[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<punto, punto, punto, ...>}`, una lista de al menos tres `{<puntos>}` separados por comas. El tipo de *símbolo* en la impresión y la verbalización se determinan acorde al número de puntos en el argumento: Si son tres puntos se aplica `\triangle` y añade el término «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el término «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spPoly{A, B, C}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\spPoly{A, B, C, D}` imprimirá ‘ $\square ABCD$ ’ y verbalizará «cuadrado A B C D»; `\spPoly{A, B, C, D, E}` imprimirá ‘ $ABCDE$ ’ y verbalizará «A B C D E».

Llaves para `\spPoly`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto, punto, punto, ...>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{A, B, C}` se verbaliza «triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el argumento o los términos «triángulo» y «cuadrado» establecido por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
print-sym = {<true | false>} default: true
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión añadiendo el término correspondiente a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spPoly[print-sym=true]{A, B, C}` imprimirá ‘ $\triangle ABC$ ’ en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; si se establece en `false`, imprimirá ‘ ABC ’ en la salida PDF y NVDA/MathCAT omitirá el término «triángulo», verbalizando solo «A B C».

```
silent-cmd = {<true | false>} default: false
```

Alias de `print-sym=false`, incluido por consistencia con el resto de los comandos de geometría plana.

El comando `\spAfig`

```
\spAfig {<punto, punto, punto, ...>}
\spAfig(<número | letra | letra_{número}>)
\spAfig[<key = val>]{<punto, punto, punto, ...>}
```

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* `(<punto, punto, punto, ...>)` (al menos tres); un *número o letra sueltos* `(<1>)`, `(<n>)`; o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, `(<F_1>)`.

Si no se proporciona el *argumento opcional* `[<key = val>]` ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ‘ $\mathcal{A}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «área». Con vértices, `\spAfig(A, B, C)` imprimirá ‘ $\mathcal{A}_{\triangle ABC}$ ’ y verbalizará «área del triángulo A B C». Con un número o letra sueltos, `\spAfig(1)` imprimirá ‘ $\mathcal{A}_1$ ’ y verbalizará «área uno»; `\spAfig(n)` imprimirá ‘ $\mathcal{A}_n$ ’ y verbalizará «área n». Con una letra de la tabla, `\spAfig(F)` imprimirá ‘ $\mathcal{A}_F$ ’ y verbalizará «área de la figura»; `\spAfig(F_{1})` imprimirá ‘ $\mathcal{A}_{F_1}$ ’ y verbalizará «área de la figura uno».

Llaves para `\spAfig`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «área del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «área». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
print-sym = {<auto | triangle | square | none>} default: auto
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión. Con vértices, `auto` infiere el símbolo según el número de puntos (tres o cuatro); `none` lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ‘ $\mathcal{A}_{ABC}$ ’ y verbalizará «área A B C».

Forzar `triangle/square` exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, `auto` y `none` se comportan igual (sin símbolo); `triangle/square` anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\Delta_1}$ ' y verbalizará «área del triángulo uno». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = {⟨true | false⟩}` default: false

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en `true`, `\spAfig[read-sub=true](1)` verbaliza «área sub uno»; si se establece en `false`, verbaliza «área uno». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {⟨nombre⟩}` default: no usado

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{A}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «área del trapezio A B C D».

El comando `\spPfig`

`\spPfig` `\spPfig`
`\spPfig(⟨punto, punto, punto, ...⟩)`
`\spPfig(⟨número | letra | letra_{número}⟩)`
`\spPfig[⟨key = val⟩](⟨punto, punto, punto, ...⟩)`

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`key = val`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá ' $\mathcal{P}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro». Con vértices, `\spPfig(A, B, C)` imprimirá ' $\mathcal{P}_{\Delta ABC}$ ' y verbalizará «perímetro del triángulo A B C». Con un número o letra sueltos, `\spPfig(2)` imprimirá ' $\mathcal{P}_2$ ' y verbalizará «perímetro dos». Con una letra de la tabla, `\spPfig(P_{\{2\}})` imprimirá ' $\mathcal{P}_{P_2}$ ' y verbalizará «perímetro del polígono dos».

Llaves para `\spPfig`

`read-cmd = {⟨school | mathcat⟩}` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «perímetro del triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «perímetro». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {⟨auto | triangle | square | none⟩}` default: auto

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

`read-sub = {⟨true | false⟩}` default: false

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

`silent-cmd = {⟨true | false⟩}` default: false

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {⟨nombre⟩}` default: no usado

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spPfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\mathcal{P}_{\square ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «perímetro del trapezio A B C D».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-08-27 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C	
Document class:	
article	1
book	1
letter	1
report	1
Commands provide by spintent :	
\setspintent	3
\spAfig	22
\spPfig	23
\spPoint	17
\spPoly	22
\spangle	20
\spang	21
\sparc	21
\spdate	11
\spdiv	13
\spdsym	12
\spmQ	16
\spmangle	20
\spmarc	21
\spmcd	15
\spmcm	15
\spmoney	6
\spmside	19
\spmsym	12
\spmul	13
\spnD	14
\spnM	14
\spnZ	16
\spnewunit	6
\spnum	3
\sprayo	18
\spread	11
\sprecta	18
\spshort	7
\spside	18
\spsiglo	11
\sptime	11
\spunitalias	6
\spunit	4
\spusep	12
K	
Keys for \spAfig provide by spintent :	
prefix	22
print-sym	22
read-arg	23
read-cmd	22
read-sub	23
silent-cmd	23
Keys for \spangle provide by spintent :	
prefix	20
read-cmd	20
recto	20
silent-cmd	20
Keys for \sparc provide by spintent :	
prefix	21
read-cmd	21
silent-cmd	21
Keys for \spdiv provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spdsym provide by spintent :	
prefix	12
read-sym	12
set-sym	12
suffix	12
Keys for \spmangle provide by spintent :	
prefix	20
print-m	21
read-cmd	20
recto	20
silent-cmd	20
Keys for \spmarc provide by spintent :	
prefix	21
read-cmd	21
silent-cmd	21
Keys for \spmcd provide by spintent :	
prefix	15
read-cmd	15
result	15
sample-arg	15
silent-cmd	15
upper-case	15
Keys for \spmcm provide by spintent :	
prefix	16
read-cmd	16
result	16
sample-arg	16
silent-cmd	16
upper-case	16
Keys for \spmoney provide by spintent :	
currency	7
dolar-math	7
sub-units	7
Keys for \spmQ provide by spintent :	
join-word	17
print-dfrac	17
read-parens	17
use-spnZ	17
wrap-parens	17
Keys for \spmside provide by spintent :	
prefix	19
print-m	19
read-arg	19
read-cmd	19
read-sty	19
silent-cmd	19
Keys for \spmsym provide by spintent :	
not-print	13
prefix	13
read-sym	13
set-sym	13
suffix	13
Keys for \spmul provide by spintent :	
read-parens	13
wrap-parens	13
Keys for \spnD provide by spintent :	
prefix	14
result-num	14

result	14	notation-sym	13
sample-arg	14	read-period-sep	13
silent-cmd	14	read-sign	13
Keys for \spnM provide by spintent :		read-sym	13
prefix	14	set-sym	13
result-num	14	Keys for \spdsym provide by spintent :	
result	14	read-sym	12
sample-arg	14	Keys for \spmcd provide by spintent :	
silent-cmd	15	read-cmd	15
Keys for \spnum provide by spintent :		Keys for \spmcm provide by spintent :	
cifras-sep	3	read-cmd	16
notation-sym	4	Keys for \spmoney provide by spintent :	
read-period-sep	3	cifras-sep	7
read-sign	3	Keys for \spmQ provide by spintent :	
Keys for \spnZ provide by spintent :		wrap-parens	17
read-parens	16	Keys for \spmsym provide by spintent :	
wrap-parens	16	read-sym	13
Keys for \spPfig provide by spintent :		Keys for \spm\l provide by spintent :	
prefix	23	cifras-sep	13
print-sym	23	notation-sym	13
read-arg	23	read-period-sep	13
read-cmd	23	read-sign	13
read-sub	23	read-sym	13
silent-cmd	23	set-sym	13
Keys for \spPoint provide by spintent :		Keys for \spnZ provide by spintent :	
prefix	17	cifras-sep	16
read-cmd	17	read-sign	16
silent-cmd	17	Keys for \spPoly provide by spintent :	
Keys for \spPoly provide by spintent :		print-sym	22
prefix	22	Keys for \spside provide by spintent :	
print-sym	22	read-sty	19
read-cmd	22	Keys for \spunit provide by spintent :	
silent-cmd	22	cifras-sep	4
Keys for \sprayo provide by spintent :		notation-sym	4
prefix	18	read-period-sep	4
read-cmd	18	Keys for \spusep provide by spintent :	
read-sty	18	silent	12
silent-cmd	18	size	12
Keys for \sprecta provide by spintent :			
prefix	18		
read-cmd	18		
silent-cmd	18		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
prefix	19		
read-arg	19		
read-cmd	19		
read-sty	19		
silent-cmd	19		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
print-cdot	4		
read-cdot	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		
size	12		
Keys for \spddiv provide by spintent :			
cifras-sep	13		

M

Lua module provide by **spintent**:

spintent.lua 6, 7, 9, 10, 17

P

Packages :

babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

All variables and functions defined in this package are “private” and are NOT intended to work or be used by another package or module.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a T_EXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

- Functions of the form `\__spintent_luafun_some:nn` or variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

11.2 Declaration of the package

First we will make sure we have a minimum (super updated) version of L^AT_EX to work correctly.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8   {
9     spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10    later.~Update~your~TeX~distribution.
11  }
12 }
```

Now declare the `spintent` package.

```
13 \ProvidesExplPackage {spintent} {2026-08-27} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n = true,
19 }
20 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, LuaT_EX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
21 \IfDocumentMetadataTF
22 {
23   \sys_if_engine luatex:TF
24   {
25     \msg_new:nnn { spintent } { package-not-load }
26     {
27       The~'#1'~package~will~be~loaded~as~a~dependency.
28     }
29     \IfPackageLoadedF { unicode-math }
30     {
31       \IfPackageLoadedF { lua-unicode-math }
32       {
33         \msg_info:nnn { spintent } { package-not-load } { unicode-math }
34         \RequirePackage{unicode-math}
35       }
36     }
37     \msg_new:nnn { spintent } { env-success }
38     { Everything~looks~correct,~spintent~will~make~the~intent! }
39     \msg_info:nn { spintent } { env-success }
40   }
41   {
42     \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
43     { The~spintent~package~requires~LuaLaTeX. }
44     { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
45     \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
```



```

46     }
47   }
48   {
49     \msg_new:nnnn { spintent } { fatal-metadata-missing }
50     { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
51     {
52       The~spintent~package~requires~tagged~PDF~active.\\
53       You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
54     }
55     \msg_fatal:nn { spintent } { fatal-metadata-missing }
56   }

```

11.3 Some kernel variants

Non-standard kernel variants.

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne
57 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
58 \cs_generate_variant:Nn \str_if_eq:nnF { V }
59 \cs_generate_variant:Nn \str_uppercase:n { V }
60 \cs_generate_variant:Nn \str_lowercase:n { V }
61 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.4 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:V` used by `\spunit` (§11.12), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.14), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.27) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.11) defined in it.

```

62 \str_new:N \l__spintent_luaset_cal_fam_ok_str
63 \str_new:N \l__spintent_luaset_sym_fam_ok_str
64 \lua_load_module:n { spintent.lua }
65 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
66 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
67 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { VV, Vn }
68 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

69 \msg_new:nnn { spintent } { mathcal-fallback }
70   {
71     Using~classic~\string\mathcal~as~fallback~(font~not~found~or~
72     no~free~math~family~available).~\msg_line_context:
73   }
74 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
75   {
76     Using~\string\rightangle~as~fallback~for~the~right-angle~
77     glyph.~\msg_line_context:
78   }
79 \cs_new_protected:Nn \__spintent_mathcal:n
80   {
81     \bool_if:NTF \l__spintent_mathcal_bool
82     {
83       \str_if_eq:VnTF \l__spintent_luaset_cal_fam_ok_str { true }
84       { \__spintent_luafun_classic_mathcal:n {#1} }
85       {
86         \msg_warning:nn { spintent } { mathcal-fallback }
87         \mathcal {#1}
88       }
89     }
90     { \mathcal {#1} }
91   }
92 \cs_new_protected:Nn \__spintent_right_angle_sqr:
93   {
94     \cs_if_exist:NTF \rightanglesqr
95     { \rightanglesqr }
96     {
97       \str_if_eq:VnTF \l__spintent_luaset_sym_fam_ok_str { true }
98       { \__spintent_luafun_right_angle_sqr: }
99       {
100         \msg_warning:nn { spintent } { right-angle-sqr-fallback }
101         \rightangle
102       }
103     }
104   }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.5 Internal copy of the unicode invisible separator

`\__spintent_invisible_sep:`
`\c__spintent_invisible_sep_tl`

The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator U-2063 that uses `\luamml_annotate:en` from the package `luamml[2]` and let `<mo>...</mo>` in the `MathML` structure embedded in the PDF output.

```
105 \cs_new_protected:Nn \__spintent_invisible_sep:
106 {
107   \luamml_annotate:en
108   {
109     core = {[0] = 'mo', intent=':silent', "^^^2063"}
110   }
111   {
112     \latelua{}
113   }
114 }
```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator U-2063 using the primitive `\Umathchardef` from `LuaTeX` which leaves `<mi>...</mi>` in the structure `MathML` embedded in the output PDF.

```
115 \hook_gput_code:nnn { begindocument } { \spintent }
116 {
117   \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
118 }
```

- Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure `MathML`, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure `MathML` when we use `\MathMLintent{<text>}`.

(End of definition for `\__spintent_invisible_sep:` and `\c__spintent_invisible_sep_tl`.)

11.6 Normalize argument and affixes

We normalize the `{<argument>}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N`

A recursive helper to collapse multiple hyphens into a single one.

```
119 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
120 {
121   \tl_if_in:NnT #1 { -- }
122   {
123     \tl_replace_all:Nnn #1 { -- } { - }
124     \__spintent_collapse_hyphens:N #1
125   }
126 }
```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn`
`\__spintent_sanitize_affix:cn`

The function `\__spintent_sanitize_affix:Nn` checks if the `{<argument>}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`␣`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string `foo-bar-baz` inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```
127 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
128 {
129   \tl_if_blank:nTF {#2}
130   { \tl_clear:N #1 }
131   {
132     \tl_set:Ne #1 {#2}
133     \tl_trim_spaces:N #1
134     \tl_replace_all:Nnn #1 { ~ } { - }
135     \__spintent_collapse_hyphens:N #1
136     \tl_put_left:Nn #1 { \q_mark }
137     \tl_put_right:Nn #1 { \q_stop }
138     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
139     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
140     \tl_remove_all:Nn #1 { \q_mark }
```

```

141         \tl_remove_all:Nn #1 { \q_stop }
142     }
143 }
144 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }

```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.7 The command `\setspintent`

The `\setspintent` command is used to configure *⟨keys⟩* “globally” for the utilities provided by `spintent`.

11.7.1 Internal variables and error messages for `\setspintent`

```

\__spintent_setspintent_input_arg_cmd_tl
unknown-command-setup

```

The variable `\__spintent_setspintent_input_arg_cmd_tl` will *store* the command or command name passed to the “first” `{⟨argument⟩} #1` of the command `\setspintent`.

```

145 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl

```

Error message if the `{⟨argument⟩} #1` passed to `\setspintent` is a “non-existent” command.

```

146 \msg_new:nnnn { spintent } { unknown-command-setup }
147 {
148     Cannot~configure~unknown~command~'\exp_not:c{#1}'.
149 }
150 {
151     You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
152     command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
153 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as `{⟨argument⟩}` exists in `#1` and set the *⟨keys⟩* passed as `{⟨argument⟩}` in `#2` to the correct path; otherwise, it will return an error.

```

154 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
155 {
156     \cs_if_exist:cTF { #1 }
157     {
158         \keys_set:nn { spintent / #1 } { #2 }
159     }
160     {
161         \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
162     }
163 }
164 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in `#1` begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

165 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
166 {
167     \tl_if_single:nTF { #1 }
168     {
169         \token_if_cs:NTF #1
170         {
171             \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
172         }
173         {
174             \__spintent_setspintent_set_keys:nn { #1 } { #2 }
175         }
176     }
177     {
178         \__spintent_setspintent_set_keys:nn { #1 } { #2 }
179     }
180 }
181 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for `\__spintent_setspintent_parse_args:nn`.)

`\setspintent` Now we define the user command `\setspintent`.

```

182 \NewDocumentCommand \setspintent { m m }
183 {
184   \tl_if_blank:nTF { #1 }
185   {
186     \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
187   }
188   {
189     \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
190     \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
191     \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
192   }
193 }

```

(End of definition for `\setspintent`. This function is documented on page 3.)

11.8 Handling unknown keys

`\__spintent_command_name_str`
`unknown-choice`
`cmd-key-unknown`
`cmd-key-value-unknown`

We define the variable `\__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by `<keys>` that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the `<key>` `unknown` in commands that use `[<key = val>]`.

```

194 \str_new:N \__spintent_command_name_str
195 \msg_new:nnn { spintent } { unknown-choice }
196 {
197   The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
198 }
199 \msg_new:nnnn { spintent } { cmd-key-unknown }
200 {
201   The~key~'#1'~is~unknown~by~command~
202   \c_backslash_str \__spintent_command_name_str \c_space_tl
203   and~is~being~ignored~\msg_line_context:.
204 }
205 {
206   The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
207   does~not~have~a~key~called~'#1'.\\
208   Check~that~you~have~spelled~the~key~name~correctly.
209 }
210 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
211 {
212   The~key~'#1=#2'~is~unknown~by~command~
213   \c_backslash_str \__spintent_command_name_str \c_space_tl
214   and~is~being~ignored~\msg_line_context:.
215 }
216 {
217   The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
218   does~not~have~a~key~called~'#1'.\\
219   Check~that~you~have~spelled~the~key~name~correctly.
220 }

```

(End of definition for `\__spintent_command_name_str` and others.)

`\__spintent_unknown_keys:n`
`\__spintent_unknown_keys:nn`

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the `<key>` `unknown`.

```

221 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
222 {
223   \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
224 }
225 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
226 {
227   \tl_if_blank:nTF {#2}
228   { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
229   { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
230 }

```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn`.)

11.9 Definition of core base variables

`\__spintent_luafun_clean_split_and_set:n`
`\__spintent_luafun_clean_split_and_set:V`

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the `{<argument>}` passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates

signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “*units*”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`)

```
\__spintent_luaset_sign_tl
\__spintent_luaset_part_int_tl
\__spintent_luaset_dec_sep_tl
\__spintent_luaset_part_dec_tl
\__spintent_luaset_part_period_tl
\__spintent_luaset_has_integer_str
\__spintent_luaset_has_natural_str
```

The variable `\__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘−’ sign if present. The variable `\__spintent_luaset_part_int_tl` stores the raw digit sequence of the “*integer part*”. The variable `\__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\__spintent_luaset_part_dec_tl` stores the raw digits of the “*finite decimal part*” that follow the separator, and the variable `\__spintent_luaset_part_period_tl` stores the digits of the “*infinite periodic decimal part*” that follow a semicolon ‘;’.

The variable `\__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “*integer*”. The variable `\__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “*natural number*”.

```
231 \tl_new:N \__spintent_luaset_sign_tl
232 \tl_new:N \__spintent_luaset_part_int_tl
233 \tl_new:N \__spintent_luaset_dec_sep_tl
234 \tl_new:N \__spintent_luaset_part_dec_tl
235 \tl_new:N \__spintent_luaset_part_period_tl
236 \str_new:N \__spintent_luaset_has_integer_str
237 \str_new:N \__spintent_luaset_has_natural_str
```

(End of definition for `\__spintent_luaset_sign_tl` and others.)

```
\__spintent_luaset_only_part_int_str
\__spintent_luaset_only_part_dec_str
\__spintent_luaset_only_part_period_str
\__spintent_luaset_format_part_int_str
\__spintent_luaset_format_part_dec_str
```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\__spintent_luaset_only_part_int_str` for the “*integer part*”, in `\__spintent_luaset_only_part_dec_str` for the “*finite decimal part*”, and in `\__spintent_luaset_only_part_period_str` for the “*infinite periodic decimal part*”.

Formatted versions (`cifras-sep=true`) are stored in `\__spintent_luaset_format_part_int_str` for the “*integer part*” and `\__spintent_luaset_format_part_dec_str` for the “*finite decimal part*”.

```
238 \str_new:N \__spintent_luaset_only_part_int_str
239 \str_new:N \__spintent_luaset_only_part_dec_str
240 \str_new:N \__spintent_luaset_only_part_period_str
241 \str_new:N \__spintent_luaset_format_part_int_str
242 \str_new:N \__spintent_luaset_format_part_dec_str
```

(End of definition for `\__spintent_luaset_only_part_int_str` and others.)

```
\__spintent_luaset_millions_str
\__spintent_luaset_decimal_period_str
\__spintent_luaset_not_decimal_period_str
\__spintent_luaset_not_decimal_not_period_str
\__spintent_luaset_dec_is_all_zeros_str
\__spintent_luaset_has_exponent_str
\__spintent_luaset_exponent_tl
```

The flag variable `\__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\__spintent_luaset_decimal_period_str`, `\__spintent_luaset_not_decimal_period_str`, and `\__spintent_luaset_not_decimal_not_period_str`. The flag variable `\__spintent_luaset_dec_is_all_zeros_str` monitors whether the “*finite decimal part*” is completely zero (useful for “*pure periodic decimals*”).

The flag variable `\__spintent_luaset_has_exponent_str` indicates whether active “*scientific notation*” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\__spintent_luaset_exponent_tl`.

```
243 \str_new:N \__spintent_luaset_millions_str
244 \str_new:N \__spintent_luaset_decimal_period_str
245 \str_new:N \__spintent_luaset_not_decimal_period_str
246 \str_new:N \__spintent_luaset_not_decimal_not_period_str
247 \str_new:N \__spintent_luaset_dec_is_all_zeros_str
248 \str_new:N \__spintent_luaset_has_exponent_str
249 \tl_new:N \__spintent_luaset_exponent_tl
```

(End of definition for `\__spintent_luaset_millions_str` and others.)

```
\__spintent_luaset_has_units_str
\__spintent_luaset_status_str
\__spintent_luaset_arg_above_tl
\__spintent_luaset_arg_below_tl
\__spintent_luaset_denom_has_number_str
```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.11) and `\spunit` (§11.12) (as well as others), it determines the following unit-related variables after processing the {*arguments*} via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “*units*”. It sets the flag variable `\__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\__spintent_luaset_arg_above_tl` and `\__spintent_luaset_arg_below_tl`. Finally, it analyzes `\__spintent_luaset_arg_below_tl`;

if the denominator starts with a number, it sets `\l__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```
250 \str_new:N \l__spintent_luaset_has_units_str
251 \str_new:N \l__spintent_luaset_status_str
252 \tl_new:N \l__spintent_luaset_arg_above_tl
253 \tl_new:N \l__spintent_luaset_arg_below_tl
254 \str_new:N \l__spintent_luaset_denom_has_number_str
```

(End of definition for `\l__spintent_luaset_has_units_str` and others.)

11.10 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```
255 \msg_new:nnnn { spintent } { need-math-mode }
256 {
257   Math~mode~required~for~'#1'~\msg_line_context:.
258 }
259 {
260   Use~inside~$...$~or~in~math~environment.
261 }
262 \msg_new:nnnn { spintent } { need-text-mode }
263 {
264   Text~mode~required~for~'#1'~\msg_line_context:.
265 }
266 {
267   Use~outside~of~$...$~or~math~environment.
268 }
```

11.11 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.11.1 Definition of keys and variables for `\spnum`

The variable `\l__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘−’. If set to `clear`, it will say “positivo” or “negativo” after reading the number; if set to `terse`, it will say “más” or “menos” before reading the number.

The variable `\l__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{(argument)}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\l__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘×’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\l__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘−’ according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```
269 \bool_new:N \l__spintent_spnum_read_terse_bool
270 \str_new:N \l__spintent_spnum_read_period_sep_str
271 \tl_new:N \l__spintent_spnum_notation_sym_tl
272 \str_new:N \l__spintent_spnum_intent_read_sign_str
273 \str_new:N \l__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

cifras-sep	Now we add the keys <code>cifras-sep</code> , <code>read-sign</code> , <code>read-period-sep</code> and <code>notation-sym</code> .
read-sign	274 \keys_define:nn { spintent/spnum }
read-period-sep	275 {
notation-sym	276 cifras-sep .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
	277 cifras-sep .initial:n = true,
	278 cifras-sep .value_required:n = true,
	279 read-sign .choices:nn =
	280 { terse , clear }
	281 {

```

282     \int_case:nn { \l_keys_choice_int }
283     {
284         {1} { \bool_set_true:N \l__spintrent_spnum_read_terse_bool }
285         {2} { \bool_set_false:N \l__spintrent_spnum_read_terse_bool }
286     }
287 },
288 read-sign .initial:n = terse,
289 read-sign .value_required:n = true,
290 read-sign / unknown .code:n =
291     \msg_error:nneee { spintrent } { unknown-choice }
292     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
293 read-period-sep .choices:nn =
294     { coma , punto }
295     {
296         \int_case:nn { \l_keys_choice_int }
297         {
298             {1} { \str_set:Nn \l__spintrent_spnum_read_period_sep_str { coma } }
299             {2} { \str_set:Nn \l__spintrent_spnum_read_period_sep_str { punto } }
300         }
301     },
302 read-period-sep .initial:n = coma,
303 read-period-sep .value_required:n = true,
304 read-period-sep / unknown .code:n =
305     \msg_error:nneee { spintrent } { unknown-choice }
306     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
307 notation-sym .choices:nn =
308     { times , cdot }
309     {
310         \int_case:nn { \l_keys_choice_int }
311         {
312             {1} { \tl_set:Nn \l__spintrent_spnum_notation_sym_tl { \times } }
313             {2} { \tl_set:Nn \l__spintrent_spnum_notation_sym_tl { \cdot } }
314         }
315     },
316 notation-sym .initial:n = cdot,
317 notation-sym .value_required:n = true,
318 notation-sym / unknown .code:n =
319     \msg_error:nneee { spintrent } { unknown-choice }
320     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
321 unknown .code:n =
322     {
323         \str_set:Nn \l__spintrent_command_name_str { spnum }
324         \__spintrent_unknown_keys_cmd:n {#1}
325     },
326 }

```

(End of definition for *cifras-sep* and others.)

11.11.2 Internal functions for \spnum

```

\__spintrent_spnum_render_part:n
\__spintrent_spnum_render_int:
\__spintrent_spnum_render_dec:

```

The function `\__spintrent_spnum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintrent_spnum_render_int:` and `\__spintrent_spnum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key *cifras-sep* is active or not.

```

327 \cs_new_protected:Nn \__spintrent_spnum_render_part:n
328 {
329     \MathMLintent { \str_use:c { l__spintrent_luaset_only_part_#1_str } :number }
330     {
331         \bool_if:NTF \l__spintrent_spnum_cifras_sep_bool
332         { \str_use:c { l__spintrent_luaset_format_part_#1_str } }
333         { \tl_use:c { l__spintrent_luaset_part_#1_tl } }
334     }
335 }
336 \cs_new_protected:Nn \__spintrent_spnum_render_int:
337 {
338     \__spintrent_spnum_render_part:n { int }
339 }
340 \cs_new_protected:Nn \__spintrent_spnum_render_dec:
341 {
342     \__spintrent_spnum_render_part:n { dec }
343 }

```

(End of definition for `\__spintent_snum_render_part:n`, `\__spintent_snum_render_int:`, and `\__spintent_snum_render_dec:`.)

`\__spintent_snum_render_only_period:`
`\__spintent_snum_print_period_bar:`
`\__spintent_snum_render_period_sep:`

The function `\__spintent_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* `:number` passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_snum_print_period_bar:`.

```

344 \cs_new_protected:Nn \__spintent_snum_render_only_period:
345 {
346   \MathMLintent { \l__spintent_luaset_only_part_period_str :number }
347   {
348     \l__spintent_luaset_part_period_tl
349   }
350 }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “infinite periodic decimal part”.

```

351 \cs_new_protected:Nn \__spintent_snum_print_period_bar:
352 {
353   \MathMLintent { _periódico:postfix($dp) }
354   {
355     {
356       \overline
357       {
358         \MathMLarg { dp } { \__spintent_snum_render_only_period: }
359       }
360     }
361   }
362 }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “pure periodic decimal” is passed.

```

363 \cs_new_protected:Nn \__spintent_snum_render_period_sep:
364 {
365   \str_if_eq:VnTF \l__spintent_snum_read_period_sep_str { coma }
366   {
367     \MathMLintent { _coma:prefix($pp) }
368     {
369       \mathord{,}
370       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
371     }
372   }
373   {
374     \MathMLintent { _punto:prefix($pp) }
375     {
376       \mathord{.}
377       \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
378     }
379   }
380 }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:`

The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_dec_sep_tl` and sets the variable `\l__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “coma” or `_punto:prefix($dp)` for reading a “punto” to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

381 \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:
382 {
383   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
384   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
385   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
386   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
387   {
388     \mathord { \l__spintent_luaset_dec_sep_tl }
389     \MathMLarg { dp }
390     {
391       \__spintent_invisible_sep:

```

```

392         \__spintent_snum_render_dec:
393     }
394 }
395 % period part
396 \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
397 {
398     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
399     {
400         \__spintent_invisible_sep:
401         \__spintent_snum_print_period_bar:
402     }
403     {
404         \MathMLintent { _con:prefix($pnum) }
405         {
406             \__spintent_invisible_sep:
407             \MathMLarg { pnum }
408             {
409                 \__spintent_snum_print_period_bar:
410             }
411         }
412     }
413 }
414 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“*coma*” or “*punto*”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

415 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
416 {
417     \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
418     {
419         \__spintent_snum_render_decimal_sep:
420     }
421 }
422 \cs_new_protected:Nn \__spintent_snum_print_part_period:
423 {
424     \tl_if_empty:NF \l__spintent_luaset_part_period_tl
425     {
426         \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
427         { \__spintent_snum_render_period_sep: }
428         {
429             \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
430             {
431                 \__spintent_snum_render_period_sep:
432             }
433         }
434     }
435 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

436 \cs_new_protected:Nn \__spintent_snum_print_integer:
437 {

```

```

438 \str_if_eq:VnTF \l__spinttent_luaset_not_decimal_not_period_str { true }
439 { \l__spinttent_spnum_render_int: }
440 {
441   \MathMLintent { \l__spinttent_luaset_only_part_int_str :number:prefix($next) }
442   {
443     \bool_if:NTF \l__spinttent_spnum_cifras_sep_bool
444     { \l__spinttent_luaset_format_part_int_str }
445     { \l__spinttent_luaset_part_int_tl }
446     \MathMLarg { next }
447     {
448       \tl_if_empty:NTF \l__spinttent_luaset_part_dec_tl
449       { \l__spinttent_spnum_print_part_period: }
450       { \l__spinttent_spnum_print_part_dec: }
451     }
452   }
453 }
454 }
455 \cs_new_protected:Nn \l__spinttent_spnum_print_num_start:
456 {
457   \tl_if_empty:NTF \l__spinttent_luaset_sign_tl
458   {
459     \l__spinttent_invisible_sep: % need
460     \l__spinttent_spnum_print_integer:
461   }
462   {
463     \bool_if:NTF \l__spinttent_spnum_read_terse_bool
464     {
465       \str_if_eq:VnTF \l__spinttent_luaset_sign_tl { + }
466       { \str_set:Nn \l__spinttent_spnum_intent_read_sign_str { _más:prefix($n) } }
467       { \str_set:Nn \l__spinttent_spnum_intent_read_sign_str { _menos:prefix($n) } }
468     }
469     {
470       \str_if_eq:VnTF \l__spinttent_luaset_sign_tl { + }
471       { \str_set:Nn \l__spinttent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
472       { \str_set:Nn \l__spinttent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
473     }
474     \l__spinttent_invisible_sep: % need
475     \MathMLintent { \l__spinttent_spnum_intent_read_sign_str }
476     {
477       \l__spinttent_luaset_sign_tl
478       \MathMLarg { n } { \l__spinttent_spnum_print_integer: }
479     }
480   }
481 }
482 \cs_new_protected:Nn \l__spinttent_spnum_print_number_core:
483 {
484   \l__spinttent_spnum_print_num_start:
485   \str_if_eq:VnT \l__spinttent_luaset_has_exponent_str { true }
486   {
487     \MathMLintent { _por } { \l__spinttent_spnum_notation_sym_tl }
488     10^{\l__spinttent_luaset_exponent_tl }
489   }
490 }

```

(End of definition for `\l__spinttent_spnum_print_integer:`, `\l__spinttent_spnum_print_num_start:`, and `\l__spinttent_spnum_print_number_core:`.)

11.11.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “units” in the $\langle argument \rangle$ passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error” is raised if the provided $\langle argument \rangle$ contains no numerical digits.

```

491 \msg_new:nnnn { spinttent } { spnum-has-units }
492 {
493   The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('1').
494 }
495 {
496   Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
497 }
498 \msg_new:nnnn { spinttent } { spnum-no-digits }
499 {
500   The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.

```

```

501   }
502   {
503     You~provided~'#1',~which~contains~no~digits.
504   }

```

(End of definition for `spnum-has-units` and `spnum-no-digits`.)

11.11.4 Implementation of the `\spnum` command

`\spnum` The public user command `\spnum` accepts an optional [`<key = val>`] argument in ‘#1’ and a mandatory `<rational number>` in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

505 \NewDocumentCommand \spnum { O{} m }
506 {
507   \mode_if_math:TF
508   {
509     \group_begin:
510     \keys_set:nn { spintent/spnum } { #1 }
511     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
512     \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
513     { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
514     {
515       \tl_if_empty:NTF \__spintent_luaset_part_int_tl
516       { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
517       { \__spintent_spnum_print_number_core: }
518     }
519     \group_end:
520   }
521   { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
522 }

```

(End of definition for `\spnum`. This function is documented on page 3.)

11.12 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the `<argument>` to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the `<argument>`.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.12.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

523 \str_new:N \__spintent_spunit_luaset_canonical_str
524 \str_new:N \__spintent_spunit_luaset_lookup_status_str
525 \str_new:N \__spintent_spunit_luaset_read_str
526 \str_new:N \__spintent_spunit_luaset_is_compact_str
527 \str_new:N \__spintent_spunit_luaset_compact_exp_str
528 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
529 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

These variables are used to process and format the “unit”. The sequences `\__spintent_spunit_mult_seq` and `\__spintent_spunit_exp_seq` split compound “units” at multiplication ‘*’ or exponent ‘^’ boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

530 \tl_new:N \__spintent_spunit_exponent_tl
531 \tl_new:N \__spintent_spunit_unit_name_tl
532 \str_new:N \__spintent_spunit_read_slash_str
533 \seq_new:N \__spintent_spunit_exp_seq
534 \seq_new:N \__spintent_spunit_mult_seq
535 \bool_new:N \__spintent_spunit_is_mult_bool
536 \bool_new:N \__spintent_spunit_read_cdot_bool

```

(End of definition for `\__spintent_spunit_exponent_tl` and others.)

11.12.2 Definition of error messages

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

We define three error messages for the `\spunit` command. The first “error” triggers if the “unit expression” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “units” are found in the $\{\langle argument \rangle\}$, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

537 \msg_new:nnnn { spintent } { spunit-invalid-unit }
538 {
539   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
540 }
541 {
542   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
543 }
544 \msg_new:nnnn { spintent } { spunit-no-units }
545 {
546   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
547 }
548 {
549   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
550 }
551 \msg_new:nnnn { spintent } { inline-numeric-denominator }
552 {
553   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
554 }
555 {
556   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
557   does~not~allow~numbers~after~the~slash~(/).
558 }
```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.12.3 Definition of keys for `\spunit`

read-cdot
read-slash
cifras-sep
read-period-sep
notation-sym

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

559 \keys_define:nn { spintent/spunit }
560 {
561   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
562   print-cdot      .initial:n   = false,
563   print-cdot      .value_required:n = true,
564   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
565   read-cdot       .initial:n   = false,
566   read-cdot       .value_required:n = true,
567   read-slash      .choices:nn =
568     { por , partido-por }
569     {
570       \int_case:nn { \l__keys_choice_int }
571       {
572         {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
573         {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
574       }
575     },
576   read-slash      .initial:n   = por,
577   read-slash      .value_required:n = true,
578   read-slash / unknown .code:n =
579     \msg_error:nneee { spintent } { unknown-choice }
580     { read-slash } { por , ~partido-por } { \exp_not:n {#1} },
581   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
582   cifras-sep      .value_required:n = true,
583   read-period-sep .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
584   read-period-sep .value_required:n = true,
585   notation-sym    .meta:nn = { spintent/spnum } { notation-sym = {#1} },
586   notation-sym    .value_required:n = true,
587   unknown         .code:n =
588     {
589       \str_set:Nn \l__spintent_command_name_str { spunit }
590       \__spintent_unknown_keys_cmd:n {#1}
591     },
592 }
```

(End of definition for `read-cdot` and others.)

11.12.4 Internal functions for \spunit

__spinttent_spunit_format_canonical:
 __spinttent_spunit_process_base_exp:n

The function `\__spinttent_spunit_format_canonical:` handles the visual formatting of the “*unit symbol*”. It specifically checks for the *liter symbol* ‘*ℓ*’ to print it as `\ell`, and formats all other valid “*units*” in standard upright text using `\mathrm`.

The function `\__spinttent_spunit_process_base_exp:n` evaluates a “*single unit*” along with its exponent. It first looks up the unit ‘*#1*’ in the dictionary. If the “*unit*” is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot ‘`\cdot`’ intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the ‘`^`’ symbol).

```

593 \cs_new_protected:Nn \__spinttent_spunit_format_canonical:
594 {
595   \str_case:VnF \l__spinttent_spunit_luaset_canonical_str
596   {
597     { ℓ } { \ell }
598   }
599   { \mathrm { \l__spinttent_spunit_luaset_canonical_str } }
600 }
601 \cs_new_protected:Nn \__spinttent_spunit_process_base_exp:n
602 {
603   \tl_clear:N \l__spinttent_spunit_exponent_tl
604   \__spinttent_luafun_spunit_lookup_alias:n { #1 }
605   \str_if_eq:VnTF \l__spinttent_spunit_luaset_lookup_status_str { notfound }
606   {
607     \str_set:Nn \l__spinttent_luaset_status_str { invalid }
608   }
609   {
610     \bool_if:NF \l__spinttent_spunit_is_mult_bool
611     {
612       \bool_if:NTF \l__spinttent_spunit_print_cdod_bool
613       {
614         \bool_if:NTF \l__spinttent_spunit_read_cdod_bool
615         { \MathMLintent { por } { \cdot } }
616         { \MathMLintent { :silent } { \cdot } }
617       }
618       {
619         \bool_if:NTF \l__spinttent_spunit_read_cdod_bool
620         { \MathMLintent { por } { \, } }
621         { \, }
622       }
623     }
624     \str_if_eq:VnTF \l__spinttent_spunit_luaset_is_compact_str { true }
625     {
626       {
627         \MathMLintent { \l__spinttent_spunit_luaset_read_str }
628         {
629           \__spinttent_spunit_format_canonical:
630         }
631       }
632       ^ { \l__spinttent_spunit_luaset_compact_exp_str }
633     }
634     {
635       \seq_set_split:Nnn \l__spinttent_spunit_exp_seq { ^ } { #1 }
636       \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_unit_name_tl
637       \seq_if_empty:NF \l__spinttent_spunit_exp_seq
638       {
639         \seq_pop_left:NN \l__spinttent_spunit_exp_seq \l__spinttent_spunit_exponent_tl
640       }
641       \MathMLintent { \l__spinttent_spunit_luaset_read_str }
642       {
643         \__spinttent_spunit_format_canonical:
644       }
645       \tl_if_empty:NF \l__spinttent_spunit_exponent_tl
646       {
647         ^ { \l__spinttent_spunit_exponent_tl }
648       }
649     }
650     \bool_set_false:N \l__spinttent_spunit_is_mult_bool
651   }
652 }

```

(End of definition for `\__spinttent_spunit_format_canonical:` and `\__spinttent_spunit_process_base_exp:n`)

`\__spintent_spunit_process_mult:n`
`\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{\langle argument \rangle\}$ at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

653 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
654 {
655   \bool_set_true:N \__spintent_spunit_is_mult_bool
656   \seq_set_split:Nnn \__spintent_spunit_mult_seq { * } { #1 }
657   \seq_map_function:NN \__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
658 }
659 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

660 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
661 {
662   \tl_if_empty:NF \__spintent_luaset_arg_above_tl
663   {
664     \__spintent_spunit_process_mult:V \__spintent_luaset_arg_above_tl
665     \tl_if_empty:NF \__spintent_luaset_arg_below_tl
666     {
667       \MathMLintent { \__spintent_spunit_read_slash_str } { / }
668       \__spintent_spunit_process_mult:V \__spintent_luaset_arg_below_tl
669     }
670   }
671 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the $\{\langle argument \rangle\}$ has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

672 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
673 {
674   \str_if_eq:VnTF \__spintent_luaset_has_units_str { false }
675   { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
676   {
677     \str_if_eq:VnTF \__spintent_luaset_status_str { valid }
678     {
679       \str_if_eq:VnTF
680       \__spintent_luaset_denom_has_number_str { true }
681       {
682         \msg_error:nnn { spintent }
683         { inline-numeric-denominator } { #1 }
684       }
685       {
686         \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
687         \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
688         {
689           \tl_if_empty:NF \__spintent_luaset_part_int_tl
690           {
691             \__spintent_spnum_print_number_core:
692           }
693           \MathMLintent { \__spintent_spunit_luaset_read_str }
694           {
695             \mathord
696             {
697               \str_case:Vn \__spintent_spunit_luaset_canonical_str
698               {
699                 { " } { \prime \! \prime }
700                 { ' } { \prime
701                 { ° } { °
702               }
703             }

```

```

704         }
705     }
706     {
707         \tl_if_empty:NF \l__spintent_lua_set_part_int_tl
708         {
709             \__spintent_spnum_print_number_core: \,
710         }
711         \__spintent_print_spunit_hierarchy:
712     }
713 }
714 }
715 { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
716 }
717 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`.)

11.12.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`(key = val)`] argument in ‘#1’ and a mandatory `{(unit expression)}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{(argument)}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

718 \NewDocumentCommand \spunit { O{} m }
719 {
720     \mode_if_math:TF
721     {
722         \group_begin:
723         \keys_set:nn { spintent/spunit } { #1 }
724         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
725         \__spintent_spunit_safe_exec:n { #2 }
726         \group_end:
727     }
728     { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
729 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.13 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to TeX using the `\l__spintent_spunit_lua_set_status_s` variable. If a collision is detected or the `{(argument)}` is invalid, the package throws the corresponding “error” message.

11.13.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

730 \msg_new:nnnn { spintent } { spnewunit-duplicate }
731 {
732     The~unit~symbol~or~alias~'#1'~is~already~defined!
733 }
734 {
735     You~cannot~redefine~with~\token_to_str:N \spnewunit.
736 }
737 \msg_new:nnnn { spintent } { spunitalias-duplicate }
738 {
739     The~alias~'#1'~is~already~defined~or~reserved!
740 }
741 {
742     You~cannot~redefine~an~existing~unit~or~another~alias.
743 }
744 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
745 {
746     The~expression~'#2'~for~alias~'#1'~is~invalid.

```

```

747   }
748   {
749       It~must~contain~only~defined~units,~exponents~and~one~'/' .
750   }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit` The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

751 \NewDocumentCommand \spnewunit { m m }
752 {
753     \mode_if_math:TF
754     {
755         \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
756     }
757     {
758         \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
759         \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
760         {
761             \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
762         }
763     }
764 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias` The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

765 \NewDocumentCommand \spunitalias { m m }
766 {
767     \mode_if_math:TF
768     {
769         \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
770     }
771     {
772         \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
773         \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
774         {
775             \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
776         }
777         \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
778         {
779             \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
780         }
781     }
782 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.14 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{⟨argument⟩}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. TeX then uses this information to render the visual layout and construct the correct MathML speech intents.

```

\l__spintent_spmoney_luaset_symbol_str
\l__spintent_spmoney_luaset_position_str
\l__spintent_spmoney_luaset_sing_str
\l__spintent_spmoney_luaset_plur_str
\l__spintent_spmoney_luaset_conde_str
\l__spintent_spmoney_luaset_subsing_str
\l__spintent_spmoney_luaset_subplur_str
\l__spintent_spmoney_luaset_print_iso_str
\l__spintent_spmoney_luaset_currency_arg_str
\l__spintent_spmoney_luaset_status_str

```

```

783 \str_new:N \l__spintent_spmoney_luaset_symbol_str
784 \str_new:N \l__spintent_spmoney_luaset_position_str
785 \str_new:N \l__spintent_spmoney_luaset_sing_str
786 \str_new:N \l__spintent_spmoney_luaset_plur_str
787 \str_new:N \l__spintent_spmoney_luaset_conde_str

```

```

788 \str_new:N \l__spintent_spmoney_luaset_subsing_str
789 \str_new:N \l__spintent_spmoney_luaset_subplur_str
790 \str_new:N \l__spintent_spmoney_luaset_print_iso_str
791 \str_new:N \l__spintent_spmoney_luaset_currency_arg_str
792 \str_new:N \l__spintent_spmoney_luaset_status_str

```

(End of definition for `\l__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```

793 \bool_new:N \l__spintent_spmoney_sub_units_bool
794 \str_new:N \l__spintent_spmoney_intent_money_str
795 \str_new:N \l__spintent_spmoney_intent_sign_str
796 \str_new:N \l__spintent_spmoney_intent_sub_unit_str
797 \str_new:N \l__spintent_spmoney_res_main_voice_str
798 \int_new:N \l__spintent_spmoney_subnum_int
799 \str_new:N \l__spintent_spmoney_extracted_curr_str
800 \tl_new:N \l__spintent_spmoney_extracted_num_tl

```

(End of definition for `\l__spintent_spmoney_sub_units_bool` and others.)

11.14.1 Definition of error messages

```

currency-no-subunits
invalid-currency

```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```

801 \msg_new:nnnn { spintent } { currency-no-subunits }
802 {
803   The~currency~'#1'~does~not~support~subunits.~
804   The~option~'sub-units=true'~is~ignored.
805 }
806 {
807   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
808   do~not~have~fractional~subunits~registered~in~Lua~module.~
809 }
810 \msg_new:nnnn { spintent } { invalid-currency }
811 {
812   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
813 }
814 {
815   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
816   the~Lua~module.
817 }

```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.14.2 Auxiliary functions for `\spmoney`

```

\__spintent_spmoney_set_currency:n
\__spintent_spmoney_normalize_key:n
\__spintent_spmoney_normalize_key:V

```

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\l__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

818 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
819 {
820   \__spintent_spmoney_normalize_key:n {#1}
821   \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
822   {
823     \msg_error:nnn { spintent } { invalid-currency } {#1}
824   }
825   { \str_set:Nn \l__spintent_spmoney_luaset_currency_arg_str {#1} }
826 }
827 \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
828 {
829   \__spintent_luafun_spmoney_normalize_key:n { #1 }
830 }
831 \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintent_spmoney_set_currency:n` and `\__spintent_spmoney_normalize_key:n`.)

11.14.3 Definition of keys for \spmoney

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

currency
sub-units
dolar-math
cifras-sep
832 \keys_define:nn { spintent / spmoney }
833 {
834   currency      .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
835   currency      .initial:n    = { pesos },
836   currency      .value_required:n = true,
837   sub-units     .bool_set:N   = \__spintent_spmoney_sub_units_bool,
838   sub-units     .value_required:n = true,
839   dolar-math    .bool_set:N   = \__spintent_spmoney_dolar_math_bool,
840   dolar-math    .initial:n    = { true },
841   dolar-math    .value_required:n = true,
842   cifras-sep    .meta:nn     = { spintent/spnum } { cifras-sep = {#1} },
843   cifras-sep    .value_required:n = true,
844   unknown       .code:n      =
845   {
846     \str_set:Nn \__spintent_command_name_str { spmoney }
847     \__spintent_unknown_keys_cmd:n {#1}
848   },
849 }
```

(End of definition for `currency` and others.)

11.14.4 Internal functions for \spmoney

`\__spintent_spmoney_validate_money:` The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\l__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\l__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\l__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

850 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
851 {
852   \str_if_eq:VnT \l__spintent_luaset_has_units_str { true }
853   {
854     \__spintent_spmoney_normalize_key:V \l__spintent_luaset_arg_above_tl
855     \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
856     { \msg_error:nne { spintent } { invalid-currency } { \l__spintent_luaset_arg_above_tl } }
857     {
858       \str_set:NV
859       \l__spintent_spmoney_luaset_currency_arg_str
860       \l__spintent_luaset_arg_above_tl
861     }
862   }
863   \str_set:Ne \l__spintent_spmoney_luaset_status_str { found }
864   \__spintent_luafun_spmoney_lookup_data:V \l__spintent_spmoney_luaset_currency_arg_str
865 }
```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:` The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luaset_currency_arg_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

866 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
867 {
868   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
869   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
870   {
871     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
872     {
873       \bool_if:NTF \l__spintent_spmoney_dolar_math_bool
874       { \$ } { \text { \$ } }
875     }
876     { \l__spintent_spmoney_luaset_symbol_str }
877   }
878 }
```

(End of definition for `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:` The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to `pre`,

it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

879 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
880 {
881   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
882   {
883     \__spintent_spmoney_print_symbol:
884     \MathMLLarg { n } { \__spintent_spmoney_render_number_node: }
885   }
886   {
887     \MathMLLarg { n } { \__spintent_spmoney_render_number_node: }
888     \__spintent_spmoney_print_symbol:
889   }
890 }

```

(End of definition for `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:`
`\__spintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

891 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
892 {
893   \__spintent_spmoney_render_int:
894   \__spintent_spmoney_print_part_dec:
895 }
896 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
897 {
898   \__spintent_spmoney_render_int:
899 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`
`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

900 \cs_new_protected:Nn \__spintent_spmoney_print_int:
901 {
902   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
903   {
904     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
905     { \__spintent_spmoney_set_and_print_sub_units: }
906     { \__spintent_spmoney_intent_int: }
907   }
908   {
909     \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
910     {
911       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
912     }
913     {
914       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
915     }
916     \__spintent_invisible_sep:
917     \MathMLintent { \l__spintent_spmoney_intent_sign_str }
918     {
919       \l__spintent_luaset_sign_tl
920       \MathMLLarg { n }
921       {
922         \bool_if:NTF \l__spintent_spmoney_sub_units_bool
923         { \__spintent_spmoney_set_and_print_sub_units: }
924         { \__spintent_spmoney_intent_int: }
925       }
926     }
927   }
928 }
929 \cs_new_protected:Nn \__spintent_spmoney_intent_int:

```

```

930 {
931   \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
932   {
933     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
934     {
935       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
936     }
937     {
938       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
939     }
940     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
941     {
942       \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
943     }
944   }
945   {
946     \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
947     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
948     {
949       \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
950       {
951         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
952       }
953       {
954         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur
955       }
956     }
957   }
958   \__spintent_invisible_sep: % need for MathCAT
959   \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
960   {
961     \__spintent_spmoney_render_layout:
962   }
963 }

```

(End of definition for `\__spintent_spmoney_print_int:` and `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`
`\__spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “subunits”. If the database returns empty subunit data, it triggers a warning and automatically disables the “subunit” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “subunits” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘5’ to ‘50’), evaluates the correct singular or plural inflections for both the main “currency” and the “subunits”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

964 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
965 {
966   \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
967   {
968     \msg_warning:nne { spintent } { currency-no-subunits }
969     { \l__spintent_spmoney_luaset_currency_arg_str }
970     \bool_set_false:N \l__spintent_spmoney_sub_units_bool
971   }
972 }
973 \cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
974 {
975   \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
976   {
977     \int_set:Nn \l__spintent_spmoney_subnum_int
978     { \l__spintent_luaset_only_part_dec_str * 10 }
979     \str_set:Ne \l__spintent_luaset_only_part_dec_str
980     { \l__spintent_luaset_only_part_dec_str 0 }
981   }
982   {
983     \int_set:Nn \l__spintent_spmoney_subnum_int
984     { \l__spintent_luaset_only_part_dec_str }
985   }
986   \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
987   {
988     \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
989     { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
990   }

```

```

991     {
992         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
993         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
994     }
995     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
996     {
997         \str_set:NV \l__spintent_spmoney_res_main_voice_str
998         \l__spintent_spmoney_luaset_sing_str
999     }
1000     {
1001         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1002         \l__spintent_spmoney_luaset_plur_str
1003     }
1004     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1005     {
1006         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1007         \l__spintent_spmoney_luaset_conde_str
1008     }
1009     \__spintent_spmoney_build_tree:
1010 }

```

(End of definition for `\__spintent_spmoney_check_sub_units:` and `\__spintent_spmoney_set_and_print_sub_units:`)

```

\__spintent_spmoney_build_tree:
\__spintent_spmoney_print_integer_continue:
\__spintent_spmoney_print_sep_con:
\__spintent_spmoney_print_dec_sub_unit:

```

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “*currencies*”. It arranges the integer component, the decimal separator, and the “*subunit*” component based on the layout position (pre or post) of the “*currency symbol*”.

The function `\__spintent_spmoney_print_integer_continue:` isolates the rendering of the integer node with its associated main “*currency*” intent. Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed “*subunit*” intent to the decimal digits, ensuring accurate accessibility output.

```

1011 \cs_new_protected:Nn \__spintent_spmoney_build_tree:
1012 {
1013     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1014     {
1015         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1016         {
1017             \__spintent_spmoney_print_symbol:
1018             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1019         }
1020         \__spintent_spmoney_print_sep_con:
1021         \__spintent_spmoney_print_dec_sub_unit:
1022     }
1023     {
1024         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1025         {
1026             \__spintent_invisible_sep:
1027             \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1028         }
1029         \__spintent_spmoney_print_sep_con:
1030         \__spintent_spmoney_print_dec_sub_unit:
1031         \MathMLintent {:silent} { \__spintent_spmoney_print_symbol: }
1032     }
1033 }
1034 \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
1035 {
1036     \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1037     {
1038         \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1039     }
1040 }
1041 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1042 {
1043     \MathMLintent { con }
1044     {
1045         \mathord { \l__spintent_luaset_dec_sep_tl }
1046     }
1047 }
1048 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1049 {
1050     \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
1051     {

```

```

1052     \__spintent_invisible_sep:
1053     \MathMLarg { sub }
1054     {
1055         \__spintent_spmoney_render_dec:
1056     }
1057 }
1058 }

```

(End of definition for `\__spintent_spmoney_build_tree`: and others.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{(argument)}` passed in ‘`#1`’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:.` Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1059 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1060 {
1061     \__spintent_luafun_spmoney_prepare_input:n { #1 }
1062     \__spintent_luafun_clean_split_and_set:V \__spintent_spmoney_extracted_num_tl
1063     \str_if_empty:NF \__spintent_spmoney_extracted_curr_str
1064     {
1065         \str_set:Ne \__spintent_luaset_has_units_str { true }
1066         \str_set:NV \__spintent_luaset_arg_above_tl \__spintent_spmoney_extracted_curr_str
1067     }
1068     \__spintent_spmoney_validate_money:
1069     \bool_if:NT \__spintent_spmoney_sub_units_bool
1070     { \__spintent_spmoney_check_sub_units: }
1071     \__spintent_spmoney_print_int:
1072 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘`#1`’ for local key-value configurations and a mandatory argument ‘`#2`’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local TeX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1073 \NewDocumentCommand \spmoney { 0{} m }
1074 {
1075     \mode_if_math:TF
1076     {
1077         \group_begin:
1078         \keys_set:nn { spintent / spmoney } { #1 }
1079         \__spintent_spmoney_parse_and_route:n { #2 }
1080         \group_end:
1081     }
1082     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1083 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.15 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf[29]` package. Specifically, it uses the E (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the E key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1084 \str_new:N \__spintent_spshort_luaset_status_str
1085 \str_new:N \__spintent_spshort_luaset_layout_str
1086 \str_new:N \__spintent_spshort_luaset_expanded_str
1087 \str_new:N \__spintent_spshort_luaset_output_str
1088 \str_new:N \__spintent_spshort_luaset_base_str
1089 \str_new:N \__spintent_spshort_luaset_suffix_str

```

(End of definition for `\__spintent_spshort_luaset_status_str` and others.)

11.15.1 Definition of error messages for \spshort

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```
1090 \msg_new:nnnn { spintent } { unknown-abbreviation }
1091 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1092 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }
```

(End of definition for `unknown-abbreviation`.)

11.15.2 Accessibility interface

__spintent_spshort_tag_span:nn

__spintent_spshort_tag_span:en

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (#1) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```
1093 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1094 {
1095   \tag_mc_end_push:
1096   \tag_struct_begin:n { tag=Span, E={ #1 } }
1097   \tag_mc_begin:n { tag=Span }
1098   \tag_suspend:n { \spshort }
1099   #2
1100   \tag_resume:n { \spshort }
1101   \tag_mc_end:
1102   \tag_struct_end:
1103   \tag_mc_begin_pop:n {}
1104 }
1105 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }
```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.15.3 Definition of keys for \spshort

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```
1106 \keys_define:nn { spintent / spshort }
1107 {
1108   small-caps .bool_set:N = \__spintent_spshort_smallcaps_bool,
1109   small-caps .initial:n = false,
1110   small-caps .value_required:n = true,
1111   read-arg .str_set:N = \__spintent_spshort_read_arg_str,
1112   read-arg .initial:n = {},
1113   read-arg .value_required:n = true,
1114   unknown .code:n =
1115   {
1116     \str_set:Nn \__spintent_command_name_str { spshort }
1117     \__spintent_unknown_keys_cmd:n { #1 }
1118   },
1119 }
```

(End of definition for `small-caps` and `read-arg`.)

11.15.4 Internal functions for \spshort

__spintent_spshort_print:nn

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spintent_spshort_read_arg_str` and evaluates the optional `⟨keys⟩` passed in ‘#1’. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the `{⟨argument⟩}` passed in ‘#2’. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```
1120 \cs_new_protected:Nn \__spintent_spshort_print:nn
1121 {
1122   \str_clear:N \__spintent_spshort_read_arg_str
1123   \keys_set:nn { spintent / spshort } { #1 }
1124   \str_if_empty:NTF \__spintent_spshort_read_arg_str
1125   {
1126     \__spintent_spshort_process_and_render:n { #2 }
1127   }
1128   {
1129     \__spintent_spshort_render_bypass:n { #2 }
1130   }
1131 }
```


(End of definition for `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `{⟨argument⟩}` passed in ‘`#1`’.

It subsequently evaluates the execution state stored within the string variable `\__spintent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine success, fallback or `error`. Any unrecognized status automatically defaults to the “error” handler.

```
1132 \cs_new_protected:Nn \__spintent_spshort_process_and_render:n
1133 {
1134   \__spintent_luafun_spshort_process:n {#1}
1135   \str_case:VnF \__spintent_spshort_luaset_status_str
1136   {
1137     { success } { \__spintent_spshort_render_success: }
1138     { fallback } { \__spintent_spshort_render_fallback: }
1139     { error } { \__spintent_spshort_render_error:n {#1} }
1140   }
1141   { \__spintent_spshort_render_error:n {#1} }
1142 }
```

(End of definition for `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

`\__spintent_spshort_render_fallback:`

`\__spintent_spshort_render_bypass:n`

`\__spintent_spshort_render_error:n`

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined “speech” to the `{⟨argument⟩}`, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `{⟨argument⟩}` to prevent fatal compilation.

```
1143 \cs_new_protected:Nn \__spintent_spshort_render_success:
1144 {
1145   \__spintent_spshort_tag_span:en
1146   { \__spintent_spshort_luaset_expanded_str }
1147   { \__spintent_spshort_execute_layout: }
1148 }
1149 \cs_new_protected:Nn \__spintent_spshort_render_fallback:
1150 {
1151   \__spintent_spshort_tag_span:en
1152   { \__spintent_spshort_luaset_expanded_str }
1153   {
1154     \bool_if:NTF \__spintent_spshort_smallcaps_bool
1155     { \textsc{ \__spintent_spshort_luaset_output_str } }
1156     { \__spintent_spshort_luaset_output_str }
1157   }
1158 }
1159 \cs_new_protected:Nn \__spintent_spshort_render_bypass:n
1160 {
1161   \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }
1162 }
1163 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1164 {
1165   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1166 }
```

(End of definition for `\__spintent_spshort_render_success: and others`.)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “stored” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional `SMALL CAPS` based on the active boolean variables.

```
1167 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1168 {
1169   \str_case:VnF \__spintent_spshort_luaset_layout_str
1170   {
1171     { linear_regular }
1172     {
1173       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1174       { \textsc{ \__spintent_spshort_luaset_output_str } }
1175       { \__spintent_spshort_luaset_output_str }
1176     }
1177   }
```

```

1177 { linear_caps }
1178 { \l__spintent_spshort_luaset_output_str }
1179 { linear_mixed }
1180 { \l__spintent_spshort_luaset_output_str }
1181 { superscript }
1182 {
1183   \l__spintent_spshort_luaset_base_str
1184   \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1185   {
1186     \textsuperscript{
1187       \textsc{
1188         \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
1189       }
1190     }
1191   }
1192   {
1193     \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
1194   }
1195 }
1196 { small_caps_pure }
1197 {
1198   \bool_if:NTF \l__spintent_spshort_smallcaps_bool
1199   {
1200     \textsc{
1201       \str_lowercase:V \l__spintent_spshort_luaset_output_str
1202     }
1203   }
1204   { \l__spintent_spshort_luaset_output_str }
1205 }
1206 { acronym_long }
1207 { \l__spintent_spshort_luaset_output_str }
1208 }
1209 { \l__spintent_spshort_luaset_output_str }
1210 }

```

(End of definition for `\l__spintent_spshort_execute_layout:`.)

`\spshort` The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\l__spintent_spshort_print:nn` for processing and printing.

```

1211 \NewDocumentCommand \spshort { 0{} m }
1212 {
1213   \mode_if_math:TF
1214   {
1215     \msg_error:nnn { spintent } { need-text-mode } { \spshort }
1216   }
1217   {
1218     \mode_leave_vertical:
1219     \group_begin:
1220     \l__spintent_spshort_print:nn {#1} {#2}
1221     \group_end:
1222   }
1223 }

```

(End of definition for `\spshort`. This function is documented on page 7.)

11.16 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as `{\argument}`, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1224 \str_new:N \l__spintent_spsiglo_luaset_error_str

```

```

1225 \str_new:N \__spintent_spsiglo_luaset_arabic_str
1226 \str_new:N \__spintent_spsiglo_luaset_roman_min_str
1227 \str_new:N \__spintent_spsiglo_print_era_str

```

(End of definition for `\__spintent_spsiglo_luaset_error_str` and others.)

11.16.1 Definition of error messages for `\spsiglo`

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of **1** to **4000**, preventing incorrect speech synthesis.

```

1228 \msg_new:nnnn { spintent } { invalid-century-format }
1229 {
1230   Invalid~century~format~in~'#1'.
1231 }
1232 {
1233   The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1234   Ensure~the~value~is~between~1~and~4000.
1235 }

```

(End of definition for `invalid-century-format`.)

11.16.2 Definition of keys for `\spsiglo`

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1236 \keys_define:nn { spintent / spsiglo }
1237 {
1238   print-era .choices:nn =
1239     { none , ac , dc }
1240   {
1241     \int_case:nn { \l_keys_choice_int }
1242     {
1243       {1} { \str_set:Nn \__spintent_spsiglo_print_era_str { none } }
1244       {2} { \str_set:Nn \__spintent_spsiglo_print_era_str { ac } }
1245       {3} { \str_set:Nn \__spintent_spsiglo_print_era_str { dc } }
1246     }
1247   },
1248   print-era .initial:n = none,
1249   print-era .value_required:n = true,
1250   print-era / unknown .code:n =
1251     \msg_error:nneee { spintent } { unknown-choice }
1252     { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1253   unknown .code:n =
1254     {
1255       \str_set:Nn \__spintent_command_name_str { spsiglo }
1256       \__spintent_unknown_keys_cmd:n {#1}
1257     },
1258 }

```

(End of definition for `print-era`.)

11.16.3 Internal functions for `\spsiglo`

__spintent_spsiglo_render_simple:nn

__spintent_spsiglo_render_simple:VV

__spintent_spsiglo_render_with_era:nnn

__spintent_spsiglo_render_with_era:VVV

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1259 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1260 {
1261   \MathMLintent { #1:number } { \text { \textsc {#2} } }
1262 }
1263 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1264 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1265 {
1266   \str_if_eq:nnTF { #1 } { ac }
1267   {
1268     \MathMLintent { antes-de-cristo:postfix($z) }
1269     {
1270       \__spintent_invisible_sep:
1271       \MathMLarg { z }
1272       {
1273         \MathMLintent { #2:number }

```

```

1274         {
1275             \text { \textsc {#3}~a.~C. }
1276         }
1277     }
1278 }
1279 }
1280 {
1281     \MathMLintent { después-de-cristo:postfix($z) }
1282     {
1283         \__spintrent_invisible_sep:
1284         \MathMLarg { z }
1285         {
1286             \MathMLintent { #2:number }
1287             {
1288                 \text { \textsc {#3}~d.~C. }
1289             }
1290         }
1291     }
1292 }
1293 }
1294 \cs_generate_variant:Nn \__spintrent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for __spintrent_spsiglo_render_simple:nn and __spintrent_spsiglo_render_with_era:nnn.)

```

\__spintrent_spsiglo_tag_span:nn
\__spintrent_spsiglo_tag_span:en

```

The function `\__spintrent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{⟨argument⟩}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1295 \cs_new_protected:Nn \__spintrent_spsiglo_tag_span:nn
1296 {
1297     \tag_mc_end_push:
1298     \tag_struct_begin:n { tag=Span, alt={ #1 } }
1299     \tag_mc_begin:n { tag=Span }
1300     \tag_suspend:n { \spsiglo }
1301     #2
1302     \tag_resume:n { \spsiglo }
1303     \tag_mc_end:
1304     \tag_struct_end:
1305     \tag_mc_begin_pop:n {}
1306 }
1307 \cs_generate_variant:Nn \__spintrent_spsiglo_tag_span:nn { en }

```

(End of definition for __spintrent_spsiglo_tag_span:nn.)

```

\__spintrent_spsiglo_print_math:n
\__spintrent_spsiglo_print_text:n

```

The functions `\__spintrent_spsiglo_print_math:n` and `\__spintrent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\l__spintrent_spsiglo_luaset_error_str` returned by the Lua module `spintrent.lua` and issue an “error” if validation fails.

If the `{⟨argument⟩}` is valid, the function `\__spintrent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintrent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1308 \cs_new_protected:Nn \__spintrent_spsiglo_print_math:n
1309 {
1310     \str_if_eq:VnTF \l__spintrent_spsiglo_luaset_error_str { true }
1311     {
1312         \msg_error:nnnn { spintrent } { invalid-century-format }
1313         { \spsiglo } { #1 }
1314     }
1315     {
1316         \str_if_eq:VnTF \l__spintrent_spsiglo_print_era_str { none }
1317         {
1318             \__spintrent_spsiglo_render_simple:VV
1319             \l__spintrent_spsiglo_luaset_arabic_str
1320             \l__spintrent_spsiglo_luaset_roman_min_str
1321         }
1322         {
1323             \__spintrent_spsiglo_render_with_era:VVV
1324             \l__spintrent_spsiglo_print_era_str
1325             \l__spintrent_spsiglo_luaset_arabic_str
1326             \l__spintrent_spsiglo_luaset_roman_min_str

```

```

1327     }
1328   }
1329 }
1330 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1331 {
1332   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1333   {
1334     \msg_error:nnnn { spintent } { invalid-century-format }
1335     { \spsiglo } { #1 }
1336   }
1337   {
1338     \str_case:Vn \l__spintent_spsiglo_print_era_str
1339     {
1340       { none }
1341       {
1342         \__spintent_spsiglo_tag_span:en
1343         { \l__spintent_spsiglo_luaset_arabic_str }
1344         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1345       }
1346       { ac }
1347       {
1348         \__spintent_spsiglo_tag_span:en
1349         { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1350         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1351       }
1352       { dc }
1353       {
1354         \__spintent_spsiglo_tag_span:en
1355         { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1356         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1357       }
1358     }
1359   }
1360 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`.)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional `\keys` passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1361 \NewDocumentCommand \spsiglo { 0{} m }
1362 {
1363   \group_begin:
1364     \keys_set:nn { spintent / spsiglo } { #1 }
1365     \__spintent_luafun_spsiglo_parse:n { #2 }
1366     \mode_if_math:TF
1367     { \__spintent_spsiglo_print_math:n { #2 } }
1368     {
1369       \mode_leave_vertical:
1370       \__spintent_spsiglo_print_text:n { #2 }
1371     }
1372   \group_end:
1373 }

```

(End of definition for `\spsiglo`. This function is documented on page 11.)

11.17 The command `\sptime`

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1374 \str_new:N \l__spintent_sptime_luaset_seconds_str
1375 \str_new:N \l__spintent_sptime_luaset_has_seconds_str
1376 \str_new:N \l__spintent_sptime_luaset_error_str
1377 \str_new:N \l__spintent_sptime_luaset_is_fraction_str
1378 \str_new:N \l__spintent_sptime_luaset_base_str

```

(End of definition for `\l__spintent_sptime_luaset_seconds_str` and others.)

11.17.1 Definition of error messages

invalid-time-format

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1379 \msg_new:nnnn { spintent } { invalid-time-format }
1380 {
1381   Invalid~time~format~in~#1.  \
1382   The~argument~'#2'~does~not~match~any~supported~time~format.
1383 }
1384 {
1385   Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss.  \
1386   Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1387 }
```

(End of definition for `invalid-time-format`.)

11.17.2 Internal functions for `\sptime`

`__spintent_sptime_build_seconds:nn`

The internal function `__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1388 \cs_new_protected:Nn __spintent_sptime_build_seconds:nn
1389 {
1390   \MathMLintent { #1:prefix($z) }
1391   {
1392     \text { : }
1393     \MathMLarg { z }
1394     {
1395       \MathMLintent { segundos:postfix($m) }
1396       {
1397         __spintent_invisible_sep:
1398         \MathMLarg { m }
1399         {
1400           \text { #2 }
1401         }
1402       }
1403     }
1404   }
1405 }
```

(End of definition for `__spintent_sptime_build_seconds:nn`.)

`__spintent_sptime_build_tree:`

The internal function `__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `__spintent_sptime_build_seconds:` with the appropriate prefix (‘con’ or ‘minutos-con’), or simply appends the ‘minutos’ intent tag if no seconds are present.

```

1406 \cs_new_protected:Nn __spintent_sptime_build_tree:
1407 {
1408   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1409   {
1410     \MathMLintent { :time }
1411     {
1412       \text { \l__spintent_sptime_luaset_base_str }
1413     }
1414     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1415     {
1416       __spintent_sptime_build_seconds:nn { con }
1417       { \l__spintent_sptime_luaset_seconds_str }
1418     }
1419     {
1420       __spintent_sptime_build_seconds:nn { minutos-con }
1421       { \l__spintent_sptime_luaset_seconds_str }
1422     }
1423   }
1424   {
1425     \MathMLintent { :time }
1426     {
1427       \text { \l__spintent_sptime_luaset_base_str }
1428     }
1429   }
```



```

1429     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1430     {
1431         \MathMLintent { minutos }
1432         {
1433             \__spintent_invisible_sep:
1434         }
1435     }
1436 }
1437 }

```

(End of definition for `\__spintent_sptime_build_tree:`.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1438 \NewDocumentCommand \sptime { m }
1439 {
1440     \mode_if_math:TF
1441     {
1442         \group_begin:
1443         \__spintent_luafun_sptime_parse:n { #1 }
1444         \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1445         {
1446             \msg_error:nnnn { spintent } { invalid-time-format }
1447             { \sptime } { #1 }
1448         }
1449         {
1450             \__spintent_sptime_build_tree:
1451         }
1452     } \group_end:
1453 }
1454 { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1455 }

```

(End of definition for `\sptime`. This function is documented on page 11.)

11.18 The command `\spdate`

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

```

\l__spintent_spdate_luaset_output_str
\l__spintent_spdate_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1456 \str_new:N \l__spintent_spdate_luaset_output_str
1457 \str_new:N \l__spintent_spdate_luaset_error_str

```

(End of definition for `\l__spintent_spdate_luaset_output_str` and `\l__spintent_spdate_luaset_error_str`.)

11.18.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```

1458 \msg_new:nnnn { spintent } { invalid-date-format }
1459 {
1460     Invalid~date~or~unsupported~format~in~#1. \\\
1461     The~argument~'#2'~does~not~match~a~supported~format.
1462 }
1463 {
1464     Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1465     Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1466 }

```

(End of definition for `invalid-date-format`.)

11.18.2 Internal functions for \spdate

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaaset_output_str`.

```

1467 \cs_new_protected:Nn \__spintent_spdate_process:n
1468 {
1469   \__spintent_luafun_spdate_parse:n { #1 }
1470   \str_if_eq:VnTF \__spintent_spdate_luaaset_error_str { true }
1471   {
1472     \msg_error:nnnn { spintent } { invalid-date-format }
1473     { \spdate } { #1 }
1474   }
1475   {
1476     \MathMLintent { :date }
1477     {
1478       \text { \__spintent_spdate_luaaset_output_str }
1479     }
1480   }
1481 }

```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```

1482 \NewDocumentCommand \spdate { m }
1483 {
1484   \mode_if_math:TF
1485   {
1486     \group_begin:
1487     \__spintent_spdate_process:n { #1 }
1488     \group_end:
1489   }
1490   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1491 }

```

(End of definition for `\spdate`. This function is documented on page 11.)

11.19 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.20 The command \spusep

For the use of parentheses, we will provide the command `\spusep` with only two `⟨keys⟩` `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank `{⟨argument⟩}`, the second this is when an input is invalid.

```

1492 \msg_new:nnnn { spintent } { spusep-empty-arg }
1493 { Empty~argument~in~\token_to_str:N \spusep. }
1494 { \token_to_str:N \spusep~requires~a~non-empty~argument. }
1495 \msg_new:nnn { spintent } { spusep-invalid-arg }
1496 {
1497   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1498 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\__spintent_spusep_arg_tl`
`\__spintent_spusep_left_tl`
`\__spintent_spusep_right_tl`

The variable `\__spintent_spusep_arg_tl` stores the `{⟨argument⟩}`, the variable `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` stores the parentheses.

```

1499 \tl_new:N \__spintent_spusep_arg_tl
1500 \tl_new:N \__spintent_spusep_left_tl
1501 \tl_new:N \__spintent_spusep_right_tl

```

(End of definition for `\__spintent_spusep_arg_tl`, `\__spintent_spusep_left_tl`, and `\__spintent_spusep_right_tl`.)

size We define the $\langle keys \rangle$ size and silent.

```

1502 \keys_define:nn { spintent / spusep }
1503 {
1504   size .choices:nn =
1505     { none , big , Big, bigg, Bigg }
1506     {
1507       \int_case:nn { \l_keys_choice_int }
1508       {
1509         { 1 }
1510         {
1511           \tl_clear:N \l__spintent_spusep_left_tl
1512           \tl_clear:N \l__spintent_spusep_right_tl
1513         }
1514         { 2 }
1515         {
1516           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1517           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1518         }
1519         { 3 }
1520         {
1521           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1522           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1523         }
1524         { 4 }
1525         {
1526           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1527           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1528         }
1529         { 5 }
1530         {
1531           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1532           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1533         }
1534       }
1535     },
1536   size .initial:n = none,
1537   size .value_required:n = true,
1538   size / unknown .code:n =
1539     \msg_error:nnee { spintent } { unknown-choice }
1540     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1541   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1542   silent .initial:n = false,
1543   silent .value_required:n = true,
1544   unknown .code:n =
1545     {
1546       \str_set:Nn \l__spintent_command_name_str { spusep }
1547       \__spintent_unknown_keys_cmd:n {#1}
1548     },
1549 }

```

(End of definition for size, silent, and unknown.)

__spintent_spusep_set_parens:nn The function __spintent_spusep_set_parens:nn will place the argument passed in ‘#2’ to the right of the variables \l__spintent_spusep_left_tl and \l__spintent_spusep_right_tl and then check the state of the variable \l__spintent_spusep_silent_bool handled by the key silent to print.

```

1550 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1551 {
1552   \tl_put_right:cn { l__spintent_spusep_ #1 _tl } { #2 }
1553   \bool_if:NTF \l__spintent_spusep_silent_bool
1554   {
1555     \MathMLintent { :silent } { \tl_use:c { l__spintent_spusep_ #1 _tl } }
1556   }
1557   {
1558     \tl_use:c { l__spintent_spusep_ #1 _tl }
1559   }
1560 }

```

(End of definition for __spintent_spusep_set_parens:nn.)

__spintent_spusep_convert:n The function __spintent_spusep_convert:n captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable \l__spintent_spusep_arg_tl, validates the cases by converting them to the macros provided by

`unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1561 \cs_new_protected:Nn \__spintent_spusep_convert:n
1562 {
1563   \tl_set:Nx \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1564   \tl_if_empty:NT \l__spintent_spusep_arg_tl
1565   {
1566     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1567   }
1568   \str_case:VnF \l__spintent_spusep_arg_tl
1569   {
1570     { ( }
1571     {
1572       \__spintent_spusep_set_parens:nn { left } { \lparen }
1573     }
1574     { ) }
1575     {
1576       \__spintent_spusep_set_parens:nn { right } { \rparen }
1577     }
1578     { [ ]
1579     {
1580       \__spintent_spusep_set_parens:nn { left } { \lbrack }
1581     }
1582     { ] }
1583     {
1584       \__spintent_spusep_set_parens:nn { right } { \rbrack }
1585     }
1586     { \{ }
1587     {
1588       \__spintent_spusep_set_parens:nn { left } { \lbrace }
1589     }
1590     { \} }
1591     {
1592       \__spintent_spusep_set_parens:nn { right } { \rbrace }
1593     }
1594   }
1595   {
1596     \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
1597   }
1598 }

```

(End of definition for `\__spintent_spusep_convert:n`.)

\spusep The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1599 \NewDocumentCommand \spusep { O{} m }
1600 {
1601   \mode_if_math:TF
1602   {
1603     \group_begin:
1604     \keys_set:nn { spintent / spusep } {#1}
1605     \__spintent_spusep_convert:n {#2}
1606     \group_end:
1607   }
1608   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1609 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.20.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

spread-empty-args The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1610 \msg_new:nnnn { spintent } { spread-empty-args }
1611 { Empty~argument~in~\token_to_str:N \spread. }
1612 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

`\l__spintent_spread_intent_tl` The variable `\l__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle\textit{argument}\rangle$ for symbol to be injected into the `\MathMLintent`.

```
1613 \tl_new:N \l__spintent_spread_intent_tl
```

(End of definition for `\l__spintent_spread_intent_tl`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle\textit{argument}\rangle$ is empty or blank. If valid, it evaluates the reading $\langle\textit{argument}\rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```
1614 \cs_new_protected:Nn \__spintent_spread_exec:nn
1615 {
1616   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1617   { \msg_error:nn { spintent } { spread-empty-args } }
1618   {
1619     \__spintent_sanitize_affix:Nn \l__spintent_spread_intent_tl { #1 }
1620     \MathMLintent { \l__spintent_spread_intent_tl } { #2 }
1621   }
1622 }
```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic reading ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```
1623 \NewDocumentCommand \spread { m m }
1624 {
1625   \mode_if_math:TF
1626   {
1627     \group_begin:
1628     \__spintent_spread_exec:nn { #1 } { #2 }
1629     \group_end:
1630   }
1631   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1632 }
```

(End of definition for `\spread`. This function is documented on page 11.)

11.20.2 The command `\spdsym`

`\l__spintent_spdsym_read_sym_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
1633 \tl_new:N \l__spintent_spdsym_read_sym_tl
1634 \tl_new:N \l__spintent_spdsym_symbol_tl
1635 \tl_new:N \l__spintent_spdsym_prefix_tl
1636 \tl_new:N \l__spintent_spdsym_suffix_tl
```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

prefix We define the $\langle\textit{keys}\rangle$ prefix, suffix, set-sym and read-sym for `\spdsym`.

```
suffix 1637 \keys_define:nn { spintent / spdsym }
set-sym 1638 {
read-sym 1639   prefix .code:n =
unknown 1640   { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1641   prefix .initial:n = ,
1642   prefix .value_required:n = true,
1643   suffix .code:n =
1644   { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1645   suffix .initial:n = ,
1646   suffix .value_required:n = true,
1647   set-sym .choices:nn =
1648   { old-style , two-dots , slash }
1649   {
1650     \int_case:nn { \l_keys_choice_int }
1651     {
1652       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1653       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1654       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1655     }
1656   },
1657   set-sym .initial:n = two-dots,
1658   set-sym .value_required:n = true,
1659   set-sym / unknown .code:n =
1660   \msg_error:nneee { spintent } { unknown-choice }
```

```

1661     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1662   read-sym      .choices:nn =
1663   { dividido , dividido-por , dividido-entre }
1664   {
1665     \int_case:nn { \l_keys_choice_int }
1666     {
1667       {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1668       {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1669       {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1670     }
1671   },
1672   read-sym      .initial:n = dividido,
1673   read-sym      .value_required:n = true,
1674   read-sym / unknown .code:n =
1675     \msg_error:nneee { spintent } { unknown-choice }
1676     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1677     { \exp_not:n {#1} },
1678   unknown      .code:n =
1679   {
1680     \str_set:Nn \l__spintent_command_name_str { spdsym }
1681     \__spintent_unknown_keys_cmd:n {#1}
1682   },
1683 }

```

(End of definition for *prefix* and others.)

\spdsym Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```

1684 \NewDocumentCommand \spdsym { 0{} }
1685 {
1686   \mode_if_math:TF
1687   {
1688     \group_begin:
1689     \keys_set:nn { spintent / spdsym } { #1 }
1690     \MathMLintent
1691     {
1692       \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1693       { \l__spintent_spdsym_prefix_tl - }
1694       \l__spintent_spdsym_read_sym_tl
1695       \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1696       { - \l__spintent_spdsym_suffix_tl }
1697     }
1698     { \l__spintent_spdsym_symbol_tl }
1699     \group_end:
1700   }
1701   { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1702 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.20.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1703 \tl_new:N \l__spintent_spmsym_read_sym_tl
1704 \tl_new:N \l__spintent_spmsym_symbol_tl
1705 \tl_new:N \l__spintent_spmsym_prefix_tl
1706 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix  We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym
read-sym
not-print
unknown
1707 \keys_define:nn { spintent / spmsym }
1708 {
1709   prefix      .code:n =
1710   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
1711   prefix      .initial:n = ,
1712   prefix      .value_required:n = true,
1713   suffix      .code:n =
1714   { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
1715   suffix      .initial:n = ,
1716   suffix      .value_required:n = true,
1717   set-sym     .choices:nn =

```



```

1718     { cross , dot , asterisk }
1719   {
1720     \int_case:nn { \l_keys_choice_int }
1721     {
1722       {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
1723       {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
1724       {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
1725     }
1726   },
1727   set-sym          .initial:n = dot,
1728   set-sym          .value_required:n = true,
1729   set-sym / unknown .code:n =
1730     \msg_error:nneee { spintent } { unknown-choice }
1731     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
1732   read-sym         .choices:nn =
1733     { por , multiplicado-por , veces }
1734     {
1735       \int_case:nn { \l_keys_choice_int }
1736       {
1737         {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
1738         {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
1739         {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
1740       }
1741     },
1742   read-sym         .initial:n = por,
1743   read-sym         .value_required:n = true,
1744   read-sym / unknown .code:n =
1745     \msg_error:nneee { spintent } { unknown-choice }
1746     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
1747   not-print        .bool_set:N = \l__spintent_spmsym_not_print_bool,
1748   not-print        .initial:n = false,
1749   not-print        .value_required:n = true,
1750   unknown          .code:n =
1751     {
1752       \str_set:Nn \l__spintent_command_name_str { spmsym }
1753       \__spintent_unknown_keys_cmd:n {#1}
1754     },
1755   }

```

(End of definition for *prefix* and others.)

\spmsym Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local *(keys)*, safely attaching prefixes and suffixes if present.

```

1756 \NewDocumentCommand \spmsym { O{} }
1757 {
1758   \mode_if_math:TF
1759   {
1760     \group_begin:
1761     \keys_set:nn { spintent / spmsym } { #1 }
1762     \MathMLintent
1763     {
1764       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1765       { \l__spintent_spmsym_prefix_tl - }
1766       \l__spintent_spmsym_read_sym_tl
1767       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1768       { - \l__spintent_spmsym_suffix_tl }
1769     }
1770     {
1771       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1772       {
1773         \invisibletimes
1774       }
1775       { \l__spintent_spmsym_symbol_tl }
1776     }
1777     \group_end:
1778   }
1779   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1780 }

```

(End of definition for `\spmsym`. This function is documented on page 12.)

11.21 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the $\{\langle dividend \rangle\}$ and the $\{\langle divisor \rangle\}$. Both $\{\langle arguments \rangle\}$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\{\langle dividend \rangle\}$ nor the $\{\langle divisor \rangle\}$ is *blank*.

11.21.1 Definition of keys for `\spdiv`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spdiv` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```

1781 \keys_define:nn { spintent / spdiv }
1782 {
1783   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1784   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1785   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1786   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1787   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1788   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1789   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1790   wrap-parens     .initial:n  = false,
1791   wrap-parens     .value_required:n = true,
1792   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1793   read-parens     .initial:n  = true,
1794   read-parens     .value_required:n = true,
1795   unknown         .code:n =
1796   {
1797     \str_set:Nn \l__spintent_command_name_str { spdiv }
1798     \__spintent_unknown_keys_cmd:n {#1}
1799   },
1800 }
```

(End of definition for `cifras-sep` and others.)

11.21.2 Definition of error messages

`spdiv-empty-argument` Raised when either the $\{\langle dividend \rangle\}$ or the $\{\langle divisor \rangle\}$ is missing or *blank*.

```

1801 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1802 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1803 {
1804   You~must~provide~both~a~dividend~and~a~divisor.\\
1805 }
```

(End of definition for `spdiv-empty-argument`.)

11.21.3 Internal functions for `\spdiv`

`\__spintent_spdiv_render_op:nn` The function `\__spintent_spdiv_render_op:nn` parses both $\{\langle arguments \rangle\}$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1806 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1807 {
1808   \spnum {#1}
1809   \MathMLintent { \l__spintent_spdsym_read_sym_tl }
1810   {
1811     \l__spintent_spdsym_symbol_tl
1812   }
1813   \spnum {#2}
1814 }
```

(End of definition for `\__spintent_spdiv_render_op:nn`.)

`\__spintent_spdiv_render_parens:nn` The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1815 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1816 {
1817   \bool_if:NTF \l__spintent_spdiv_read_parens_bool
1818   {
1819     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1820   }
1821   {
1822     \MathMLintent { :silent } { ( }
1823     \__spintent_spdiv_render_op:nn {#1} {#2}
```

```

1824     \MathMLintent { :silent } { } }
1825   }
1826 }

```

(End of definition for `\__spintent_spdiv_render_parens:nn`.)

`\__spintent_spdiv_process:nnn` The function `\__spintent_spdiv_process:nnn` sets $\langle keys \rangle$, validates $\langle arguments \rangle$, and decides on parenthesis wrapping.

```

1827 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1828 {
1829   \keys_set:nn { spintent / spdiv } { #1 }
1830   \tl_if_blank:nTF { #2 }
1831   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1832   {
1833     \tl_if_blank:nTF { #3 }
1834     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1835     {
1836       \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool
1837       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1838       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1839     }
1840   }
1841 }

```

(End of definition for `\__spintent_spdiv_process:nnn`.)

`\spdiv` Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1842 \NewDocumentCommand \spdiv { O{} m m }
1843 {
1844   \mode_if_math:TF
1845   {
1846     \group_begin:
1847     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1848     \group_end:
1849   }
1850   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1851 }

```

(End of definition for `\spdiv`. This function is documented on page 13.)

11.22 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the $\langle first\ factor \rangle$ and the $\langle second\ factor \rangle$. Both $\langle arguments \rangle$ are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the $\langle first\ factor \rangle$ nor the $\langle second\ factor \rangle$ is *blank*.

11.22.1 Definition of keys for `\spmul`

We define the $\langle keys \rangle$ `wrap-parens` and `read-parens` for `\spmul` command. Number-related $\langle keys \rangle$ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator $\langle keys \rangle$ `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1852 \keys_define:nn { spintent / spmul }
1853 {
1854   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1855   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1856   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1857   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1858   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1859   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1860   wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1861   wrap-parens     .initial:n  = false,
1862   wrap-parens     .value_required:n = true,
1863   read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1864   read-parens     .initial:n  = true,
1865   read-parens     .value_required:n = true,
1866   unknown         .code:n =
1867   {
1868     \str_set:Nn \l__spintent_command_name_str { spmul }
1869     \__spintent_unknown_keys_cmd:n {#1}
1870   },
1871 }

```

(End of definition for `cifras-sep` and others.)

11.22.2 Definition of error messages

`spmul-empty-argument` Raised when either the $\langle\textit{first factor}\rangle$ or the $\langle\textit{second factor}\rangle$ is missing or *blank*.

```

1872 \msg_new:nnnn { spintent } { spmul-empty-argument }
1873 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1874 {
1875   You~must~provide~both~first~factor~and~second~factor.\\
1876 }

```

(End of definition for `spmul-empty-argument`.)

11.22.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1877 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1878 {
1879   \spnum {#1}
1880   \MathMLintent { \__spintent_spmsym_read_sym_tl }
1881   {
1882     \__spintent_spmsym_symbol_tl
1883   }
1884   \spnum {#2}
1885 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1886 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1887 {
1888   \bool_if:NTF \__spintent_spmul_read_parens_bool
1889   {
1890     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1891   }
1892   {
1893     \MathMLintent { :silent } { ( }
1894     \__spintent_spmul_render_op:nn {#1} {#2}
1895     \MathMLintent { :silent } { ) }
1896   }
1897 }

```

(End of definition for `\__spintent_spmul_render_parens:nn`.)

`\__spintent_spmul_process:nnn` The function `\__spintent_spmul_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1898 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1899 {
1900   \keys_set:nn { spintent / spmul } { #1 }
1901   \tl_if_blank:nTF { #2 }
1902   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1903   {
1904     \tl_if_blank:nTF { #3 }
1905     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1906     {
1907       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1908       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1909       { \__spintent_spmul_render_op:nn {#2} {#3} }
1910     }
1911   }
1912 }

```

(End of definition for `\__spintent_spmul_process:nnn`.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1913 \NewDocumentCommand \spmul { 0{ } m m }
1914 {
1915   \mode_if_math:TF
1916   {
1917     \group_begin:
1918     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1919     \group_end:
1920   }
1921   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1922 }

```

(End of definition for `\spmcd`. This function is documented on page 13.)

11.23 The commands `\spmcd` and `\spmcm`

The user-level commands `\spmcd` and `\spmcm` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.23.1 Variables for `\spmcd` and `\spmcm`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\__spintent_spmcm_spmcd_luaerror_str
\__spintent_mc_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl
```

```
1923 \str_new:N \__spintent_spmcm_spmcd_luaerror_str
1924 \seq_new:N \__spintent_mc_args_seq
1925 \clist_map_inline:nn { mcd , mcm }
1926 {
1927   \tl_new:c { \__spintent_sp #1 _luaerror_str }
1928   \bool_new:c { \__spintent_sp #1 _print_result_bool }
1929   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1930   \bool_new:c { \__spintent_sp #1 _print_upper_bool }
1931   \tl_new:c { \__spintent_sp #1 _prefix_tl }
1932   \tl_new:c { \__spintent_sp #1 _suffix_tl }
1933 }
```

(End of definition for `\__spintent_spmcm_spmcd_luaerror_str` and others.)

11.23.2 Definition of error messages

`mc-invalid-arguments` Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
1934 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1935 {
1936   Invalid~arguments~provided~to~#1.
1937 }
1938 {
1939   The~arguments~must~be~a~comma-separated~list. \\\
1940 }
```

(End of definition for `mc-invalid-arguments`.)

11.23.3 Keys for `\spmcd` and `\spmcm`

We define the shared `<keys>` `result`, `prefix`, `read-cmd`, `upper-case`, `sample-arg` and `silent-cmd`.

```
result
prefix
read-cmd
upper-case
sample-arg
silent-cmd
```

```
1941 \clist_map_inline:nn { mcd , mcm }
1942 {
1943   \keys_define:nn { spintent / sp #1 }
1944   {
1945     result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
1946     result      .initial:n = false,
1947     result      .value_required:n = true,
1948     prefix      .code:n =
1949       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
1950     prefix      .initial:n = ,
1951     prefix      .value_required:n = true,
1952     read-cmd    .choices:nn =
1953       { terse , clear }
1954       {
1955         \int_case:nn { \l_keys_choice_int }
1956         {
1957           {1} { \bool_set_true:c { \__spintent_sp #1 _read_terse_bool } }
1958           {2} { \bool_set_false:c { \__spintent_sp #1 _read_terse_bool } }
1959         }
1960       },
1961     read-cmd    .initial:n = clear,
1962     read-cmd    .value_required:n = true,
1963     read-cmd / unknown .code:n =
1964       \msg_error:nneee { spintent } { unknown-choice }
1965       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1966     upper-case  .bool_set:c = { \__spintent_sp #1 _print_upper_bool },
1967     upper-case  .initial:n = false,
1968     upper-case  .value_required:n = true,
1969     sample-arg  .bool_set:c = { \__spintent_sp #1 _sample_arg_bool },
1970     sample-arg  .initial:n = false,
1971     sample-arg  .value_required:n = true,
1972     silent-cmd  .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
```

```

1973         silent-cmd          .initial:n = false,
1974         silent-cmd          .value_required:n = true,
1975         unknown              .code:n =
1976         {
1977             \str_set:Nn \l__spintent_command_name_str { sp #1 }
1978             \__spintent_unknown_keys_cmd:n {##1}
1979         },
1980     }
1981 }

```

(End of definition for `result` and others.)

11.23.4 Internal functions for `\spmc` and `\spmcm`

`\__spintent_mc_process_args:nn`

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its arguments ‘#2’, injecting MathML intents.

```

1982 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1983 {
1984     \seq_set_from_clist:Nn \l__spintent_mc_args_seq {#2}
1985     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
1986     {
1987         \seq_set_map:NNn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
1988         { \MathMLintent { :silent } { ##1 } }
1989         \MathMLintent { entre:silent } { ( }
1990         \seq_use:Nnnn \l__spintent_mc_args_seq
1991         { \MathMLintent { _y:silent } { , } }
1992         { \MathMLintent { :silent } { , } }
1993         { \MathMLintent { _y:silent } { , } }
1994     }
1995     {
1996         \MathMLintent { entre } { ( }
1997         \seq_use:Nnnn \l__spintent_mc_args_seq
1998         { \MathMLintent { _y } { , } }
1999         { \MathMLintent { :silent } { , } }
2000         { \MathMLintent { _y } { , } }
2001     }
2002     \MathMLintent { :silent } { ) }
2003 }

```

(End of definition for `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmc` and `\spmcm` commands. It takes the command identifier ‘#1’, its full reading string ‘#2’, the local *(keys)* ‘#3’, and the optional values ‘#4’. It dynamically resolves variables using the identifier to build the MathML `intent` string (including any prefixes or suffixes) and prints the math operator. If values are provided in ‘#4’, it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```

2004 \cs_new_protected:Nn \__spintent_spmc_process:nnnn
2005 {
2006     \keys_set:nn { spintent / sp #1 } { #3 }
2007     \MathMLintent
2008     {
2009         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
2010         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
2011         \bool_if:cTF { \l__spintent_sp #1 _read_terse_bool }
2012         { #1 } { #2 }
2013         \bool_if:cT { \l__spintent_sp #1 _silent_cmd_bool }
2014         {
2015             \tl_if_novalue:nTF {#4}
2016             { :silent }
2017             { \bool_if:cT { \l__spintent_sp #1 _sample_arg_bool } { :silent } }
2018         }
2019     }
2020     {
2021         \operatorname
2022         {
2023             \bool_if:cTF { \l__spintent_sp #1 _print_upper_bool }
2024             { \text_uppercase:n {#1} } { #1 }
2025         }
2026     }
2027     \tl_if_novalue:nF { #4 }
2028     {

```



```

2029     \bool_if:cTF { \__spintent_sp #1 _sample_arg_bool }
2030     {
2031         \__spintent_mc_process_args:nn {#1} {#4}
2032     }
2033     {
2034         \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2035         \str_if_eq:VnTF \__spintent_spmcm_spmcd_luaset_error_str { true }
2036         {
2037             \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2038         }
2039         {
2040             \__spintent_mc_process_args:nn {#1} {#4}
2041             \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2042             {
2043                 \MathMLintent { es-igual-a } { = }
2044                 \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2045             }
2046         }
2047     }
2048 }
2049 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2050 \NewDocumentCommand \spmc { 0{} } d() {
2051     {
2052         \mode_if_math:TF
2053         {
2054             \group_begin:
2055             \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2056             \group_end:
2057         }
2058         { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2059     }
2060 \NewDocumentCommand \spmcm { 0{} } d() {
2061     {
2062         \mode_if_math:TF
2063         {
2064             \group_begin:
2065             \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
2066             \group_end:
2067         }
2068         { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2069     }

```

(End of definition for `\spmc` and `\spmcm`. These functions are documented on page 15.)

11.24 The commands `\spnM` and `\spnD`

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.24.1 Variables for `\spnM` and `\spnD`

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spnM_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spnM_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spnM_print_result_bool
\__spintent_spnD_print_result_bool
\__spintent_spnM_silent_cmd_bool
\__spintent_spnD_silent_cmd_bool
\__spintent_spnM_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spnM_print_result_num_tl
\__spintent_spnD_print_result_num_tl

```

```

2070 \seq_new:N \__spintent_md_out_seq
2071 \clist_map_inline:nn { nM , nD }
2072 {
2073     \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2074     \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2075     \bool_new:c { \__spintent_sp #1 _print_result_bool }
2076     \bool_new:c { \__spintent_sp #1 _silent_cmd_bool }
2077     \tl_new:c { \__spintent_sp #1 _prefix_tl }
2078     \tl_new:c { \__spintent_sp #1 _print_result_num_tl }
2079 }

```

(End of definition for `\__spintent_md_out_seq` and others.)

11.24.2 Keys for \spnM and \spnD

We define the shared *(keys)* result, silent-cmd, result-num and prefix.

```

result      \clist_map_inline:nn { nM , nD }
silent-cmd  2080 {
result-num  2081 {
prefix      2082   \keys_define:nn { spintent / sp #1 }
sample-arg  2083   {
2084       result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
2085       result      .initial:n = false,
2086       result      .value_required:n = true,
2087       silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
2088       silent-cmd .initial:n = false,
2089       silent-cmd .value_required:n = true,
2090       result-num .tl_set:c = { l__spintent_sp #1 _print_result_num_tl },
2091       result-num .initial:n = ,
2092       prefix      .code:n =
2093       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
2094       prefix      .initial:n = ,
2095       prefix      .value_required:n = true,
2096       sample-arg .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
2097       sample-arg .initial:n = false,
2098       sample-arg .value_required:n = true,
2099       unknown    .code:n =
2100       {
2101           \str_set:Nn \__spintent_command_name_str { sp #1 }
2102           \__spintent_unknown_keys_cmd:n {##1}
2103       },
2104   }
2105 }
```

(End of definition for result and others.)

11.24.3 Definition of error messages for \spnM and \spnD

md-invalid-argument Exception triggered when the argument for multiples or divisors is invalid.

```

2106 \msg_new:nnnn { spintent } { md-invalid-argument }
2107 { Invalid~argument~provided~to~#1. }
2108 {
2109     The~argument~must~be~a~single~positive~integer. \
2110 }
```

(End of definition for md-invalid-argument.)

11.24.4 Internal functions for \spnM and \spnD

__spintent_spnM_spnD_process:nnnn

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spnM_and_D_exec:nnnn`.

```

2111 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnn
2112 {
2113     \keys_set:nn { spintent / sp #1 } { #4 }
2114     \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2115 }
```

(End of definition for __spintent_spnM_spnD_process:nnnn.)

__spintent_spnM_spnD_intent:nn

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for “múltiplos”.

```

2116 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2117 {
2118     \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2119     { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2120     \str_case:nnF { #2 }
2121     { { múltiplos } { múltiplos } }
2122     { #2 }
2123 }
```

(End of definition for __spintent_spnM_spnD_intent:nn.)

`\__spintent_spnM_spnD_silent:nn`

The internal function `\__spintent_spnM_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2124 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2125 {
2126   \MathMLintent { :silent } { \operatorname { #1 } }
2127   \MathMLintent { :silent } { ( }
2128   \MathMLintent { :silent } { \__spintent_invisible_sep: }
2129   \MathMLintent { :silent } { #2 }
2130   \MathMLintent { :silent } { ) }
2131 }
```

(End of definition for `\__spintent_spnM_spnD_silent:nn`.)

`\__spintent_spnM_spnD_result:n`

The internal function `\__spintent_spnM_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2132 \cs_new_protected:Nn \__spintent_spnM_spnD_result:n
2133 {
2134   \use:c { __spintent_luafun_calcular_ #1 :nn }
2135   { \__spintent_luaset_part_int_tl }
2136   { \tl_use:c { l__spintent_sp #1 _print_result_num_tl } }
2137   \MathMLintent { son } { = { \ } }
2138   \seq_set_from_clist:Ne
2139     \__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
2140   \seq_use:Nnnn \__spintent_md_out_seq
2141     { \MathMLintent { _y } { , } }
2142     { \MathMLintent { :silent } { , } }
2143     { \MathMLintent { _y } { , } }
2144   \str_if_eq:vnT
2145     { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2146     {
2147       \MathMLintent { :silent } { , } ~ \dots
2148     }
2149   \MathMLintent { :silent } { \ } }
2150 }
```

(End of definition for `\__spintent_spnM_spnD_result:n`.)

`\__spintent_spnM_spnD_exec:nnnn`

The function `\__spintent_spnM_and_D_exec:nnnn` implements the multiples and divisors commands. It takes the $\langle \text{argument} \rangle$ ‘#1’ (the command name), the $\langle \text{argument} \rangle$ ‘#2’ (the mathematical symbol), the base reading $\langle \text{argument} \rangle$ ‘#3’, and the optional $\langle \text{argument} \rangle$ ‘#4’. If no $\langle \text{argument} \rangle$ is provided, it evaluates the key `silent-cmd` using the default token ‘n’. If an $\langle \text{argument} \rangle$ is given, it evaluates the key `sample-arg`. If true, it processes the $\langle \text{argument} \rangle$ directly and evaluates the key `silent-cmd`. If false, it bypasses the *silent mode*, validates the $\langle \text{argument} \rangle$ as a natural number, and evaluates the key `result` optionally calculate and append the resulting set.

```
2151 \cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
2152 {
2153   \tl_if_novalue:nTF { #4 }
2154   {
2155     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2156     { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
2157     {
2158       \MathMLintent
2159         { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2160         { \operatorname { #2 } ( n ) }
2161     }
2162   }
2163   {
2164     \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
2165     {
2166       \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
2167       { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
2168       {
2169         \MathMLintent
2170           { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2171           { \operatorname { #2 } ( #4 ) }
2172       }
2173     }
2174   }
2175 }
```

```

2173     }
2174     {
2175         \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2176         \str_if_eq:VnTF \l__spintent_luaset_has_natural_str { true }
2177         {
2178             \MathMLintent
2179             { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
2180             { \operatorname { #2 } } ( #4 )
2181             \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2182             { \__spintent_spnM_spnD_result:n { #1 } }
2183         }
2184     }
2185     \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
2186 }
2187 }
2188 }
2189 }

```

(End of definition for __spintent_spnM_spnD_exec:nnnn.)

\spnM The commands **\spnM** and **\spnD** act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function **__spintent_spnM_spnD_process:nnnnn**, passing along the optional *(keys)* ‘#1’ and the numeric *{(argument)}* ‘#2’.

```

2190 \NewDocumentCommand \spnM { O{} d() }
2191 {
2192     \mode_if_math:TF
2193     {
2194         \group_begin:
2195         \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2196         \group_end:
2197     }
2198     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2199 }
2200 \NewDocumentCommand \spnD { O{} d() }
2201 {
2202     \mode_if_math:TF
2203     {
2204         \group_begin:
2205         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2206         \group_end:
2207     }
2208     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2209 }

```

(End of definition for \spnM and \spnD. These functions are documented on page 14.)

11.25 The command \spang

The command **\spang** is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function **__spintent_luafun_spang_parse:n**. This separates the degrees, minutes, and seconds to construct a **MathML intent** tree with the correct postfixes for NVDA/MathCAT.

These internal string variables store the values returned from the Lua module **spintent.lua**. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2210 \str_new:N \l__spintent_spang_luaset_grado_str
2211 \str_new:N \l__spintent_spang_luaset_minuto_str
2212 \str_new:N \l__spintent_spang_luaset_segundo_str
2213 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for \l__spintent_spang_luaset_grado_str and others.)

11.25.1 Definition of error messages for \spang

invalid-angle-format

The error message **invalid-angle-format** defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2214 \msg_new:nnnn { spintent } { invalid-angle-format }
2215 { Invalid~angle~format~in~#1. }
2216 {
2217     The~argument~'~#2'~could~not~be~parsed. \\\
2218 }

```

(End of definition for `invalid-angle-format`.)

11.25.2 Internal functions for `\spang`

`__spintent_spang_process:n`

The internal function `__spintent_spang_process:n` parses the angle input using `__spintent_luafun_spang_parse:n` after resetting the error flag in `\__spintent_spang_luaerror_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\__spintent_spang_luaerror_grado_str`, `\__spintent_spang_luaerror_minuto_str`, and `\__spintent_spang_luaerror_segundo_str`. For each non-empty variable, it constructs the corresponding MathML intent structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2219 \cs_new_protected:Nn __spintent_spang_process:n
2220 {
2221   \str_set:Nn \__spintent_spang_luaerror_str { false }
2222   __spintent_luafun_spang_parse:n { #1 }
2223   \str_if_eq:NnTF \__spintent_spang_luaerror_str { true }
2224   {
2225     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2226   }
2227   {
2228     \str_if_empty:NF \__spintent_spang_luaerror_grado_str
2229     {
2230       \spunit{ \__spintent_spang_luaerror_grado_str °}
2231     }
2232     \str_if_empty:NF \__spintent_spang_luaerror_minuto_str
2233     {
2234       \MathMLintent { minutos : postfix($m) }
2235       {
2236         \MathMLarg { m }
2237         {
2238           \spunit{ \__spintent_spang_luaerror_minuto_str ' }
2239         }
2240       }
2241     }
2242     \str_if_empty:NF \__spintent_spang_luaerror_segundo_str
2243     {
2244       \MathMLintent { segundos : postfix($s) }
2245       {
2246         \MathMLarg { s }
2247         {
2248           \spunit { \__spintent_spang_luaerror_segundo_str " }
2249         }
2250       }
2251     }
2252   }
2253 }

```

(End of definition for `__spintent_spang_process:n`.)

`\spang`

The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `__spintent_spang_process:n` to process the $\langle argument \rangle$.

```

2254 \NewDocumentCommand \spang { m }
2255 {
2256   \mode_if_math:TF
2257   {
2258     \group_begin:
2259     __spintent_spang_process:n { #1 }
2260     \group_end:
2261   }
2262   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2263 }

```

(End of definition for `\spang`. This function is documented on page 21.)

11.26 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the $\langle argument \rangle$ is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.26.1 Definition of keys for \spnZ

Now we define the $\langle keys \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle keys \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2264 \keys_define:nn { spintrent / spnZ }
2265 {
2266   cifras-sep .meta:nn = { spintrent / spnum } { cifras-sep = {#1} },
2267   read-sign .meta:nn = { spintrent / spnum } { read-sign = {#1} },
2268   wrap-parens .bool_set:N = \l__spintrent_spnz_wrap_parens_bool,
2269   wrap-parens .initial:n = false,
2270   wrap-parens .value_required:n = true,
2271   read-parens .bool_set:N = \l__spintrent_spnz_read_parens_bool,
2272   read-parens .initial:n = true,
2273   read-parens .value_required:n = true,
2274   unknown .code:n =
2275     {
2276       \str_set:Nn \l__spintrent_command_name_str { spnZ }
2277       \__spintrent_unknown_keys_cmd:n {#1}
2278     },
2279 }

```

(End of definition for `cifras-sep` and others.)

11.26.2 Definition of error messages

`spnz-not-integer` Raised when the $\{ \langle argument \rangle \}$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2280 \msg_new:nnnn { spintrent } { spnz-not-integer }
2281 { The~value~'~\the~#1~'~is~not~a~valid~integer.~ }
2282 {
2283   Command~\token_to_str:N \spnZ~only~accepts~integers.~\
2284 }

```

(End of definition for `spnz-not-integer`.)

11.26.3 Internal functions for \spnZ

`\__spintrent_spnz_render_parens:` The function `\__spintrent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintrent_spnz_read_parens_bool` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2285 \cs_new_protected:Nn \__spintrent_spnz_render_parens:
2286 {
2287   \bool_if:NTF \l__spintrent_spnz_read_parens_bool
2288   {
2289     ( \__spintrent_spnum_print_num_start: )
2290   }
2291   {
2292     \MathMLintent { :silent } { ( }
2293     \__spintrent_spnum_print_num_start:
2294     \MathMLintent { :silent } { ) }
2295   }
2296 }

```

(End of definition for `\__spintrent_spnz_render_parens:`.)

`\__spintrent_spnz_process:nn`

The function `\__spintrent_spnz_process:nn` first processes the $\langle keys \rangle$ passed in ‘#1’ and then calls `\__spintrent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintrent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintrent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintrent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintrent_spnz_render_parens:` or `\__spintrent_spnum_print_num_start:`.

```

2297 \cs_new_protected:Nn \__spintrent_spnz_process:nn
2298 {
2299   \keys_set:nn { spintrent / spnZ } {#1}
2300   \__spintrent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2301   \str_if_eq:VnTF \l__spintrent_luaset_has_integer_str { true }
2302   {
2303     \tl_if_empty:NTF \l__spintrent_luaset_part_int_tl
2304     { \msg_error:nnn { spintrent } { spnum-no-digits } {#2} }
2305     {
2306       \bool_if:NTF \l__spintrent_spnz_wrap_parens_bool

```



```

2307         { \__spintent_spnz_render_parens: }
2308         { \__spintent_spnz_print_num_start: }
2309     }
2310 }
2311 { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2312 }

```

(End of definition for __spintent_spnz_process:nn.)

\spnZ Defines the user command **\spnZ**, which is executed within a *group* and strictly enforces *math mode*.

```

2313 \NewDocumentCommand \spnZ { 0{} m }
2314 {
2315     \mode_if_math:TF
2316     {
2317         \group_begin:
2318         \__spintent_spnz_process:nn { #1 } { #2 }
2319         \group_end:
2320     }
2321     { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2322 }

```

(End of definition for \spnZ. This function is documented on page 16.)

11.27 The command \spmQ

The command **\spmQ** typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module **spintent.lua** via **__spintent_luafun_spmQ_scale_int:Vn**, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.27.1 Definition of variables for \spmQ

The variable **__spintent_spmQ_join_word_str** stores the value set by the key **join-word** which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable **__spintent_spmQ_math_style_tl** stores the current mathematical style used by the function **__spintent_spmQ_set_wrap_parens_size:**.

The variable **__spintent_spnQ_intent_read_sign_str** stores the **:intent** passed to **\MathMLintent** when the signs ‘+’ or ‘-’ are present in the integer part, used by the function **__spintent_spmQ_print_sacle_int:n**.

The variables **__spintent_spnQ_wrap_parens_l_tl** and **__spintent_spnQ_wrap_parens_r_tl** store the sized parentheses set by the function **__spintent_spmQ_set_wrap_parens_size:** and used by the key **wrap-parens**.

The integer variable **__spintent_saved_luamml_flag_int** temporarily stores the current value of **__luamml_flag_int** before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents **luamml** from generating a spurious **<math>** element around the sub_box processed in isolation, before **\frac** has expanded.

```

2323 \str_new:N \__spintent_spmQ_join_word_str
2324 \tl_new:N \__spintent_spmQ_math_style_tl
2325 \str_new:N \__spintent_spnQ_intent_read_sign_str
2326 \tl_new:N \__spintent_spnQ_wrap_parens_l_tl
2327 \tl_new:N \__spintent_spnQ_wrap_parens_r_tl
2328 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for __spintent_spmQ_join_word_str and others.)

11.27.2 Definition of error messages for \spmQ

The error message **spmQ-zero-denominator** is raised when a zero denominator is passed to **\spmQ**. The second error **spmQ-not-integer** is returned when the integer part passed in **{\argument}** is not actually an integer.

```

2329 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2330 { Zero-denominator~in~\token_to_str:N \spmQ. }
2331 {
2332     The~denominator~of~a~fraction~cannot~be~zero.\!
2333 }
2334 \msg_new:nnnn { spintent } { spmQ-not-integer }
2335 { The~value~'#1'~is~not~a~valid~integer. }
2336 {
2337     Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\!
2338 }

```

(End of definition for spmQ-zero-denominator and spmQ-not-integer.)

11.27.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

join-word 2339 \keys_define:nn { spintenc / spmQ }
use-spnZ   2340 {
wrap-parens 2341   join-word           .choices:nn =
read-parens 2342   { entero , enteros , y }
print-dfrac 2343   {
unknown     2344     \int_case:nn { \l_keys_choice_int }
2345     {
2346       {1} { \str_set:Nn \l__spintenc_spmQ_join_word_str { entero } }
2347       {2} { \str_set:Nn \l__spintenc_spmQ_join_word_str { enteros } }
2348       {3} { \str_set:Nn \l__spintenc_spmQ_join_word_str { y } }
2349     }
2350   },
2351   join-word           .initial:n = y,
2352   join-word           .value_required:n = true,
2353   join-word/unknown   .code:n =
2354     \msg_error:nneee { spintenc } { unknown-choice }
2355     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2356   use-spnZ           .bool_set:N = \l__spintenc_spmQ_use_spnZ_bool,
2357   use-spnZ           .initial:n = true,
2358   use-spnZ           .value_required:n = true,
2359   print-dfrac        .bool_set:N = \l__spintenc_spmQ_print_dfrac_bool,
2360   print-dfrac        .initial:n = false,
2361   print-dfrac        .value_required:n = true,
2362   wrap-parens        .bool_set:N = \l__spintenc_spmQ_wrap_parens_bool,
2363   wrap-parens        .initial:n = false,
2364   wrap-parens        .value_required:n = true,
2365   read-parens        .bool_set:N = \l__spintenc_spmQ_read_parens_bool,
2366   read-parens        .initial:n = true,
2367   read-parens        .value_required:n = true,
2368   unknown            .code:n =
2369     {
2370       \str_set:Nn \l__spintenc_command_name_str { spmQ }
2371       \__spintenc_unknown_keys_cmd:n {#1}
2372     },
2373   }

```

(End of definition for `join-word` and others.)

11.27.4 Internal functions for \spmQ

`\__spintenc_spmQ_set_wrap_parens_size:`

The function `\__spintenc_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spintenc_spmQ_wrap_parens_l_tl` and `\l__spintenc_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2374 \cs_new_protected:Nn \__spintenc_spmQ_set_wrap_parens_size:
2375 {
2376   \int_case:nnF { \mathstyle }
2377   {
2378     { 0 }
2379     {
2380       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2381       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2382     }
2383     { 1 }
2384     {
2385       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2386       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2387     }
2388     { 2 }
2389     {
2390       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \bigl \lparen }
2391       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_r_tl { \bigr \rparen }
2392     }
2393     { 3 }
2394     {
2395       \tl_set:Nn \l__spintenc_spmQ_wrap_parens_l_tl { \bigl \lparen }

```

```

2396         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2397     }
2398 }
2399 {
2400     \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2401     \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2402 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2403     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2404     {
2405         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2406         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2407     }
2408 }

```

(End of definition for `\l__spintent_spmQ_set_wrap_parens_size:`)

```

\__spintent_spmQ_start_wrap_parens:
\__spintent_spmQ_stop_wrap_parens:

```

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2409 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2410 {
2411     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2412     {
2413         \__spintent_spmQ_set_wrap_parens_size:
2414         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2415         { \l__spintent_spmQ_wrap_parens_l_tl }
2416         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2417     }
2418 }
2419 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2420 {
2421     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2422     {
2423         \__spintent_spmQ_set_wrap_parens_size:
2424         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2425         { \l__spintent_spmQ_wrap_parens_r_tl }
2426         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2427     }
2428 }

```

(End of definition for `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:`)

```
\__spintent_spmQ_set_join_word:
```

The function `\__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2429 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2430 {
2431     \str_case:Vn \l__spintent_spmQ_join_word_str
2432     {
2433         { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_tl } }
2434         { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_tl } }
2435         { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_tl } }
2436     }
2437 }

```

(End of definition for `\__spintent_spmQ_set_join_word:`)

```
\__spintent_spmQ_print_scale_int:
```

The function `\__spintent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintent_luafun_spmQ_scale_int` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to 0 (skip completely). This prevents `luamml` from generating a

spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintent_saved_luamml_flag_int`.

The Lua function receives the integer digits from `\l__spintent_luaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2438 \cs_new_protected:Nn \__spintent_spmQ_print_scale_int:
2439 {
2440   \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { l__luamml_flag_int }
2441   \int_set:Nn \l__luamml_flag_int { 0 }
2442   \bool_if:NTF \l__spintent_spmQ_print_dfrac_bool
2443     { \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl { dfrac } }
2444     { \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl { frac } }
2445   \int_set_eq:cN { l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
2446 }

```

(End of definition for `\__spintent_spmQ_print_scale_int:`)

`\__spintent_spmQ_print_sacle_int:n`

The function `\__spintent_spmQ_print_sacle_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\__spintent_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintent_luaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintent_luaset_sign_tl`. When the sign is *empty*, it delegates directly to `\__spintent_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintent_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a MathML argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\__spintent_invisible_sep:` call between the sign and the `\MathMLarg` forces the opening of a new `<mrow>` in the MathML structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2447 \cs_new_protected:Nn \__spintent_spmQ_print_sacle_int:n
2448 {
2449   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #1 } }
2450   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2451   {
2452     \tl_if_empty:NTF \l__spintent_luaset_sign_tl
2453     {
2454       \__spintent_spmQ_print_scale_int:
2455     }
2456     {
2457       \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
2458       {
2459         \str_set:Nn
2460           \l__spintent_spmQ_intent_read_sign_str { _más:prefix($e) }
2461       }
2462       {
2463         \str_set:Nn
2464           \l__spintent_spmQ_intent_read_sign_str { _menos:prefix($e) }
2465       }
2466       \MathMLintent { \l__spintent_spmQ_intent_read_sign_str }
2467       {
2468         \l__spintent_luaset_sign_tl \__spintent_invisible_sep:
2469         \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
2470       }
2471     }
2472   }
2473   { \msg_error:nn { spintent } { spmQ-not-integer } }
2474 }

```

(End of definition for `\__spintent_spmQ_print_sacle_int:n`)

`\__spintent_spmQ_print_spnZ_dfrac:nn`

The function `\__spintent_spmQ_print_spnZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\__spintent_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintent_luaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintent_spmQ_use_spnZ_bool` and `\l__spintent_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spnZ` and `print-dfrac`. When `use-spnZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain MathML integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both

branches, the fraction command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2475 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2476 {
2477   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2478   \str_if_eq:VnT \__spintent_luaset_has_integer_str { false }
2479   { \msg_error:nn { spintent } { spmQ-not-integer } }
2480   \bool_if:NTF \__spintent_spmQ_use_spnZ_bool
2481   {
2482     \use:c { \bool_if:NT \__spintent_spmQ_print_dfrac_bool { d } frac }
2483     { \spnZ { #1 } } { \spnZ { #2 } }
2484   }
2485   {
2486     \use:c { \bool_if:NT \__spintent_spmQ_print_dfrac_bool { d } frac }
2487     { #1 } { #2 }
2488   }
2489 }

```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn` The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part ‘#1’, the numerator ‘#2’, and the denominator ‘#3’. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_sacle_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2490 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2491 {
2492   \__spintent_spmQ_start_wrap_parens:
2493   \__spintent_spmQ_print_sacle_int:n { #1 }
2494   \__spintent_spmQ_set_join_word:
2495   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2496   \__spintent_spmQ_stop_wrap_parens:
2497 }

```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn` The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional *(keys)* ‘#1’, the integer ‘#2’, the numerator ‘#3’, and the denominator ‘#4’. First sets the optional *(keys)* from ‘#1’, then processes the denominator input in ‘#4’ using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\__spintent_luaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2498 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2499 {
2500   \keys_set:nn { spintent / spmQ } { #1 }
2501   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2502   \str_if_eq:VnTF \__spintent_luaset_has_integer_str { true }
2503   {
2504     \int_compare:nNnTF { \__spintent_luaset_part_int_tl } = { 0 }
2505     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2506     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2507   }
2508   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2509 }

```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ` The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional *(keys)* and the *(arguments)*.

```

2510 \NewDocumentCommand \spmQ { O{} m m m }
2511 {
2512   \mode_if_math:TF
2513   {
2514     \group_begin:
2515     \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2516     \group_end:

```

```

2517     }
2518     { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2519 }

```

(End of definition for \spmQ. This function is documented on page 16.)

11.28 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\__spintent_geo_luaset_error_str`, `\__spintent_geo_luaset_intent_str` and `\__spintent_geo_luaset_print_tl`, constructing a valid MathCAT intent.

11.28.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\__spintent_geo_luaset_error_str
\__spintent_geo_luaset_intent_str
\__spintent_geo_luaset_print_tl
\__spintent_geo_luaset_is_name_str
\__spintent_geo_luaset_poly_sym_str
\__spintent_geo_luaset_intent_pts_str
\__spintent_geo_luaset_angulo_case_str
\__spintent_geo_side_cmd_tl
\__spintent_geo_fig_op_tl
2520 \str_new:N \__spintent_geo_luaset_error_str
2521 \str_new:N \__spintent_geo_luaset_intent_str
2522 \tl_new:N \__spintent_geo_luaset_print_tl
2523 \str_new:N \__spintent_geo_luaset_is_name_str
2524 \str_new:N \__spintent_geo_luaset_poly_sym_str
2525 \str_new:N \__spintent_geo_luaset_intent_pts_str
2526 \str_new:N \__spintent_geo_luaset_angulo_case_str
2527 \tl_new:N \__spintent_geo_side_cmd_tl
2528 \tl_new:N \__spintent_geo_fig_op_tl
2529 \str_new:N \__spintent_geo_luaset_concept_str
2530 \str_new:N \__spintent_geo_luaset_letra_word_str
2531 \str_new:N \__spintent_geo_luaset_sub_spoken_str

```

(End of definition for __spintent_geo_luaset_error_str and others.)

11.29 The command \spPoint

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.29.1 Definition of variables for \spPoint

Internal variables holding the values set by `read-cmd` and `prefix`.

```

\__spintent_spPoint_mode_str
\__spintent_spPoint_prefix_tl
2532 \str_new:N \__spintent_spPoint_mode_str
2533 \tl_new:N \__spintent_spPoint_prefix_tl

```

(End of definition for __spintent_spPoint_mode_str and __spintent_spPoint_prefix_tl.)

11.29.2 Definition of error messages for \spPoint

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘#1’ is not a single uppercase letter with an optional subscript or prime modifier.

```

2534 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2535 {
2536     Invalid~point~format~in~\token_to_str:N \spPoint:~'#1'.
2537 }
2538 {
2539     The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2540     subscript~(\_{...})~or~prime~(').\
2541 }

```

(End of definition for `spPoint-invalid-point`.)

11.29.3 Definition of keys for \spPoint

We define the (keys) `read-cmd`, `prefix` and `silent-cmd`.

```

read-cmd
prefix
silent-cmd
unknown
2542 \keys_define:nn { spintent / spPoint }
2543 {
2544     read-cmd .choices:nn = { school , mathcat }
2545     {
2546         \int_case:nn { \l_keys_choice_int }
2547         {
2548             {1} { \str_set:Nn \__spintent_spPoint_mode_str { school } }
2549             {2} { \str_set:Nn \__spintent_spPoint_mode_str { mathcat } }
2550         }
2551     },
2552     read-cmd .initial:n = school,
2553     read-cmd .value_required:n = true,
2554     read-cmd / unknown .code:n =
2555         \msg_error:nneee { spintent } { unknown-choice }
2556         { read-cmd } { school,~mathcat } { \exp_not:n {#1} },

```



```

2557     prefix                .code:n =
2558     { \__spintent_sanitiz
2559     prefix                .initial:n = ,
2560     prefix                .value_required:n = true,
2561     silent-cmd            .bool_set:N = \__spintent_spPoint_silent_cmd_bool,
2562     silent-cmd            .initial:n = false,
2563     silent-cmd            .value_required:n = true,
2564     unknown               .code:n =
2565     {
2566         \str_set:Nn \__spintent_command_name_str { spPoint }
2567         \__spintent_unknown_keys_cmd:n {#1}
2568     },
2569 }

```

(End of definition for read-cmd and others.)

11.29.4 Internal functions for \spPoint

__spintent_spPoint_print:n

The internal function `\__spintent_spPoint_print:n` calls `\__spintent_luafun_geo_point_parse_and_set:` which no longer receives the concept name: it always returns the bare body (e.g. A or A-sub-uno) in `\__spintent_geo_luaset_intent_str`. Only *after* the call does `expl3` assemble the final literal intent — prepending `punto-`, or wrapping the body in `_...` — when `silent-cmd` is active — since neither depends on anything Lua computes.

```

2570 \cs_new_protected:Nn \__spintent_spPoint_print:n
2571 {
2572     \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2573     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
2574     {
2575         \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
2576     }
2577     \str_if_eq:VnTF \__spintent_spPoint_mode_str { school }
2578     {
2579         \bool_if:NTF \__spintent_spPoint_silent_cmd_bool
2580         {
2581             \str_set:Ne \__spintent_geo_luaset_intent_str { _ \__spintent_geo_luaset_intent_str
2582         }
2583         {
2584             \str_set:Ne \__spintent_geo_luaset_intent_str { punto- \__spintent_geo_luaset_inten
2585         }
2586     }
2587     {
2588         \str_set:Ne \__spintent_geo_luaset_intent_str { _ punto :prefix($pt) }
2589         \bool_if:NT \__spintent_spPoint_silent_cmd_bool
2590         {
2591             \str_put_right:Nn \__spintent_geo_luaset_intent_str { :silent }
2592         }
2593     }
2594     \__spintent_invisible_sep: % need by MathCAT (bug in lualatex https://github.com/latex3/luamml/issues/26)
2595     \str_if_eq:VnTF \__spintent_spPoint_mode_str { school }
2596     {
2597         \MathMLintent
2598         {
2599             \tl_if_empty:NF \__spintent_spPoint_prefix_tl
2600             {
2601                 \tl_use:N \__spintent_spPoint_prefix_tl -
2602             }
2603             \__spintent_geo_luaset_intent_str
2604         }
2605         { \c__spintent_invisible_sep_tl \__spintent_geo_luaset_print_tl }
2606     }
2607     {
2608         \MathMLintent
2609         {
2610             \tl_if_empty:NF \__spintent_spPoint_prefix_tl
2611             {
2612                 \tl_use:N \__spintent_spPoint_prefix_tl -
2613             }
2614             \__spintent_geo_luaset_intent_str
2615         }
2616         {
2617             \c__spintent_invisible_sep_tl
2618             \MathMLarg { pt } { \__spintent_geo_luaset_print_tl }

```

```

2619     }
2620   }
2621 }

```

(End of definition for `\__spintent_spPoint_print:n`.)

`\__spintent_spPoint_exec:nn` The function `\__spintent_spPoint_exec:nn` receives the optional $\langle keys \rangle$ in ‘#1’ and the point in ‘#2’. It sets the keys via `\keys_set:nn`, then delegates rendering to `\__spintent_spPoint_print:n`.

```

2622 \cs_new_protected:Nn \__spintent_spPoint_exec:nn
2623 {
2624   \keys_set:nn { spintent / spPoint } { #1 }
2625   \__spintent_spPoint_print:n { #2 }
2626 }

```

(End of definition for `\__spintent_spPoint_exec:nn`.)

`\spPoint` The public command `\spPoint` is the interface for typesetting a single geometric point. It enforces math mode, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nn`.

```

2627 \NewDocumentCommand \spPoint { 0{} m }
2628 {
2629   \mode_if_math:TF
2630   {
2631     \group_begin:
2632     \__spintent_spPoint_exec:nn { #1 } { #2 }
2633     \group_end:
2634   }
2635   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
2636 }

```

(End of definition for `\spPoint`. This function is documented on page 17.)

11.30 The command `\sprecta`

The user command `\sprecta` supports two notations, detected automatically by the Lua parser: when the argument contains a comma, it is interpreted as two points and rendered with `\overleftarrow{\overrightarrow{\hspace{0.5em}}}`; when it contains no comma, it is interpreted as a proper name and typeset without any visual decorator. The shared variable `\l__spintent_geo_luaset_is_name_str` carries the detection result from Lua to the rendering function.

11.30.1 Definition of variables for `\sprecta`

`\l__spintent_sprecta_mode_str` These variables follow the same semantics as `\l__spintent_spPoint_mode_str` and `\l__spintent_spPoint_prefix_tl` (§11.29).

```

2637 \str_new:N \l__spintent_sprecta_mode_str
2638 \tl_new:N \l__spintent_sprecta_prefix_tl

```

(End of definition for `\l__spintent_sprecta_mode_str` and `\l__spintent_sprecta_prefix_tl`.)

11.30.2 Definition of error messages for `\sprecta`

`sprecta-invalid-points` The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

2639 \msg_new:nnnn { spintent } { sprecta-invalid-points }
2640 { Invalid~format~in~\token_to_str:N \sprecta:~'#1'. }
2641 {
2642   The~argument~must~be~either~a~comma-separated~pair~of~uppercase~
2643   letters~(two-point~notation)~or~a~single~letter~(uppercase~or~
2644   lowercase)~with~optional~subscript~or~prime~(name~notation).\\
2645 }

```

(End of definition for `sprecta-invalid-points`.)

11.30.3 Definition of keys for `\sprecta`

`read-cmd` We define the keys `read-cmd`, `prefix` and `silent-cmd`, following the same pattern as `\spPoint` (§11.29).

```

2646 \keys_define:nn { spintent / sprecta }
2647 {
2648   read-cmd .choices:nn = { school , mathcat }
2649   {
2650     \int_case:nn { \l_keys_choice_int }
2651     {
2652       {1} { \str_set:Nn \l__spintent_sprecta_mode_str { school } }
2653       {2} { \str_set:Nn \l__spintent_sprecta_mode_str { mathcat } }
2654     }
2655   },

```

```

2656   read-cmd          .initial:n = school,
2657   read-cmd          .value_required:n = true,
2658   read-cmd / unknown .code:n =
2659     \msg_error:nneee { spintent } { unknown-choice }
2660     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2661   prefix            .code:n =
2662     { \__spintent_sanitize_affix:Nn \__spintent_sprecta_prefix_tl {#1} },
2663   prefix            .initial:n = ,
2664   prefix            .value_required:n = true,
2665   silent-cmd        .bool_set:N = \__spintent_sprecta_silent_cmd_bool,
2666   silent-cmd        .initial:n = false,
2667   silent-cmd        .value_required:n = true,
2668   unknown           .code:n =
2669     {
2670       \str_set:Nn \__spintent_command_name_str { sprecta }
2671       \__spintent_unknown_keys_cmd:n {#1}
2672     },
2673 }

```

(End of definition for `read-cmd` and others.)

11.30.4 Internal functions for `\sprecta`

`\__spintent_sprecta_print:n` The internal function `\__spintent_sprecta_print:n` first sets `\l__spintent_geo_side_cmd_tl` to `recta` — a fixed literal, since `\sprecta` has no term-selecting key. Only then does it decide, according to mode and `silent-cmd`, whether to send `\l__spintent_geo_side_cmd_tl` or an empty group to `\__spintent_luafun_geo_parse_and_set:nnn`.

In `mathcat` mode, `:silent` is appended to `\l__spintent_geo_luaset_intent_str` afterward when needed. It then dispatches on `\l__spintent_geo_luaset_is_name_str` to select the visual branch: `\overleftarrow{\rightarrow}` for the two-point notation, or the raw name with no decorator otherwise.

```

2674 \cs_new_protected:Nn \__spintent_sprecta_print:n
2675 {
2676   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
2677   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2678   {
2679     \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
2680   }
2681   \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2682   {
2683     \bool_if:NTF \l__spintent_sprecta_silent_cmd_bool
2684     {
2685       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
2686     }
2687     {
2688       \str_set:Ne \l__spintent_geo_luaset_intent_str { recta- \l__spintent_geo_luaset_inten
2689     }
2690   }
2691   {
2692     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ recta :prefix($pts) }
2693     \bool_if:NT \l__spintent_sprecta_silent_cmd_bool
2694     {
2695       \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
2696     }
2697   }
2698   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
2699   \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
2700   {
2701     \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2702     {
2703       \MathMLintent
2704       {
2705         \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2706         {
2707           \tl_use:N \l__spintent_sprecta_prefix_tl -
2708         }
2709         \l__spintent_geo_luaset_intent_str
2710       }
2711       { \c__spintent_invisible_sep_tl \l__spintent_geo_luaset_print_tl }
2712     }
2713     {
2714       \MathMLintent
2715       {

```

```

2716         \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2717         {
2718             \tl_use:N \l__spintent_sprecta_prefix_tl -
2719         }
2720         \l__spintent_geo_luaset_intent_str
2721     }
2722     {
2723         \c__spintent_invisible_sep_tl
2724         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2725     }
2726 }
2727 }
2728 {
2729     \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
2730     {
2731         \MathMLintent
2732         {
2733             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2734             {
2735                 \tl_use:N \l__spintent_sprecta_prefix_tl -
2736             }
2737             \l__spintent_geo_luaset_intent_str
2738         }
2739         {
2740             \c__spintent_invisible_sep_tl
2741             \overleftrightharpoon { \l__spintent_geo_luaset_print_tl }
2742         }
2743     }
2744     {
2745         \MathMLintent
2746         {
2747             \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
2748             {
2749                 \tl_use:N \l__spintent_sprecta_prefix_tl -
2750             }
2751             \l__spintent_geo_luaset_intent_str
2752         }
2753         {
2754             \c__spintent_invisible_sep_tl
2755             \overleftrightharpoon
2756             {
2757                 \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2758             }
2759         }
2760     }
2761 }
2762 }

```

(End of definition for `\__spintent_sprecta_print:n`.)

`\__spintent_sprecta_exec:nn` Receives the optional $\langle keys \rangle$ in ‘#1’ and the argument in ‘#2’. Sets the keys via `\keys_set:nn`, then delegates rendering to `\__spintent_sprecta_print:n`.

```

2763 \cs_new_protected:Nn \__spintent_sprecta_exec:nn
2764 {
2765     \keys_set:nn { spintent / sprecta } { #1 }
2766     \__spintent_sprecta_print:n { #2 }
2767 }

```

(End of definition for `\__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces math mode, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

2768 \NewDocumentCommand \sprecta { 0{} m }
2769 {
2770     \mode_if_math:TF
2771     {
2772         \group_begin:
2773         \__spintent_sprecta_exec:nn { #1 } { #2 }
2774         \group_end:
2775     }
2776     { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
2777 }

```

(End of definition for `\sprecta`. This function is documented on page 18.)

11.31 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, `rayo` or `semirrecta`, both rendered identically with `\overrightarrow`.

11.31.1 Definition of variables for `\sprayo`

The first two follow the same semantics as `\__spintent_spPoint_mode_str` and `\__spintent_spPoint_prefix_tl` (§11.29). The variable `\__spintent_sprayo_read_sty_str` stores the term chosen by `read-sty`, and is passed directly to Lua as the concept name.

```
2778 \str_new:N \__spintent_sprayo_mode_str
2779 \tl_new:N \__spintent_sprayo_prefix_tl
2780 \str_new:N \__spintent_sprayo_read_sty_str
```

(End of definition for `\__spintent_sprayo_mode_str`, `\__spintent_sprayo_prefix_tl`, and `\__spintent_sprayo_read_sty_str`.)

11.31.2 Definition of error messages for `\sprayo`

sprayo-invalid-points

Raised when the argument is not a comma-separated pair of uppercase letters.

```
2781 \msg_new:nnnn { spintent } { sprayo-invalid-points }
2782 {
2783   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'.
2784 }
2785 {
2786   The~argument~must~be~a~comma-separated~list~of~exactly~two~
2787   uppercase~letters~with~optional~subscripts~(\_{...})~or~primes~(').\
2788 }
```

(End of definition for `sprayo-invalid-points`.)

11.31.3 Definition of keys for `\sprayo`

read-cmd
prefix
silent-cmd
read-sty
unknown

We define the `(keys)` `read-cmd`, `prefix`, `silent-cmd` and `read-sty`. The key `read-sty` selects the verbalized term: it defaults to `rayo` and accepts `semirrecta` as the alternative term.

```
2789 \keys_define:nn { spintent / sprayo }
2790 {
2791   read-cmd .choices:nn = { school , mathcat }
2792   {
2793     \int_case:nn { \l_keys_choice_int }
2794     {
2795       {1} { \str_set:Nn \__spintent_sprayo_mode_str { school } }
2796       {2} { \str_set:Nn \__spintent_sprayo_mode_str { mathcat } }
2797     }
2798   },
2799   read-cmd .initial:n = school,
2800   read-cmd .value_required:n = true,
2801   read-cmd / unknown .code:n =
2802     \msg_error:nneee { spintent } { unknown-choice }
2803     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2804   prefix .code:n =
2805     { \__spintent_sanitizé_affix:Nn \__spintent_sprayo_prefix_tl {#1} },
2806   prefix .initial:n = ,
2807   prefix .value_required:n = true,
2808   silent-cmd .bool_set:N = \__spintent_sprayo_silent_cmd_bool,
2809   silent-cmd .initial:n = false,
2810   silent-cmd .value_required:n = true,
2811   read-sty .choices:nn = { rayo , semirrecta }
2812   {
2813     \int_case:nn { \l_keys_choice_int }
2814     {
2815       {1} { \str_set:Nn \__spintent_sprayo_read_sty_str { rayo } }
2816       {2} { \str_set:Nn \__spintent_sprayo_read_sty_str { semirrecta } }
2817     }
2818   },
2819   read-sty .initial:n = rayo,
2820   read-sty .value_required:n = true,
2821   read-sty / unknown .code:n =
2822     \msg_error:nneee { spintent } { unknown-choice }
2823     { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
2824   unknown .code:n =
2825     {
2826       \str_set:Nn \__spintent_command_name_str { sprayo }
```

```

2827     \__spintent_unknown_keys_cmd:n {#1}
2828   },
2829 }

```

(End of definition for read-cmd and others.)

11.31.4 Internal functions for \sprayo

__spintent_sprayo_print:n

Resolves the concept name to \l__spintent_sprayo_read_sty_str or an empty group before calling Lua, following the same pattern as \sprecta (§11.30), and uses \overrightarrow as the visual decorator in both branches.

```

2830 \cs_new_protected:Nn \__spintent_sprayo_print:n
2831 {
2832   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
2833   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2834   {
2835     \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
2836   }
2837   \str_if_eq:VnTF \l__spintent_sprayo_mode_str { school }
2838   {
2839     \bool_if:NTF \l__spintent_sprayo_silent_cmd_bool
2840     {
2841       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
2842     }
2843     {
2844       \str_set:Ne \l__spintent_geo_luaset_intent_str
2845       {
2846         \l__spintent_sprayo_read_sty_str - \l__spintent_geo_luaset_intent_str
2847       }
2848     }
2849   }
2850   {
2851     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_sprayo_read_sty_str :pref
2852     \bool_if:NT \l__spintent_sprayo_silent_cmd_bool
2853     {
2854       \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
2855     }
2856   }
2857   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
2858   \str_if_eq:VnTF \l__spintent_sprayo_mode_str { school }
2859   {
2860     \MathMLintent
2861     {
2862       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
2863       {
2864         \tl_use:N \l__spintent_sprayo_prefix_tl -
2865       }
2866       \l__spintent_geo_luaset_intent_str
2867     }
2868     {
2869       \c__spintent_invisible_sep_tl
2870       \overrightarrow { \l__spintent_geo_luaset_print_tl }
2871     }
2872   }
2873   {
2874     \MathMLintent
2875     {
2876       \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
2877       {
2878         \tl_use:N \l__spintent_sprayo_prefix_tl -
2879       }
2880       \l__spintent_geo_luaset_intent_str
2881     }
2882     {
2883       \c__spintent_invisible_sep_tl
2884       \overrightarrow
2885       {
2886         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
2887       }
2888     }
2889   }
2890 }

```


(End of definition for `\__spintent_sprayo_print:n`.)

```
\__spintent_sprayo_exec:nn Sets the keys via \keys_set:nn, then delegates rendering to \__spintent_sprayo_print:n.
2891 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
2892 {
2893   \keys_set:nn { spintent / sprayo } { #1 }
2894   \__spintent_sprayo_print:n { #2 }
2895 }
```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo` Enforces math mode and delegates to `\__spintent_sprayo_exec:nn`.

```
2896 \NewDocumentCommand \sprayo { 0{} m }
2897 {
2898   \mode_if_math:TF
2899   {
2900     \group_begin:
2901     \__spintent_sprayo_exec:nn { #1 } { #2 }
2902     \group_end:
2903   }
2904   { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
2905 }
```

(End of definition for `\sprayo`. This function is documented on page 18.)

11.32 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — `lado`, `segmento`, or `trazo`, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. `\spmside` additionally provides `print-m`, controlling whether `\operatorname{m}` is typeset before the segment. Both commands are generated by a single generic backend over the short identifiers `side` and `mside`, via `\clist_map_inline:nn`, following the pattern already used by `\spmcd/\spmcm`.

11.32.1 Definition of variables for `\spside` and `\spmside`

```
\l__spintent_sp\#1_mode_str Each pair of variables is declared once for both side and mside via \clist_map_inline:nn. The _mode_str
\l__spintent_sp\#1_prefix_tl and _prefix_tl variables follow the same semantics as \l__spintent_spPoint_mode_str and \l__
\l__spintent_sp\#1_read_sty_str spintent_spPoint_prefix_tl (§11.29). The _read_sty_str variable stores the value set by read-sty;
\l__spintent_sp\#1_read_arg_str the _read_arg_str variable stores the optional override set by read-arg. Both are assigned through custom
.code:n bodies rather than a direct property, so they require explicit declaration here — unlike
the boolean keys defined further below.
```

```
2906 \clist_map_inline:nn { side , mside }
2907 {
2908   \str_new:c { \l__spintent_sp #1 _mode_str }
2909   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
2910   \str_new:c { \l__spintent_sp #1 _read_sty_str }
2911   \str_new:c { \l__spintent_sp #1 _read_arg_str }
2912 }
```

(End of definition for `\l__spintent_sp\#1_mode_str` and others.)

11.32.2 Definition of error messages for `\spside` and `\spmside`

```
spside-invalid-points
spmside-invalid-points
```

Both messages are generated by the same map. The message text uses `##1/##2` — doubled because it is written one level inside the map’s own `#1` substitution — for the command name and the offending argument, both supplied later when `\msg_error:nnee` is called.

```
2913 \clist_map_inline:nn { side , mside }
2914 {
2915   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
2916   {
2917     Invalid~point~format~in~##1:~'##2'.
2918   }
2919   {
2920     The~argument~must~be~a~comma-separated~list~of~uppercase~
2921     letters~with~optional~subscripts~(\_{...})~or~primes~(').\
2922   }
2923 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.32.3 Definition of keys for `\spside` and `\spmside`

The keyspaces `spintenc/spside` and `spintenc/spmside` are both generated by the same loop, each with its own independent set of keys pointing at its own `:c`-named variables. The keys `silent-cmd` and `print-m` use the direct `.bool_set:c` property, which `l3keys` declares automatically — no separate `\bool_new:c` call is needed for either. `print-m` defaults to `false`, matching the `\angle` symbol convention documented for `\spmangle` (§11.33).

```

2924 \clist_map_inline:nn { side , mside }
2925 {
2926   \keys_define:nn { spintenc / sp #1 }
2927   {
2928     read-cmd .choices:nn = { school , mathcat }
2929     {
2930       \int_case:nn { \l_keys_choice_int }
2931       {
2932         {1} { \str_set:cn { l__spintenc_sp #1 _mode_str } { school } }
2933         {2} { \str_set:cn { l__spintenc_sp #1 _mode_str } { mathcat } }
2934       }
2935     },
2936     read-cmd .initial:n = school,
2937     read-cmd .value_required:n = true,
2938     read-cmd / unknown .code:n =
2939       \msg_error:nneee { spintenc } { unknown-choice }
2940       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
2941     prefix .code:n =
2942       { \__spintenc_sanitizet_affix:cn { l__spintenc_sp #1 _prefix_tl } {##1} },
2943     prefix .initial:n = ,
2944     prefix .value_required:n = true,
2945     silent-cmd .bool_set:c = { l__spintenc_sp #1 _silent_cmd_bool },
2946     silent-cmd .initial:n = false,
2947     silent-cmd .value_required:n = true,
2948     read-sty .choices:nn = { lado , segmento , trazo }
2949     {
2950       \int_case:nn { \l_keys_choice_int }
2951       {
2952         {1} { \str_set:cn { l__spintenc_sp #1 _read_sty_str } { lado } }
2953         {2} { \str_set:cn { l__spintenc_sp #1 _read_sty_str } { segmento } }
2954         {3} { \str_set:cn { l__spintenc_sp #1 _read_sty_str } { trazo } }
2955       }
2956     },
2957     read-sty .initial:n = lado,
2958     read-sty .value_required:n = true,
2959     read-sty / unknown .code:n =
2960       \msg_error:nneee { spintenc } { unknown-choice }
2961       { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
2962     read-arg .code:n =
2963       { \str_set:cn { l__spintenc_sp #1 _read_arg_str } {##1} },
2964     read-arg .initial:n = ,
2965     read-arg .value_required:n = true,
2966     print-m .bool_set:c = { l__spintenc_sp #1 _print_m_bool },
2967     print-m .initial:n = false,
2968     print-m .value_required:n = true,
2969     unknown .code:n =
2970     {
2971       \str_set:Nn \l__spintenc_command_name_str { sp #1 }
2972       \__spintenc_unknown_keys_cmd:n {##1}
2973     },
2974   }
2975 }

```

(End of definition for `read-cmd` and others.)

11.32.4 Internal functions for `\spside` and `\spmside`

`\__spintenc_spside_process:nn`

The internal function `\__spintenc_spside_process:nn` takes the short identifier `side/mside` in ‘#1’ and the point list in ‘#2’. It first resolves the term into `\l__spintenc_geo_side_cmd_tl`: `l__spintenc_sp#1_read_arg` when non-empty, otherwise `l__spintenc_sp#1_read_sty_str`. When #1 is `mside`, `medida-del-` is prepended. Only then does it decide, according to mode and `silent-cmd`, whether to send that resolved term or an empty group to `\__spintenc_luafun_geo_parse_and_set:nnn`.

In `mathcat` mode, `:silent` is appended to `\l__spintenc_geo_luaaset_intent_str` afterward when needed. When #1 is `mside` and `print-m` is `true`, `\operatorname{m}` is typeset wrapped in `\MathMLintenc{<:silent>}`.

before `\overline`, following the same pause-avoiding pattern already used for `\spmarc` (§11.34).

```

2976 \cs_new_protected:Nn \__spintent_spside_process:nn
2977 {
2978   \tl_set:Nn \l__spintent_geo_side_cmd_tl
2979   {
2980     \str_if_empty:cTF { \l__spintent_sp #1 _read_arg_str }
2981     { \tl_use:c { \l__spintent_sp #1 _read_sty_str } }
2982     { \tl_use:c { \l__spintent_sp #1 _read_arg_str } }
2983   }
2984   \str_if_eq:eeT {#1} { mside }
2985   {
2986     \tl_put_left:Nn \l__spintent_geo_side_cmd_tl { medida-del- }
2987   }
2988   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
2989   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
2990   {
2991     \msg_error:nnee { spintent } { sp #1 -invalid-points }
2992     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
2993   }
2994   \str_if_eq:eeTF { \str_use:c { \l__spintent_sp #1 _mode_str } } { school }
2995   {
2996     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
2997     {
2998       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
2999     }
3000     {
3001       \str_set:Ne \l__spintent_geo_luaset_intent_str
3002       {
3003         \l__spintent_geo_side_cmd_tl - \l__spintent_geo_luaset_intent_str
3004       }
3005     }
3006   }
3007   {
3008     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_side_cmd_tl :prefix($
3009     \bool_if:cT { \l__spintent_sp #1 _silent_cmd_bool }
3010     {
3011       \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
3012     }
3013   }
3014   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3015   \str_if_eq:eeTF { \str_use:c { \l__spintent_sp #1 _mode_str } } { school }
3016   {
3017     \MathMLintent
3018     {
3019       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3020       {
3021         \tl_use:c { \l__spintent_sp #1 _prefix_tl } -
3022       }
3023       \l__spintent_geo_luaset_intent_str
3024     }
3025     {
3026       \c__spintent_invisible_sep_tl
3027       \str_if_eq:eeTF {#1} { side }
3028       { \overline { \l__spintent_geo_luaset_print_tl } }
3029       {
3030         \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
3031         {
3032           \MathMLintent { :silent } { \operatorname { m } } }
3033         }
3034         \l__spintent_geo_luaset_print_tl
3035       }
3036     }
3037   }
3038   {
3039     \MathMLintent
3040     {
3041       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3042       {
3043         \tl_use:c { \l__spintent_sp #1 _prefix_tl } -
3044       }
3045       \l__spintent_geo_luaset_intent_str

```

```

3046     }
3047     {
3048       \c__spintent_invisible_sep_tl
3049       \str_if_eq:eeTF {#1} { side }
3050       { \overline { \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl } } }
3051       {
3052         \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3053         {
3054           \MathMLintent { :silent } { \operatorname { m } }
3055         }
3056         \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3057       }
3058     }
3059   }
3060 }

```

(End of definition for `\__spintent_spside_process:nn`.)

`\__spintent_spside_exec:nnn` Takes the short identifier in ‘#1’, the optional $\langle\textit{keys}\rangle$ in ‘#2’, and the point list in ‘#3’. Sets the keys in the correct key space via `\keys_set:nn` — the key space name is plain text substituted from ‘#1’, not a csname, so no `:c` variant is required here.

```

3061 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3062 {
3063   \keys_set:nn { spintent / sp #1 } { #2 }
3064   \__spintent_spside_process:nn { #1 } { #3 }
3065 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside` Both public commands are thin wrappers around `\__spintent_spside_exec:nnn`, passing their own short identifier as the first argument.

`\spmside`

```

3066 \NewDocumentCommand \spside { O{} m }
3067 {
3068   \mode_if_math:TF
3069   {
3070     \group_begin:
3071     \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3072     \group_end:
3073   }
3074   { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3075 }
3076 \NewDocumentCommand \spmside { O{} m }
3077 {
3078   \mode_if_math:TF
3079   {
3080     \group_begin:
3081     \__spintent_spside_exec:nnn { mside } { #1 } { #2 }
3082     \group_end:
3083   }
3084   { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3085 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 18.)

11.33 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle` — renamed from the earlier `\spangulo`/`\spmangulo` — provide a semantic interface for angles, following the same generic pattern as `\spside`/`\spmside`. The key `print-m` chooses between the `\angle`/`\rightangle` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle`/`\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.33.1 Definition of variables for `\spangle` and `\spmangle`

`\l__spintent_sp\#1_mode_str` Declared once for both `angle` and `mangle` via `\clist_map_inline:nn`. These follow the same semantics as `\l__spintent_spPoint_mode_str` and `\l__spintent_spPoint_prefix_tl` (§11.29).

`\l__spintent_sp\#1_prefix_tl`

```

3086 \clist_map_inline:nn { angle , mangle }
3087 {
3088   \str_new:c { l__spintent_sp #1 _mode_str }
3089   \tl_new:c { l__spintent_sp #1 _prefix_tl }
3090 }

```

(End of definition for `\l__spintent_sp\#1_mode_str` and `\l__spintent_sp\#1_prefix_tl`.)

11.33.2 Definition of error messages for \spangle and \spmangle

spangle-invalid-arg
spmangle-invalid-arg

Both messages are generated by the same map, using the same ##1/##2 doubling convention as \spside/\spmside (§11.32).

```

3091 \clist_map_inline:nn { angle , mangle }
3092 {
3093   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3094   { Invalid~format~in~##1:~'##2'. }
3095   {
3096     The~argument~must~be~one~of:\\
3097     - three~uppercase~points~separated~by~commas~(e.g.\ 'A,~O,~B');\\
3098     - a~single~uppercase~point~with~optional~subscript~or~prime~
3099     (e.g.\ 'A',~'A_{1}')\\
3100     - an~angle~name~(e.g.\ '\string\alpha',~'1',~'ABC').
3101   }
3102 }
```

(End of definition for spangle-invalid-arg and spmangle-invalid-arg.)

11.33.3 Definition of keys for \spangle and \spmangle

read-cmd
prefix
silent-cmd
recto
print-m
unknown

The keys `silent-cmd`, `recto` and `print-m` all use the direct `.bool_set:c` property, auto-declared by `l3keys`; none needs a separate `\bool_new:c` call.

```

3103 \clist_map_inline:nn { angle , mangle }
3104 {
3105   \keys_define:nn { spintent / sp #1 }
3106   {
3107     read-cmd .choices:nn = { school , mathcat }
3108     {
3109       \int_case:nn { \l_keys_choice_int }
3110       {
3111         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3112         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3113       }
3114     },
3115     read-cmd .initial:n = school,
3116     read-cmd .value_required:n = true,
3117     read-cmd / unknown .code:n =
3118     \msg_error:nnee { spintent } { unknown-choice }
3119     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3120     prefix .code:n =
3121     { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3122     prefix .initial:n = ,
3123     prefix .value_required:n = true,
3124     silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3125     silent-cmd .initial:n = false,
3126     silent-cmd .value_required:n = true,
3127     recto .bool_set:c = { l__spintent_sp #1 _recto_bool },
3128     recto .initial:n = false,
3129     recto .value_required:n = true,
3130     print-m .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3131     print-m .initial:n = false,
3132     print-m .value_required:n = true,
3133     unknown .code:n =
3134     {
3135       \str_set:Nn \l__spintent_command_name_str { sp #1 }
3136       \__spintent_unknown_keys_cmd:n {##1}
3137     },
3138   }
3139 }
```

(End of definition for read-cmd and others.)

11.33.4 Internal functions for \spangle and \spmangle

__spintent_spangle_sym:n

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle/\rightangle` when `print-m` is `true`; `\measuredangle/\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3140 \cs_new_protected:Nn \__spintent_spangle_sym:n
3141 {
3142   \str_if_eq:eeTF {#1} { mangle }
3143   {
3144     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
```

```

3145     {
3146       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3147       { \__spintent_right_angle_sqr_sym: } { \angle }
3148     }
3149     {
3150       \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3151       { \measuredrightangle } { \measuredangle }
3152     }
3153   }
3154   {
3155     \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3156     { \__spintent_right_angle_sqr_sym: } { \angle }
3157   }
3158 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\__spintent_spangle_process:nn`

Resolves the base name — `ángulo` or `medida-del-ángulo` — into `\l__spintent_geo_side_cmd_tl`, appends `-recto` when `l__spintent_sp#1_recto_bool` is `true`, then applies the same silent-cmd resolution as `\__spintent_spside_process:nn` (§11.32) before calling `\__spintent_luafun_geo_angulo_parse_and_set:n`.

```

3159 \cs_new_protected:Nn \__spintent_spangle_process:nn
3160 {
3161   \str_if_eq:eeTF {#1} { mangle }
3162   { \tl_set:Nn \l__spintent_geo_side_cmd_tl { medida-del-ángulo } }
3163   { \tl_set:Nn \l__spintent_geo_side_cmd_tl { ángulo } }
3164   \bool_if:cT { l__spintent_sp #1 _recto_bool }
3165   {
3166     \tl_put_right:Nn \l__spintent_geo_side_cmd_tl { -recto }
3167   }
3168   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3169   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3170   {
3171     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3172     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3173   }
3174   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3175   {
3176     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3177     {
3178       \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_luaset_intent_str
3179     }
3180     {
3181       \str_set:Ne \l__spintent_geo_luaset_intent_str
3182       {
3183         \l__spintent_geo_side_cmd_tl - \l__spintent_geo_luaset_intent_str
3184       }
3185     }
3186   }
3187   {
3188     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_side_cmd_tl :prefix($
3189     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
3190     {
3191       \str_put_right:Nn \l__spintent_geo_luaset_intent_str { :silent }
3192     }
3193   }
3194   \__spintent_invisible_sep: % need by MathCAT (bug in lualatex https://github.com/latex3/luamml/issues/26)
3195   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3196   {
3197     \MathMLintent
3198     {
3199       \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3200       {
3201         \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3202       }
3203       \l__spintent_geo_luaset_intent_str
3204     }
3205     {
3206       \str_if_eq:eeT {#1} { mangle }
3207       {
3208         \bool_if:cT { l__spintent_sp #1 _print_m_bool }
3209         {

```

```

3210         \MathMLintent { :silent } { \operatorname { m } } }
3211     }
3212 }
3213 \__spinttent_spangle_sym:n {#1}
3214 \l__spinttent_geo_luaset_print_tl
3215 }
3216 }
3217 {
3218   \MathMLintent
3219   {
3220     \tl_if_empty:cF { l__spinttent_sp #1 _prefix_tl }
3221     {
3222       \tl_use:c { l__spinttent_sp #1 _prefix_tl } -
3223     }
3224     \l__spinttent_geo_luaset_intent_str
3225   }
3226   {
3227     \str_if_eq:eeT {#1} { mangle }
3228     {
3229       \bool_if:cT { l__spinttent_sp #1 _print_m_bool }
3230       {
3231         \MathMLintent { :silent } { \operatorname { m } } }
3232       }
3233     }
3234     \__spinttent_spangle_sym:n {#1}
3235     \MathMLarg { pts } { \l__spinttent_geo_luaset_print_tl }
3236   }
3237 }
3238 }

```

(End of definition for `\__spinttent_spangle_process:nn`.)

`\__spinttent_spangle_exec:nnn` Sets the keys in the correct keyspace via `\keys_set:nn`, then delegates rendering to `\__spinttent_spangle_process:nnn`.

```

3239 \cs_new_protected:Nn \__spinttent_spangle_exec:nnn
3240 {
3241   \keys_set:nn { spinttent / sp #1 } { #2 }
3242   \__spinttent_spangle_process:nn { #1 } { #3 }
3243 }

```

(End of definition for `\__spinttent_spangle_exec:nnn`.)

`\spangle` Both public commands are thin wrappers around `\__spinttent_spangle_exec:nnn`, passing their own short identifier as the first argument.

`\spmangle`

```

3244 \NewDocumentCommand \spangle { 0{} m }
3245 {
3246   \mode_if_math:TF
3247   {
3248     \group_begin:
3249     \__spinttent_spangle_exec:nnn { angle } { #1 } { #2 }
3250     \group_end:
3251   }
3252   { \msg_error:nnn { spinttent } { need-math-mode } { \spangle } }
3253 }
3254 \NewDocumentCommand \spmangle { 0{} m }
3255 {
3256   \mode_if_math:TF
3257   {
3258     \group_begin:
3259     \__spinttent_spangle_exec:nnn { mangle } { #1 } { #2 }
3260     \group_end:
3261   }
3262   { \msg_error:nnn { spinttent } { need-math-mode } { \spmangle } }
3263 }

```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 20.)

11.34 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` — renamed from `\sparco`/`\spmarco` — provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.34.1 Definition of variables for \sparc and \spmarc

Declared once for both arc and marc via \clist_map_inline:nn. These follow the same semantics as \l__spinttent_spPoint_mode_str and \l__spinttent_spPoint_prefix_tl (§11.29).

```
3264 \clist_map_inline:nn { arc , marc }
3265 {
3266   \str_new:c { l__spinttent_sp #1 _mode_str }
3267   \tl_new:c { l__spinttent_sp #1 _prefix_tl }
3268 }
```

(End of definition for l__spinttent_sparc_mode_str and others.)

11.34.2 Definition of error messages for \sparc and \spmarc

sparc-invalid-points
spmarc-invalid-points

Both messages are generated by the same map, using the same ##1/##2 doubling convention as \spside/\spmside (§11.32).

```
3269 \clist_map_inline:nn { arc , marc }
3270 {
3271   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
3272   {
3273     Invalid~point~format~in~##1:~'##2'.
3274   }
3275   {
3276     The~argument~must~be~a~comma-separated~list~of~uppercase~
3277     letters~with~optional~subscripts~(\_{...})~or~primes~(').\\
3278   }
3279 }
```

(End of definition for sparc-invalid-points and spmarc-invalid-points.)

11.34.3 Definition of keys for \sparc and \spmarc

read-cmd
prefix
silent-cmd
unknown

The keyspaces spinttent/sparc and spinttent/spmarc are both generated by the same loop, each with its own independent set of keys pointing at its own :c-named variables.

```
3280 \clist_map_inline:nn { arc , marc }
3281 {
3282   \keys_define:nn { spinttent / sp #1 }
3283   {
3284     read-cmd .choices:nn = { school , mathcat }
3285     {
3286       \int_case:nn { \l_keys_choice_int }
3287       {
3288         {1} { \str_set:cn { l__spinttent_sp #1 _mode_str } { school } }
3289         {2} { \str_set:cn { l__spinttent_sp #1 _mode_str } { mathcat } }
3290       }
3291     },
3292     read-cmd .initial:n = school,
3293     read-cmd .value_required:n = true,
3294     read-cmd / unknown .code:n =
3295       \msg_error:nneee { spinttent } { unknown-choice }
3296       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3297     prefix .code:n =
3298       { \__spinttent_sanitise_affix:cn { l__spinttent_sp #1 _prefix_tl } {##1} },
3299     prefix .initial:n = ,
3300     prefix .value_required:n = true,
3301     silent-cmd .bool_set:c = { l__spinttent_sp #1 _silent_cmd_bool },
3302     silent-cmd .initial:n = false,
3303     silent-cmd .value_required:n = true,
3304     unknown .code:n =
3305       {
3306         \str_set:Nn \l__spinttent_command_name_str { sp #1 }
3307         \__spinttent_unknown_keys_cmd:n {##1}
3308       },
3309   }
3310 }
```

(End of definition for read-cmd and others.)

11.34.4 Internal functions for \sparc and \spmarc

The function __spinttent_sparc_wide_arc:n acts as a fallback when the mathematical font used in the document does not provide the command \widearc. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```
3311 \cs_new_protected_nopar:Nn \__spinttent_sparc_wide_arc:n
```

```

3312 {
3313   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
3314 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`.)

`\__spintent_sparc_print_arc:n`

The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

3315 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
3316 {
3317   \cs_if_exist:NTF \widearc
3318   { \widearc {#1} }
3319   { \__spintent_sparc_wide_arc:n {#1} }
3320 }

```

(End of definition for `\__spintent_sparc_print_arc:n`.)

`\__spintent_sparc_process:nn`

Follows the same resolution pattern as `\__spintent_spside_process:nn` (§11.32), with the base names `arco`/`medida-del-arco`. When `#1` is `marc`, `\operatorname{m}` is always typeset, wrapped in `\MathMLIntant{<:silent>}` before `\__spintent_sparc_print_arc:n`.

```

3321 \cs_new_protected:Nn \__spintent_sparc_process:nn
3322 {
3323   \str_if_eq:eeTF {#1} { marc }
3324   { \tl_set:Nn \__spintent_geo_side_cmd_tl { medida-del-arco } }
3325   { \tl_set:Nn \__spintent_geo_side_cmd_tl { arco } }
3326   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3327   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3328   {
3329     \msg_error:nnee { spintent } { sp #1 -invalid-points }
3330     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3331   }
3332   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3333   {
3334     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
3335     {
3336       \str_set:Ne \__spintent_geo_luaset_intent_str { _ \__spintent_geo_luaset_intent_str
3337     }
3338     {
3339       \str_set:Ne \__spintent_geo_luaset_intent_str
3340       {
3341         \__spintent_geo_side_cmd_tl - \__spintent_geo_luaset_intent_str
3342       }
3343     }
3344   }
3345   {
3346     \str_set:Ne \__spintent_geo_luaset_intent_str { _ \__spintent_geo_side_cmd_tl :prefix($
3347     \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
3348     {
3349       \str_put_right:Nn \__spintent_geo_luaset_intent_str { :silent }
3350     }
3351   }
3352   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3353   \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3354   {
3355     \MathMLIntant
3356     {
3357       \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3358       {
3359         \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3360       }
3361       \__spintent_geo_luaset_intent_str
3362     }
3363     {
3364       \__spintent_invisible_sep_tl
3365       \str_if_eq:eeT {#1} { marc }
3366       {
3367         \MathMLIntant { :silent } { \operatorname { m } }
3368       }
3369       \__spintent_sparc_print_arc:n { \__spintent_geo_luaset_print_tl }
3370     }

```

```

3371     }
3372     {
3373       \MathMLintent
3374       {
3375         \tl_if_empty:cF { \__spintenc_sp #1 _prefix_tl }
3376         { \tl_use:c { \__spintenc_sp #1 _prefix_tl } - }
3377         \l__spintenc_geo_luaset_intent_str
3378       }
3379       {
3380         \c__spintenc_invisible_sep_tl
3381         \str_if_eq:eeT {#1} { marc }
3382         {
3383           \MathMLintent { :silent } { \operatorname { m } }
3384         }
3385         \__spintenc_sparc_print_arc:n
3386         {
3387           \MathMLarg { pts } { \l__spintenc_geo_luaset_print_tl }
3388         }
3389       }
3390     }
3391   }

```

(End of definition for __spintenc_sparc_process:nn.)

__spintenc_sparc_exec:nnn Sets the keys in the correct keyspace via \keys_set:nn, then delegates rendering to __spintenc_sparc_process:nn.

```

3392 \cs_new_protected:Nn \__spintenc_sparc_exec:nnn
3393 {
3394   \keys_set:nn { spintenc / sp #1 } { #2 }
3395   \__spintenc_sparc_process:nn { #1 } { #3 }
3396 }

```

(End of definition for __spintenc_sparc_exec:nnn.)

\sparc Both public commands are thin wrappers around __spintenc_sparc_exec:nnn, passing their own short identifier as the first argument.

\spmarc

```

3397 \NewDocumentCommand \sparc { O{} m }
3398 {
3399   \mode_if_math:TF
3400   {
3401     \group_begin:
3402       \__spintenc_sparc_exec:nnn { arc } { #1 } { #2 }
3403     \group_end:
3404   }
3405   { \msg_error:nnn { spintenc } { need-math-mode } { \sparc } }
3406 }
3407 \NewDocumentCommand \spmarc { O{} m }
3408 {
3409   \mode_if_math:TF
3410   {
3411     \group_begin:
3412       \__spintenc_sparc_exec:nnn { marc } { #1 } { #2 }
3413     \group_end:
3414   }
3415   { \msg_error:nnn { spintenc } { need-math-mode } { \spmarc } }
3416 }

```

(End of definition for \sparc and \spmarc. These functions are documented on page 21.)

11.35 The command \spPoly

The user command \spPoly typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: \l__spintenc_geo_luaset_intent_str (the full intent, with the figure name) and \l__spintenc_geo_luaset_intent_pts_str (vertices only) are both returned unconditionally, and expl3 picks between them according to print-sym.

11.35.1 Definition of variables for \spPoly

\l__spintenc_spPoly_mode_str These follow the same semantics as \l__spintenc_spPoint_mode_str and \l__spintenc_spPoint_prefix_tl (§11.29).

```

3417 \str_new:N \l__spintenc_spPoly_mode_str
3418 \tl_new:N \l__spintenc_spPoly_prefix_tl

```

(End of definition for \l__spintenc_spPoly_mode_str and \l__spintenc_spPoly_prefix_tl.)

11.35.2 Definition of error messages for \spPoly

spPoly-invalid-points

The “error” spPoly-invalid-points is raised when the argument in ‘#1’ is not a comma-separated list of at least three uppercase letters.

```

3419 \msg_new:nnnn { spintrent } { spPoly-invalid-points }
3420 {
3421   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'.
3422 }
3423 {
3424   The~argument~must~be~a~comma-separated~list~of~at~least~three~
3425   uppercase~letters~with~optional~subscripts~(\_{...})~or~primes~(').\
3426 }
```

(End of definition for spPoly-invalid-points.)

11.35.3 Definition of keys for \spPoly

read-cmd

prefix

print-sym

silent-cmd

unknown

The key print-sym uses the direct .bool_set:N property, auto-declared by l3keys. The key silent-cmd is a \keys alias for print-sym=false.

```

3427 \keys_define:nn { spintrent / spPoly }
3428 {
3429   read-cmd          .choices:nn = { school , mathcat }
3430   {
3431     \int_case:nn { \l_keys_choice_int }
3432     {
3433       {1} { \str_set:Nn \l__spintrent_spPoly_mode_str { school } }
3434       {2} { \str_set:Nn \l__spintrent_spPoly_mode_str { mathcat } }
3435     }
3436   },
3437   read-cmd          .initial:n = school,
3438   read-cmd          .value_required:n = true,
3439   read-cmd / unknown .code:n =
3440     \msg_error:nneee { spintrent } { unknown-choice }
3441     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3442   prefix            .code:n =
3443     { \__spintrent_sanitise_affix:Nn \l__spintrent_spPoly_prefix_tl {#1} },
3444   prefix            .initial:n = ,
3445   prefix            .value_required:n = true,
3446   print-sym         .bool_set:N = \l__spintrent_spPoly_print_sym_bool,
3447   print-sym         .initial:n = true,
3448   print-sym         .value_required:n = true,
3449   silent-cmd        .meta:n = { print-sym = false },
3450   unknown           .code:n =
3451     {
3452       \str_set:Nn \l__spintrent_command_name_str { spPoly }
3453       \__spintrent_unknown_keys_cmd:n {#1}
3454     },
3455 }
```

(End of definition for read-cmd and others.)

11.35.4 Internal functions for \spPoly

__spintrent_spPoly_sym:

Reads \l__spintrent_geo_luaset_poly_sym_str and typesets the corresponding symbol: \triangle for triangle, \mdlgwhtsquare for square.

```

3456 \cs_new_protected:Nn \__spintrent_spPoly_sym:
3457 {
3458   \str_case:Vn \l__spintrent_geo_luaset_poly_sym_str
3459   {
3460     { triangle } { \triangle }
3461     { square } { \mdlgwhtsquare }
3462   }
3463 }
```

(End of definition for __spintrent_spPoly_sym:.)

__spintrent_spPoly_print:n

When print-sym is true, uses \l__spintrent_geo_luaset_intent_str and wraps __spintrent_spPoly_sym: in \MathMLintent{<:silent>} whenever a symbol is available; when false, uses \l__spintrent_geo_luaset_intent_pts_str and typesets no symbol at all.

```

3464 \cs_new_protected:Nn \__spintrent_spPoly_print:n
3465 {
3466   \__spintrent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
3467   \str_if_eq:VnT \l__spintrent_geo_luaset_error_str { true }
3468   {
```

```

3469     \msg_error:nne { spintent } { spPoly-invalid-points }
3470     { \exp_not:n {#1} }
3471   }
3472   \str_if_eq:VnT \l__spintent_spPoly_mode_str { mathcat }
3473   {
3474     \str_set:Nx \l__spintent_geo_luaset_intent_str
3475     { _ \l__spintent_geo_luaset_concept_str :prefix($pts) }
3476   }
3477   \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3478   \bool_if:NTF \l__spintent_spPoly_print_sym_bool
3479   {
3480     \str_if_eq:VnTF \l__spintent_spPoly_mode_str { school }
3481     {
3482       \MathMLintent
3483       {
3484         \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3485         { \tl_use:N \l__spintent_spPoly_prefix_tl - }
3486         \l__spintent_geo_luaset_intent_str
3487       }
3488       {
3489         \c__spintent_invisible_sep_tl
3490         \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
3491         {
3492           \MathMLintent { :silent } { \__spintent_spPoly_sym: }
3493         }
3494         \l__spintent_geo_luaset_print_tl
3495       }
3496     }
3497   }
3498   \MathMLintent
3499   {
3500     \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3501     {
3502       \tl_use:N \l__spintent_spPoly_prefix_tl -
3503     }
3504     \l__spintent_geo_luaset_intent_str
3505   }
3506   {
3507     \c__spintent_invisible_sep_tl
3508     \str_if_empty:NF \l__spintent_geo_luaset_poly_sym_str
3509     {
3510       \MathMLintent { :silent } { \__spintent_spPoly_sym: }
3511     }
3512     \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3513   }
3514 }
3515 }
3516 {
3517   \str_if_eq:VnTF \l__spintent_spPoly_mode_str { school }
3518   {
3519     \MathMLintent
3520     {
3521       \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3522       { \tl_use:N \l__spintent_spPoly_prefix_tl - }
3523       \l__spintent_geo_luaset_intent_pts_str
3524     }
3525     { \c__spintent_invisible_sep_tl \l__spintent_geo_luaset_print_tl }
3526   }
3527   {
3528     \MathMLintent
3529     {
3530       \tl_if_empty:NF \l__spintent_spPoly_prefix_tl
3531       { \tl_use:N \l__spintent_spPoly_prefix_tl - }
3532       \l__spintent_geo_luaset_intent_pts_str
3533     }
3534     {
3535       \c__spintent_invisible_sep_tl
3536       \MathMLarg { pts } { \l__spintent_geo_luaset_print_tl }
3537     }
3538   }
3539 }

```

```
3540 }
```

(End of definition for `\__spintent_spPoly_print:n`.)

`\__spintent_spPoly_exec:nn` Sets the keys via `\keys_set:nn`, then delegates rendering to `\__spintent_spPoly_print:n`.

```
3541 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
3542 {
3543   \keys_set:nn { spintent / spPoly } { #1 }
3544   \__spintent_spPoly_print:n { #2 }
3545 }
```

(End of definition for `\__spintent_spPoly_exec:nn`.)

`\spPoly` Enforces math mode and delegates to `\__spintent_spPoly_exec:nn`.

```
3546 \NewDocumentCommand \spPoly { 0{} m }
3547 {
3548   \mode_if_math:TF
3549   {
3550     \group_begin:
3551     \__spintent_spPoly_exec:nn { #1 } { #2 }
3552     \group_end:
3553   }
3554   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
3555 }
```

(End of definition for `\spPoly`. This function is documented on page 22.)

11.36 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcn`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{<:silent>}` — the same mechanism already used by `\spPoly` (§11.35).

11.36.1 Definition of variables for `\spAfig` and `\spPfig`

The `_read_arg_str` variable stores the figure-name override set by `read-arg`, independent of `print-sym`.

```
3556 \tl_new:N \l__spintent_geo_sym_word_tl
3557 \tl_new:N \l__spintent_geo_final_print_tl
3558 \str_new:N \l__spintent_geo_final_intent_str
3559 \str_new:N \l__spintent_geo_mathcat_concept_str
3560 \clist_map_inline:nn { Afig , Pfig }
3561 {
3562   \str_new:c { l__spintent_sp #1 _mode_str }
3563   \str_new:c { l__spintent_sp #1 _print_sym_str }
3564   \tl_new:c { l__spintent_sp #1 _prefix_tl }
3565   \tl_new:c { l__spintent_sp #1 _read_arg_tl }
3566 }
```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

11.36.2 Definition of error messages for `\spAfig` and `\spPfig`

`spAfig-invalid-points` Both messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside` (§11.32).

```
3567 \clist_map_inline:nn { Afig , Pfig }
3568 {
3569   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
3570   {
3571     Invalid~point~format~in~##1:~'##2'.
3572   }
3573   {
3574     The~argument~must~be~empty,~or~a~comma-separated~list~of~at~
3575     least~three~uppercase~letters~with~optional~subscripts~
3576     (\_{...})~or~primes~(').\
3577   }
3578   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3579   { Invalid~argument~format~in~##1:~'##2'. }
3580   {
3581     The~argument~must~be~either~empty,~a~comma-separated~list~of~
3582     vertices,~a~bare~number~or~letter~(e.g.\ '1',~'n'),~or~a~
3583     table~letter~with~an~optional~subscript~(e.g.\ 'F',~'F_{1}\').
3584   }
}
```

```

3585 \msg_new:nnnn { spintent } { sp #1 -sym-lettra-conflict }
3586 { print-sym~cannot~be~combined~with~a~lettered~argument~in~##1. }
3587 {
3588   When~print-sym~is~not~'none',~the~argument~(if~given)~must~
3589   be~a~bare~number~or~letter~(e.g.~'1',~'n')~--table~letters~
3590   like~'F'~or~'P'~are~not~allowed~in~combination~with~print-sym.
3591 }
3592 \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
3593 { print-sym~does~not~match~the~vertex~count~in~##1. }
3594 {
3595   The~forced~symbol~(triangle/square)~must~match~the~number~of~
3596   vertices~given~--use~'auto'~to~infer~it~automatically,~or~'none'~
3597   to~omit~it.
3598 }
3599 }

```

(End of definition for `spAfig-invalid-points` and `spPfig-invalid-points`.)

11.36.3 Definition of keys for `\spAfig` and `\spPfig`

We define the `<keys>` `read-cmd`, `prefix`, `print-sym`, `read-arg`, `read-sub` and `silent-cmd` for both commands.

```

read-cmd 3600 \clist_map_inline:nn { Afig , Pfig }
prefix    3601 {
print-sym 3602   \keys_define:nn { spintent / sp #1 }
read-arg   3603   {
read-sub   3604     read-cmd           .choices:nn = { school , mathcat }
silent-cmd 3605     {
unknown    3606       \int_case:nn { \l_keys_choice_int }
3607       {
3608         {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
3609         {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
3610       }
3611     },
3612     read-cmd           .initial:n = school,
3613     read-cmd           .value_required:n = true,
3614     read-cmd / unknown .code:n =
3615       \msg_error:nneee { spintent } { unknown-choice }
3616       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3617     prefix             .code:n =
3618       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
3619     prefix             .initial:n = ,
3620     prefix             .value_required:n = true,
3621     print-sym          .choices:nn = { auto , triangle , square , none }
3622     {
3623       \int_case:nn { \l_keys_choice_int }
3624       {
3625         {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
3626         {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
3627         {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
3628         {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
3629       }
3630     },
3631     print-sym          .initial:n = auto,
3632     print-sym          .value_required:n = true,
3633     print-sym / unknown .code:n =
3634       \msg_error:nneee { spintent } { unknown-choice }
3635       { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
3636     read-arg           .code:n =
3637       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
3638     read-arg           .initial:n = ,
3639     read-arg           .value_required:n = true,
3640     read-sub           .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
3641     read-sub           .initial:n = false,
3642     read-sub           .value_required:n = true,
3643     silent-cmd         .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3644     silent-cmd         .initial:n = false,
3645     silent-cmd         .value_required:n = true,
3646     unknown            .code:n =
3647     {
3648       \str_set:Nn \l__spintent_command_name_str { sp #1 }
3649       \__spintent_unknown_keys_cmd:n {##1}

```



```

3650     },
3651   }
3652 }

```

(End of definition for `read-cmd` and others.)

11.36.4 Internal functions for `\spAfig` and `\spPfig`

`\__spintent_spfig_sym:` Reads `\l__spintent_geo_luaset_poly_sym_str` and typesets the corresponding symbol: `\triangle` for `triangle`, `\mdlgwhtsquare` for square.

```

3653 \cs_new_protected:Nn \__spintent_spfig_sym:
3654 {
3655   \str_case:Nn \l__spintent_geo_luaset_poly_sym_str
3656   {
3657     { triangle } { \triangle }
3658     { square } { \mdlgwhtsquare }
3659   }
3660 }

```

(End of definition for `\__spintent_spfig_sym:`.)

`\__spintent_spfig_process:nn` Sets `\l__spintent_geo_fig_op_tl` to `área` or `perímetro` — the word Lua needs to build the `intent` — distinct from the visual symbol $\mathcal{A}/\mathcal{P}$, which is chosen separately when typesetting. When the point list is non-empty, the subscript is built with `\c_math_subscript_token` rather than a literal `_`, since the latter is not guaranteed to act as a subscript operator inside the token stream built by `\MathMLintent`.

For the single-argument grammar (§11.36), Lua returns `\l__spintent_geo_luaset_letra_word_str` and `\l__spintent_geo_luaset_letra_conector_str` separately: the connector (de-la, del, or empty) depends on the grammatical gender and part of speech of each table letter (F/P are nouns needing an article; B/L are adjectives, taking none), and is inserted before the word only when non-empty.

```

3661 \cs_new_protected:Nn \__spintent_spfig_process:nn
3662 {
3663   \str_if_eq:eeTF {#1} { Afig }
3664   { \tl_set:Nn \l__spintent_geo_fig_op_tl { área } }
3665   { \tl_set:Nn \l__spintent_geo_fig_op_tl { perímetro } }
3666   \str_set:Ne \l__spintent_geo_print_sym_str { \str_use:c { l__spintent_sp #1 _print_sym_str } }
3667   \str_if_eq:eeTF {#1} { Afig }
3668   { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
3669   { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
3670   \str_if_eq:NnT \l__spintent_geo_luaset_error_str { true }
3671   {
3672     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3673     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3674   }
3675   \tl_if_empty:cF { l__spintent_sp #1 _read_arg_tl }
3676   {
3677     \str_if_empty:VF \l__spintent_geo_luaset_vertices_str
3678     {
3679       \str_set:Ne \l__spintent_geo_luaset_intent_str
3680       {
3681         \l__spintent_geo_fig_op_tl -del-
3682         \tl_use:c { l__spintent_sp #1 _read_arg_tl } -
3683         \l__spintent_geo_luaset_vertices_str
3684       }
3685     }
3686   }
3687   \str_if_eq:NnTF \l__spintent_geo_print_sym_str { triangle }
3688   { \tl_set:Nn \l__spintent_geo_sym_word_tl { triángulo } }
3689   {
3690     \str_if_eq:NnTF \l__spintent_geo_print_sym_str { square }
3691     { \tl_set:Nn \l__spintent_geo_sym_word_tl { cuadrado } }
3692     { \tl_clear:N \l__spintent_geo_sym_word_tl }
3693   }
3694   \cs_set_protected:Npn \__spintent_spfig_sym_choice:
3695   {
3696     \str_if_eq:NnTF \l__spintent_geo_print_sym_str { triangle }
3697     { \triangle } { \mdlgwhtsquare }
3698   }
3699   \tl_if_empty:nTF {#2}
3700   {
3701     \str_if_eq:NnTF \l__spintent_geo_print_sym_str { none }
3702     {
3703       \tl_clear:N \l__spintent_geo_final_print_tl

```

```

3704         \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_fig_op_tl }
3705         \str_set:Ne \l__spintent_geo_mathcat_concept_str { \l__spintent_geo_fig_op_tl }
3706     }
3707     {
3708         \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfig_sym_choice: }
3709         \str_set:Ne \l__spintent_geo_final_intent_str
3710         {
3711             \l__spintent_geo_fig_op_tl - del - \l__spintent_geo_sym_word_tl
3712         }
3713         \str_set:Ne \l__spintent_geo_mathcat_concept_str
3714         {
3715             \l__spintent_geo_fig_op_tl - del - \l__spintent_geo_sym_word_tl
3716         }
3717     }
3718 }
3719 {
3720     \str_if_eq:VnTF \l__spintent_geo_luaset_intent_str { }
3721     {
3722         \str_if_empty:VTF \l__spintent_geo_luaset_letra_word_str
3723         {
3724             \str_if_eq:VnTF \l__spintent_geo_print_sym_str { none }
3725             {
3726                 \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3727                 \str_set:Ne \l__spintent_geo_final_intent_str
3728                 {
3729                     \l__spintent_geo_fig_op_tl -
3730                     \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
3731                     \l__spintent_geo_luaset_sub_spoken_str
3732                 }
3733             }
3734             {
3735                 \tl_set:Nn \l__spintent_geo_final_print_tl
3736                 {
3737                     \__spintent_spfig_sym_choice:
3738                     \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
3739                 }
3740                 \str_set:Ne \l__spintent_geo_final_intent_str
3741                 {
3742                     \l__spintent_geo_fig_op_tl - del - \l__spintent_geo_sym_word_tl -
3743                     \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
3744                     \l__spintent_geo_luaset_sub_spoken_str
3745                 }
3746             }
3747         }
3748     }
3749     \str_if_eq:VnF \l__spintent_geo_print_sym_str { none }
3750     {
3751         \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
3752         { \c_backslash_str sp #1 }
3753     }
3754     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3755     \str_set:Ne \l__spintent_geo_final_intent_str
3756     {
3757         \l__spintent_geo_fig_op_tl -
3758         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
3759         { \l__spintent_geo_luaset_letra_conector_str - }
3760         \l__spintent_geo_luaset_letra_word_str
3761         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
3762         {
3763             -
3764             \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
3765             \l__spintent_geo_luaset_sub_spoken_str
3766         }
3767     }
3768 }
3769 \str_set:Ne \l__spintent_geo_mathcat_concept_str { \l__spintent_geo_fig_op_tl }
3770 }
3771 {
3772     \str_case:VnTF \l__spintent_geo_print_sym_str
3773     {
3774         { auto }

```

```

3775         {
3776             \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3777             \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_intent_str }
3778         }
3779     { none }
3780     {
3781         \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
3782         \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_intent_str }
3783     }
3784 }
3785 {
3786     \str_if_eq:VVTTF \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
3787     {
3788         \tl_set:Nn \l__spintent_geo_final_print_tl
3789         {
3790             \__spintent_spfig_sym_choice:
3791             \l__spintent_geo_luaset_print_tl
3792         }
3793         \str_set:Ne \l__spintent_geo_final_intent_str { \l__spintent_geo_luaset_intent_str }
3794     }
3795     {
3796         \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
3797         { \c_backslash_str sp #1 }
3798     }
3799 }
3800 \str_set:Ne \l__spintent_geo_mathcat_concept_str { \l__spintent_geo_luaset_concept_str }
3801 }
3802 }
3803 \str_if_eq:eeT { \str_use:c { l__spintent_sp #1 _mode_str } } { mathcat }
3804 {
3805     \str_set:Ne \l__spintent_geo_luaset_intent_str { _ \l__spintent_geo_mathcat_concept_str }
3806 }
3807 \__spintent_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
3808 \str_if_eq:eeTF { \str_use:c { l__spintent_sp #1 _mode_str } } { school }
3809 {
3810     \MathMLintent
3811     {
3812         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3813         {
3814             \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3815         }
3816         \l__spintent_geo_final_intent_str
3817     }
3818     {
3819         \c__spintent_invisible_sep_tl
3820         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n { A } }
3821         \tl_if_empty:NF \l__spintent_geo_final_print_tl
3822         {
3823             \c_math_subscript_token { \l__spintent_geo_final_print_tl }
3824         }
3825     }
3826 }
3827 {
3828     \MathMLintent
3829     {
3830         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
3831         {
3832             \tl_use:c { l__spintent_sp #1 _prefix_tl } -
3833         }
3834         \l__spintent_geo_luaset_intent_str
3835     }
3836     {
3837         \c__spintent_invisible_sep_tl
3838         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n { A } }
3839         \tl_if_empty:NF \l__spintent_geo_final_print_tl
3840         {
3841             \c_math_subscript_token { \l__spintent_geo_final_print_tl }
3842         }
3843     }
3844 }
3845 }

```

(End of definition for `\__spintent_spfig_process:nn`.)

```

\__spintent_spfig_exec:nnn Sets the keys in the correct key space via \keys_set:nn, then delegates rendering to \__spintent_spfig_process:nn
3846 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
3847 {
3848   \keys_set:nn { spintent / sp #1 } { #2 }
3849   \__spintent_spfig_process:nn { #1 } { #3 }
3850 }

```

(End of definition for `\__spintent_spfig_exec:nnn`.)

\spAfig Converts the `-NoValue-` marker of an omitted `d()` argument into an explicit empty group before forwarding, since the Lua parser interprets an empty point list as “no figure — bare symbol only”.

\spPfig

```

3851 \NewDocumentCommand \spAfig { 0{} d() }
3852 {
3853   \mode_if_math:TF
3854   {
3855     \group_begin:
3856     \tl_if_novalue:nTF {#2}
3857       { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
3858       { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
3859     \group_end:
3860   }
3861   { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
3862 }
3863 \NewDocumentCommand \spPfig { 0{} d() }
3864 {
3865   \mode_if_math:TF
3866   {
3867     \group_begin:
3868     \tl_if_novalue:nTF {#2}
3869       { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
3870       { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
3871     \group_end:
3872   }
3873   { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
3874 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 22.)

11.37 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.37.1 Definition of variables for `\spcirc`

`\__spintent_geo_luaset_circ_radio_type_str` Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\__spintent_geo_luaset_circ_radio_type_str` holds one of `number`, `measure`, `label`, `segment`, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

3875 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
3876 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

`\__spintent_spcirc_mode_str` `\__spintent_spcirc_mode_str` and `\__spintent_spcirc_prefix_tl` follow the same semantics as the corresponding variables of `\spPoint` (§11.29). `\__spintent_spcirc_print_sym_str` stores the value of `print-sym` (usual or `odot`). `\__spintent_spcirc_read_sty_str` stores `read-sty` (circun or circle). `\__spintent_spcirc_radio_sty_str` stores `radio-sty`, only meaningful when the radius is a segment. `\__spintent_spcirc_sub_index_str` stores `sub-index`. `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

3877 \str_new:N \__spintent_spcirc_mode_str
3878 \tl_new:N \__spintent_spcirc_prefix_tl
3879 \str_new:N \__spintent_spcirc_print_sym_str
3880 \str_new:N \__spintent_spcirc_read_sty_str
3881 \str_new:N \__spintent_spcirc_radio_sty_str

```

```

3882 \str_new:N \l__spintent_spcirc_sub_index_str
3883 \str_new:N \l__spintent_spcirc_articulo_str
3884 \tl_new:N \l__spintent_spcirc_read_arg_tl

```

(End of definition for `\l__spintent_spcirc_mode_str` and others.)

```

\l__spintent_geo_circ_word_tl
\l__spintent_geo_circ_intent_str

```

Scratch variables used by `\__spintent_spcirc_print:n` to assemble the chosen word (circunferencia/círculo, or the `read-arg` override) and the final literal `intent` string.

```

3885 \tl_new:N \l__spintent_geo_circ_word_tl
3886 \str_new:N \l__spintent_geo_circ_intent_str

```

(End of definition for `\l__spintent_geo_circ_word_tl` and `\l__spintent_geo_circ_intent_str`.)

11.37.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg`

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

3887 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
3888 { Invalid~argument~format~in~\token_to_str:N \spcirc:~'#1'. }
3889 {
3890   The~argument~must~be~a~single~point~(the~center),~optionally~
3891   followed~by~a~comma~and~a~radius:~a~pure~number,~a~measure~
3892   (number~+~unit),~a~single~letter,~or~two~concatenated~letters~
3893   (a~segment).
3894 }

```

(End of definition for `spcirc-invalid-arg`.)

11.37.3 Definition of keys for `\spcirc`

`read-cmd`

`prefix`

`print-sym`

`read-sty`

`radio-sty`

`sub-index`

`read-sub`

`read-arg`

`silent-cmd`

`unknown`

`print-sym` chooses the visual symbol: usual for a plain C, `odot` (default) for `\odot`. `read-sty` chooses the verbalized word and its article: `circun` (default) for «la circunferencia», `circle` for «el círculo». `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). `sub-index` sets a subscript on the symbol; `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig` (§11.36).

```

3895 \keys_define:nn { spintent / spcirc }
3896 {
3897   read-cmd      .choices:nn = { school , mathcat }
3898   {
3899     \int_case:nn { \l_keys_choice_int }
3900     {
3901       {1} { \str_set:Nn \l__spintent_spcirc_mode_str { school } }
3902       {2} { \str_set:Nn \l__spintent_spcirc_mode_str { mathcat } }
3903     }
3904   },
3905   read-cmd      .initial:n = school,
3906   read-cmd      .value_required:n = true,
3907   prefix        .code:n =
3908   { \__spintent_sanitize_affix:Nn \l__spintent_spcirc_prefix_tl {#1} },
3909   prefix        .initial:n = ,
3910   prefix        .value_required:n = true,
3911   print-sym     .choices:nn = { usual , odot }
3912   {
3913     \int_case:nn { \l_keys_choice_int }
3914     {
3915       {1} { \str_set:Nn \l__spintent_spcirc_print_sym_str { usual } }
3916       {2} { \str_set:Nn \l__spintent_spcirc_print_sym_str { odot } }
3917     }
3918   },
3919   print-sym     .initial:n = odot,
3920   print-sym     .value_required:n = true,
3921   read-sty      .choices:nn = { circun , circle }
3922   {
3923     \int_case:nn { \l_keys_choice_int }
3924     {
3925       {1} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circun } }
3926       {2} { \str_set:Nn \l__spintent_spcirc_read_sty_str { circle } }
3927     }
3928   },
3929   read-sty      .initial:n = circun,
3930   read-sty      .value_required:n = true,
3931   radio-sty     .choices:nn = { side , mside }

```

```

3932     {
3933         \int_case:nn { \l_keys_choice_int }
3934         {
3935             {1} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { side } }
3936             {2} { \str_set:Nn \l__spintent_spcirc_radio_sty_str { mside } }
3937         }
3938     },
3939     radio-sty      .initial:n = mside,
3940     radio-sty      .value_required:n = true,
3941     sub-index      .code:n =
3942     { \str_set:Nn \l__spintent_spcirc_sub_index_str {#1} },
3943     sub-index      .initial:n = ,
3944     sub-index      .value_required:n = true,
3945     read-sub       .bool_set:N = \l__spintent_spcirc_read_sub_bool,
3946     read-sub       .initial:n = false,
3947     read-sub       .value_required:n = true,
3948     read-arg       .code:n =
3949     { \__spintent_sanitizé_affix:Nn \l__spintent_spcirc_read_arg_tl {#1} },
3950     read-arg       .initial:n = ,
3951     read-arg       .value_required:n = true,
3952     silent-cmd     .bool_set:N = \l__spintent_spcirc_silent_cmd_bool,
3953     silent-cmd     .initial:n = false,
3954     silent-cmd     .value_required:n = true,
3955     unknown        .code:n =
3956     {
3957         \str_set:Nn \l__spintent_command_name_str { spcirc }
3958         \__spintent_unknown_keys_cmd:n {#1}
3959     },
3960 }

```

(End of definition for read-cmd and others.)

11.37.4 Internal functions for \spcirc

__spintent_spcirc_sym: Typesets \odot or the plain letter C, according to print-sym.

```

3961 \cs_new_protected:Nn \__spintent_spcirc_sym:
3962 {
3963     \str_if_eq:VnTF \l__spintent_spcirc_print_sym_str { odot }
3964     {
3965         \odot
3966     }
3967     { C }
3968 }

```

(End of definition for __spintent_spcirc_sym:.)

__spintent_spcirc_word_and_article: Sets \l__spintent_geo_circ_word_tl to the chosen word — either read-arg, or circunferencia/círculo per read-sty — and \l__spintent_spcirc_articulo_str to the matching article (la/el). The article always follows read-sty, even when read-arg overrides the word itself.

```

3969 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
3970 {
3971     \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
3972     {
3973         \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
3974         {
3975             \tl_set:Nn \l__spintent_geo_circ_word_tl { circunferencia }
3976             \str_set:Nn \l__spintent_spcirc_articulo_str { la }
3977         }
3978         {
3979             \tl_set:Nn \l__spintent_geo_circ_word_tl { círculo }
3980             \str_set:Nn \l__spintent_spcirc_articulo_str { el }
3981         }
3982     }
3983     {
3984         \tl_set:Nn \l__spintent_geo_circ_word_tl { \l__spintent_spcirc_read_arg_tl }
3985         \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
3986         { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
3987         { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
3988     }
3989 }

```

(End of definition for __spintent_spcirc_word_and_article:.)

`\__spintenc_spcirc_radio_print:` Dispatches on `\l__spintenc_geo_luaset_circ_radio_type_str`, reusing the actual public sub-commands rather than reimplementing any parsing: `number` and `measure` delegate to `\spnum/\spunit`; `segment` delegates to `\spside/\spmside` according to `radio-sty` — both already accept the concatenated two-letter form (§11.32) that Lua hands over here. `label` is typeset as a bare letter, with no special intent wrapping. In every case, `\exp_args:No` is required to expand `\l__spintenc_geo_luaset_circ_radio_raw_str` into the sub-command’s argument, since a plain brace group does not expand a `str` variable on its own.

```

3990 \cs_new_protected:Nn \__spintenc_spcirc_radio_print:
3991 {
3992   \str_case:Vn \l__spintenc_geo_luaset_circ_radio_type_str
3993   {
3994     { number } { \exp_args:No \spnum { \l__spintenc_geo_luaset_circ_radio_raw_str } }
3995     { measure } { \exp_args:No \spunit { \l__spintenc_geo_luaset_circ_radio_raw_str } }
3996     { label } { \l__spintenc_geo_luaset_circ_radio_raw_str }
3997     { segment }
3998     {
3999       \str_if_eq:VnTF \l__spintenc_spcirc_radio_sty_str { side }
4000       { \exp_args:No \spside { \l__spintenc_geo_luaset_circ_radio_raw_str } }
4001       { \exp_args:No \spmside { \l__spintenc_geo_luaset_circ_radio_raw_str } }
4002     }
4003   }
4004 }

```

(End of definition for `\__spintenc_spcirc_radio_print:`.)

`\__spintenc_spcirc_print:n` Calls `\__spintenc_luafun_geo_circ_parse_and_set:n` to resolve the center and classify the radius, then assembles `\l__spintenc_geo_circ_intent_str` — the word, an optional `sub-index` suffix, `de-centro-en-`, the center, and `-y-radius` when a radius is present. Critically, when a radius is present, the visual is split across *two* separate `\MathMLintent` calls with `\__spintenc_spcirc_radio_print:` — which typesets a sub-command carrying its own independent *intent* — sitting as a *sibling* in between, rather than nested inside either one. Embedding a full sub-command’s own `\MathMLintent` inside another command’s content argument makes MathCAT read only the outer literal string, exactly as observed with `mover/msub` structures (§11.34); as *siblings*, both are read in sequence as one continuous phrase.

```

4005 \cs_new_protected:Nn \__spintenc_spcirc_print:n
4006 {
4007   \__spintenc_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
4008   \str_if_eq:VnT \l__spintenc_geo_luaset_error_str { true }
4009   {
4010     \msg_error:nne { spintenc } { spcirc-invalid-arg } { \exp_not:n {#1} }
4011   }
4012   \__spintenc_spcirc_word_and_article:
4013   \str_set:N \l__spintenc_geo_circ_intent_str { \l__spintenc_geo_circ_word_tl }
4014   \str_if_empty:NF \l__spintenc_spcirc_sub_index_str
4015   {
4016     \str_put_right:Nn \l__spintenc_geo_circ_intent_str
4017     {
4018       -
4019       \bool_if:NT \l__spintenc_spcirc_read_sub_bool { sub - }
4020       \l__spintenc_spcirc_sub_index_str
4021     }
4022   }
4023   \str_put_right:Nn \l__spintenc_geo_circ_intent_str
4024   {
4025     -de-centro-en-\l__spintenc_geo_luaset_print_tl
4026   }
4027   \str_if_empty:VF \l__spintenc_geo_luaset_circ_radio_type_str
4028   {
4029     \str_put_right:Nn \l__spintenc_geo_circ_intent_str { -y-radius }
4030   }
4031   \__spintenc_invisible_sep: % need by MathCAT (bug in luamml https://github.com/latex3/luamml/issues/26)
4032   \str_if_empty:VTF \l__spintenc_geo_luaset_circ_radio_type_str
4033   {
4034     \MathMLintent
4035     {
4036       \tl_if_empty:NF \l__spintenc_spcirc_prefix_tl
4037       {
4038         \tl_use:N \l__spintenc_spcirc_prefix_tl -
4039       }
4040       \l__spintenc_geo_circ_intent_str
4041     }
4042   }

```



```

4043         \c__spintent_invisible_sep_tl
4044         \__spintent_spcirc_sym:
4045         \str_if_empty:NF \l__spintent_spcirc_sub_index_str
4046         {
4047             \c_math_subscript_token { \l__spintent_spcirc_sub_index_str }
4048         }
4049         ( \l__spintent_geo_luaset_print_tl )
4050     }
4051 }
4052 {
4053     \MathMLintent
4054     {
4055         \tl_if_empty:NF \l__spintent_spcirc_prefix_tl
4056         {
4057             \tl_use:N \l__spintent_spcirc_prefix_tl -
4058         }
4059         \l__spintent_geo_circ_intent_str
4060     }
4061     {
4062         \c__spintent_invisible_sep_tl
4063         \__spintent_spcirc_sym:
4064         \str_if_empty:NF \l__spintent_spcirc_sub_index_str
4065         {
4066             \c_math_subscript_token { \l__spintent_spcirc_sub_index_str }
4067         }
4068         ( \l__spintent_geo_luaset_print_tl ,
4069         }
4070         \__spintent_invisible_sep:
4071         \__spintent_spcirc_radio_print:
4072         \MathMLintent { :silent } { } ) }
4073     }
4074 }

```

(End of definition for `\__spintent_spcirc_print:n`.)

`\__spintent_spcirc_exec:nn`

```

4075 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
4076 {
4077     \keys_set:nn { spintent / spcirc } { #1 }
4078     \__spintent_spcirc_print:n { #2 }
4079 }

```

(End of definition for `\__spintent_spcirc_exec:nn`.)

`\spcirc` Follows the same `d()`-argument pattern as `\spmcd/\spmcm` and `\spAfig/\spPfig`; an omitted radius forwards an explicit empty group rather than the `-NoValue-` marker.

```

4080 \NewDocumentCommand \spcirc { 0{} d() }
4081 {
4082     \mode_if_math:TF
4083     {
4084         \group_begin:
4085         \tl_if_novalue:nTF {#2}
4086         { \__spintent_spcirc_exec:nn { #1 } { } }
4087         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
4088         \group_end:
4089     }
4090     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
4091 }

```

(End of definition for `\spcirc`. This function is documented on page ??.)

11.38 Finish package

Finish package implementation.

```

4092 \file_input_stop:
4093 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	92
C	
cifras-sep	<u>274</u> , <u>559</u> , <u>832</u> , <u>1781</u> , <u>1852</u> , <u>2264</u>
clist commands:	
\clist_map_inline:nn	88, <u>91</u> , <u>95</u>
cmd-key-unknown	<u>194</u>
cmd-key-value-unknown	<u>194</u>
Commands provide by spintent :	
\setspintent	31
\spmoney	44
\spnum	34
\spsiglo	53, <u>56</u>
\spunit	39
currency	<u>832</u>
currency-no-subunits	<u>801</u>
D	
dolar-math	<u>832</u>
E	
exp commands:	
\exp_args:No	108
I	
inline-numeric-denominator	<u>537</u>
invalid-angle-format	<u>2214</u>
invalid-century-format	<u>1228</u>
invalid-currency	<u>801</u>
invalid-date-format	<u>1458</u>
invalid-time-format	<u>1379</u>
J	
join-word	<u>2339</u>
K	
Keys for \spAfig provide by spintent :	
prefix	101
print-sym	100, <u>101</u>
read-arg	100, <u>101</u>
read-cmd	101
read-sub	101
silent-cmd	101
Keys for \spangle provide by spintent :	
recto	91, <u>92</u>
silent-cmd	92
Keys for \spcirc provide by spintent :	
print-sym	105–107
radio-sty	105, <u>106</u> , <u>108</u>
read-arg	106, <u>107</u>
read-sty	105–107
read-sub	106
sub-index	105, <u>106</u> , <u>108</u>
Keys for \spdiv provide by spintent :	
cifras-sep	65
notation-sym	65
read-parens	65
read-period-sep	65
read-sign	65
read-sym	65
set-sym	65
suffix	65
upper-case	65
Keys for \spdsym provide by spintent :	
prefix	62
read-sym	62
set-sym	62
silent-cmd	62
upper-case	62
Keys for \spmangle provide by spintent :	
print-m	91, <u>92</u>
Keys for \spmcd provide by spintent :	
prefix	68, <u>71</u>
read-cmd	68
result-num	71
result	68
sample-arg	68
silent-cmd	68
upper-case	68
Keys for \spmcm provide by spintent :	
prefix	68, <u>71</u>
read-cmd	68
result-num	71
result	68
sample-arg	68
silent-cmd	68
upper-case	68
Keys for \spmoney provide by spintent :	
cifras-sep	46
currency	46
dolar-math	46
sub-units	46
Keys for \spmQ provide by spintent :	
join-word	76–78
print-dfrac	77–80
read-parens	77, <u>78</u>
use-spnZ	77, <u>79</u> , <u>80</u>
wrap-parens	76–78
Keys for \spmside provide by spintent :	
print-m	88, <u>89</u>
Keys for \spmSYM provide by spintent :	
not-print	63
prefix	63
read-sym	63
set-sym	63
suffix	63
Keys for \spmul provide by spintent :	
cifras-sep	66
notation-sym	66
read-parens	66
read-period-sep	66
read-sign	66
read-sym	66
set-sym	66
wrap-parens	66
Keys for \spnD provide by spintent :	
result	71
silent-cmd	71
Keys for \spnM provide by spintent :	
result	71
silent-cmd	71

Keys for \spnum provide by spintent :		
cifras-sep	33–35	
notation-sym	34	
read-period-sep	34, 36	
read-sign	34	
Keys for \spnZ provide by spintent :		
cifras-sep	75	
read-parens	75	
read-sign	75	
wrap-parens	75	
Keys for \spPfig provide by spintent :		
prefix	101	
print-sym	101	
read-arg	101	
read-cmd	101	
read-sub	101	
silent-cmd	101	
Keys for \spPoint provide by spintent :		
prefix	81	
read-cmd	81	
silent-cmd	81, 82	
Keys for \spPoly provide by spintent :		
print-sym	97, 98	
silent-cmd	98	
Keys for \sprayo provide by spintent :		
prefix	86	
read-cmd	86	
read-sty	86	
silent-cmd	86	
Keys for \sprecta provide by spintent :		
prefix	83	
read-cmd	83	
silent-cmd	83, 84	
Keys for \spshort provide by spintent :		
read-arg	51	
small-caps	51	
Keys for \spside provide by spintent :		
read-arg	88	
read-sty	88	
silent-cmd	89	
Keys for \spsiglo provide by spintent :		
print-era	54, 55	
Keys for \spunit provide by spintent :		
cifras-sep	40	
notation-sym	40	
print-cdot	40	
read-cdot	40	
read-period-sep	40	
read-slash	40	
Keys for \spusep provide by spintent :		
silent	60	
size	60	
\keys	98	
keys commands:		
\keys_set:nn	83, 85, 88, 91, 94, 97, 100, 105	
M		
\MathMLint	98, 100, 102, 108	
mc-invalid-arguments	1934	
md-invalid-argument	2106	
\mdlgwhtsquare	98, 102	
\measuredangle	92	
\measuredrightangle	92	
Lua module provide by spintent :		
spintent.lua	28, 29, 32–34, 39, 43–46, 50–56, 58, 68, 70, 73, 76, 81	
msg commands:		
\msg_error:nnnn	88	
N		
not-print	1707	
notation-sym	274, 559, 1781, 1852	
O		
\odot	107	
\overlefttrightharpoon	83, 84	
P		
Packages :		
expl3	28, 33, 34	
l3doc	28	
lua-unicode-math	28	
luamml	30, 76, 78	
tagpdf	50, 51, 55	
unicode-math	28, 61	
prefix	1637, 1707, 1941, 2080, 2542, 2646, 2789, 2924, 3103, 3280, 3427, 3600, 3895	
print-dfrac	2339	
print-era	1236	
print-m	2924, 3103	
print-sym	3427, 3600, 3895	
R		
radio-sty	3895	
read-arg	1106, 2924, 3600, 3895	
read-cdot	559	
read-cmd	1941, 2542, 2646, 2789, 2924, 3103, 3280, 3427, 3600, 3895	
read-parens	1781, 1852, 2264, 2339	
read-period-sep	274, 559, 1781, 1852	
read-sign	274, 1781, 1852, 2264	
read-slash	559	
read-sty	2789, 2924, 3895	
read-sub	3600, 3895	
read-sym	1637, 1707, 1781, 1852	
recto	3103	
result	1941, 2080	
result-num	2080	
\rightangle	92	
S		
sample-arg	1941, 2080	
seq commands:		
\seq_set_from_clist:Nn	57	
set-sym	1637, 1707, 1781, 1852	
\setspintent	3, 31, 182	
silent	1502	
silent-cmd	1941, 2080, 2542, 2646, 2789, 2924, 3103, 3280, 3427, 3600, 3895	
size	1502	
small-caps	1106	
\spAfig	22, 100, 3851	
spAfig-invalid-points	3567	
\spang	21, 73, 2254	
\spangle	20, 91, 3244	
spangle-invalid-arg	3091	
\sparc	21, 94, 3397	
sparc-invalid-points	3269	
\spcirc	105, 4080	

spcirc-invalid-arg	3887	\l__spintent_luaset_not_decimal_period_- str	243
\spdate	11, 58, 1482	\l__spintent_luaset_only_part_dec_str .	238
\spdiv	13, 65, 1842	\l__spintent_luaset_only_part_int_str .	238
spdiv-empty-argument	1801	\l__spintent_luaset_only_part_period_- str	238
\spdsym	12, 62, 1684	\l__spintent_luaset_part_dec_tl	231
spintent internal commands:		\l__spintent_luaset_part_int_tl	79, 231
__spintent_collapse_hyphens:N	119	\l__spintent_luaset_part_period_tl ...	231
\l__spintent_command_name_str	194	\l__spintent_luaset_sign_tl	79, 231
\l__spintent_geo_circ_intent_str ..	108, 3885	\l__spintent_luaset_status_str	250
\l__spintent_geo_circ_word_tl	107, 3885	\l__spintent_mc_args_seq	1923
\l__spintent_geo_fig_op_tl	102, 2520	__spintent_mc_process_args:nn	1982
\l__spintent_geo_luaset_angulo_case_- str	2520	\l__spintent_md_out_seq	2070
\l__spintent_geo_luaset_circ_radio_raw_- str	108, 3875	__spintent_print_spunit_hierarchy: ...	660
\l__spintent_geo_luaset_circ_radio_type_- str	108, 3875	__spintent_sanitize_affix:Nn	127
\l__spintent_geo_luaset_error_str .	81, 2520	\l__spintent_saved_luamml_flag_int ..	78, 79, 2323
\l__spintent_geo_luaset_intent_pts_str .	97, 98, 2520	\l__spintent_setspintent_input_arg_cmd_- tl	145
\l__spintent_geo_luaset_intent_str	81, 84, 97, 98, 2520	__spintent_setspintent_parse_args:nn .	165
\l__spintent_geo_luaset_is_name_str ..	83, 84, 2520	__spintent_setspintent_set_keys:nn ...	154
\l__spintent_geo_luaset_letra_conector_- str	102	\l__spintent_smparc_prefix_tl	3264
\l__spintent_geo_luaset_letra_word_str	102	\l__spintent_sp\#1_mode_str	2906, 3086
\l__spintent_geo_luaset_poly_sym_str	98, 100, 102, 2520	\l__spintent_sp\#1_prefix_tl	2906, 3086
\l__spintent_geo_luaset_print_tl ..	81, 2520	\l__spintent_sp\#1_read_arg_str	2906
\l__spintent_geo_side_cmd_tl ..	84, 89, 93, 2520	\l__spintent_sp\#1_read_sty_str	2906
__spintent_invisible_sep:	105	\l__spintent_sPAfig_mode_str	3556
\c__spintent_invisible_sep_tl	78, 105	\l__spintent_sPAfig_prefix_tl	3556
__spintent_luafun_clean_split_and_set:n	62, 231	\l__spintent_sPAfig_print_sym_str	3556
__spintent_luafun_spang_parse:n	74	\l__spintent_sPAfig_read_arg_tl	3556
__spintent_luafun_spdate_parse:n	59	\l__spintent_spang_luaset_error_str ...	2210
__spintent_luafun_spmoney_lookup_data:n	62	\l__spintent_spang_luaset_grado_str ...	2210
__spintent_luafun_spmQ_scale_int:nn ...	62	\l__spintent_spang_luaset_minuto_str ..	2210
__spintent_luafun_sptime_parse:n	58	\l__spintent_spang_luaset_segundo_str .	2210
__spintent_luafun_spunit_lookup_alias:n	62	__spintent_spang_process:n	2219
\l__spintent_luaset_arg_above_tl	250	__spintent_spangle_exec:nnn	3239
\l__spintent_luaset_arg_below_tl	250	__spintent_spangle_process:nn	3159
\l__spintent_luaset_dec_is_all_zeros_- str	243	__spintent_spangle_sym:n	3140
\l__spintent_luaset_dec_sep_tl	231	__spintent_sparc_exec:nnn	3392
\l__spintent_luaset_decimal_period_str	243	\l__spintent_sparc_mode_str	3264
\l__spintent_luaset_denom_has_number_- str	250	\l__spintent_sparc_prefix_tl	3264
\l__spintent_luaset_exponent_tl	243	__spintent_sparc_print_arc:n	3315
\l__spintent_luaset_format_part_dec_str	238	__spintent_sparc_process:nn	3321
\l__spintent_luaset_format_part_int_str	238	__spintent_sparc_wide_arc:n	3311
\l__spintent_luaset_has_exponent_str ..	243	\l__spintent_spcirc_articulo_str	105, 107, 3877
\l__spintent_luaset_has_integer_str ..	79, 80, 231	__spintent_spcirc_exec:nn	4075
\l__spintent_luaset_has_natural_str ...	231	\l__spintent_spcirc_mode_str	105, 3877
\l__spintent_luaset_has_units_str	250	\l__spintent_spcirc_prefix_tl	105, 3877
\l__spintent_luaset_millions_str	243	__spintent_spcirc_print:n	4005
\l__spintent_luaset_not_decimal_not_- period_str	243	\l__spintent_spcirc_print_sym_str .	105, 3877
		__spintent_spcirc_radio_print:	3990
		\l__spintent_spcirc_radio_sty_str .	105, 3877
		\l__spintent_spcirc_read_sty_str ..	105, 3877
		\l__spintent_spcirc_sub_index_str .	105, 3877
		__spintent_spcirc_sym:	3961
		__spintent_spcirc_word_and_article: ..	3969
		\l__spintent_spdate_luaset_error_str ..	1456

<code>\l__spintent_spdate_luaset_output_str</code>	1456	<code>__spintent_spmoney_render_only_integer_-</code>	
<code>__spintent_spdate_process:n</code>	59, 1467	<code>node:</code>	891
<code>__spintent_spdiv_process:nnn</code>	1827	<code>\l__spintent_spmoney_res_main_voice_str</code>	793
<code>__spintent_spdiv_render_op:nn</code>	1806	<code>__spintent_spmoney_set_and_print_sub_-</code>	
<code>__spintent_spdiv_render_parens:nn</code>	1815	<code>units:</code>	964
<code>\l__spintent_spdsym_prefix_tl</code>	1633	<code>__spintent_spmoney_set_currency:n</code>	818
<code>\l__spintent_spdsym_read_sym_tl</code>	1633	<code>\l__spintent_spmoney_sub_units_bool</code>	793
<code>\l__spintent_spdsym_suffix_tl</code>	1633	<code>\l__spintent_spmoney_subnum_int</code>	793
<code>\l__spintent_spdsym_symbol_tl</code>	1633	<code>__spintent_spmoney_validate_money:</code>	850
<code>__spintent_spfig_exec:nnn</code>	3846	<code>__spintent_spmQ_exec:nnnn</code>	2498
<code>__spintent_spfig_process:nn</code>	3661	<code>\l__spintent_spmQ_join_word_str</code>	78, 2323
<code>__spintent_spfig_sym:</code>	3653	<code>\l__spintent_spmQ_math_style_tl</code>	2323
<code>\l__spintent_spmarc_mode_str</code>	3264	<code>__spintent_spmQ_print:nnn</code>	2490
<code>__spintent_spmc_process:nnnn</code>	2004	<code>\l__spintent_spmQ_print_dfrac_bool</code>	79
<code>\l__spintent_spmcd_prefix_tl</code>	1923	<code>__spintent_spmQ_print_sacle_int:n</code>	2447
<code>\l__spintent_spmcd_print_result_bool</code>	1923	<code>__spintent_spmQ_print_scale_int:</code>	2438
<code>\l__spintent_spmcd_print_upper_bool</code>	1923	<code>__spintent_spmQ_print_spnZ_dfrac:nn</code>	2475
<code>\l__spintent_spmcd_read_terse_bool</code>	1923	<code>__spintent_spmQ_set_join_word:</code>	2429
<code>\l__spintent_spmcd_suffix_tl</code>	1923	<code>__spintent_spmQ_set_wrap_parens_size:</code>	2374
<code>\l__spintent_spmcm_prefix_tl</code>	1923	<code>__spintent_spmQ_start_wrap_parens:</code>	2409
<code>\l__spintent_spmcm_print_result_bool</code>	1923	<code>__spintent_spmQ_stop_wrap_parens:</code>	2409
<code>\l__spintent_spmcm_print_upper_bool</code>	1923	<code>\l__spintent_spmQ_use_spnZ_bool</code>	79
<code>\l__spintent_spmcm_read_terse_bool</code>	1923	<code>\l__spintent_spmQ_sym_prefix_tl</code>	1703
<code>\l__spintent_spmcm_spmcd_luaset_error_-</code>		<code>\l__spintent_spmQ_sym_read_sym_tl</code>	1703
<code>str</code>	1923	<code>\l__spintent_spmQ_sym_suffix_tl</code>	1703
<code>\l__spintent_spmcm_suffix_tl</code>	1923	<code>\l__spintent_spmQ_sym_symbol_tl</code>	1703
<code>__spintent_spmoney_build_tree:</code>	1011	<code>__spintent_spmul_process:nnn</code>	1898
<code>__spintent_spmoney_check_sub_units:</code>	964	<code>__spintent_spmul_render_op:nn</code>	1877
<code>\l__spintent_spmoney_extracted_curr_str</code>	793	<code>__spintent_spmul_render_parens:nn</code>	1886
<code>\l__spintent_spmoney_extracted_num_tl</code>	793	<code>\l__spintent_spnD_luaset_is_truncated_-</code>	
<code>__spintent_spmoney_intent_int:</code>	900	<code>str</code>	2070
<code>\l__spintent_spmoney_intent_money_str</code>	793	<code>\l__spintent_spnD_luaset_result_tl</code>	2070
<code>\l__spintent_spmoney_intent_sign_str</code>	793	<code>\l__spintent_spnD_prefix_tl</code>	2070
<code>\l__spintent_spmoney_intent_sub_unit_-</code>		<code>\l__spintent_spnD_print_result_bool</code>	2070
<code>str</code>	793	<code>\l__spintent_spnD_print_result_num_tl</code>	2070
<code>\l__spintent_spmoney_luaset_conde_str</code>	783	<code>\l__spintent_spnD_silent_cmd_bool</code>	2070
<code>\l__spintent_spmoney_luaset_currency_arg_-</code>		<code>\l__spintent_spnM_luaset_is_truncated_-</code>	
<code>str</code>	783	<code>str</code>	2070
<code>\l__spintent_spmoney_luaset_plur_str</code>	783	<code>\l__spintent_spnM_luaset_result_tl</code>	2070
<code>\l__spintent_spmoney_luaset_position_-</code>		<code>\l__spintent_spnM_prefix_tl</code>	2070
<code>str</code>	783	<code>\l__spintent_spnM_print_result_bool</code>	2070
<code>\l__spintent_spmoney_luaset_print_iso_-</code>		<code>\l__spintent_spnM_print_result_num_tl</code>	2070
<code>str</code>	783	<code>\l__spintent_spnM_silent_cmd_bool</code>	2070
<code>\l__spintent_spmoney_luaset_sing_str</code>	783	<code>__spintent_spnM_spnD_exec:nnnn</code>	2151
<code>\l__spintent_spmoney_luaset_status_str</code>	783	<code>__spintent_spnM_spnD_intent:nn</code>	2116
<code>\l__spintent_spmoney_luaset_subplur_str</code>	783	<code>__spintent_spnM_spnD_process:nnnnn</code>	2111
<code>\l__spintent_spmoney_luaset_subsing_str</code>	783	<code>__spintent_spnM_spnD_result:n</code>	2132
<code>\l__spintent_spmoney_luaset_symbol_str</code>	783	<code>__spintent_spnM_spnD_silent:nn</code>	2124
<code>__spintent_spmoney_normalize_key:n</code>	818	<code>\l__spintent_spnQ_intent_read_sign_str</code>	2323
<code>__spintent_spmoney_parse_and_route:n</code>	1059	<code>\l__spintent_spnQ_read_parens_bool</code>	78
<code>__spintent_spmoney_print_dec_sub_-</code>		<code>\l__spintent_spnQ_wrap_parens_l_tl</code>	77, 78, 2323
<code>unit:</code>	1011	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_print_int:</code>	900	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_print_integer_-</code>		<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>continue:</code>	1011	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_print_sep_con:</code>	1011	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_print_symbol:</code>	866	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_render_layout:</code>	879	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323
<code>__spintent_spmoney_render_number_node:</code>	891	<code>\l__spintent_spnQ_wrap_parens_r_tl</code>	77, 78, 2323

<code>\l__spintent_spnun_notation_sym_tl</code> ...	269	<code>__spintent_spsshort_render_success:</code> ...	1143
<code>__spintent_spnun_print_integer:</code>	436	<code>__spintent_spsshort_tag_span:nn</code>	1093
<code>__spintent_spnun_print_num_start:</code> ...	436	<code>__spintent_spside_exec:nnn</code>	3061
<code>__spintent_spnun_print_number_core:</code> ..	436	<code>__spintent_spside_process:nn</code>	2976
<code>__spintent_spnun_print_part_dec:</code>	415	<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1224
<code>__spintent_spnun_print_part_period:</code> ..	415	<code>\l__spintent_spsiglo_luaset_error_str</code> .	1224
<code>__spintent_spnun_print_period_bar:</code> ...	344	<code>\l__spintent_spsiglo_luaset_roman_min_-</code> <code>str</code>	1224
<code>\l__spintent_spnun_read_period_sep_str</code> .	34 , 269	<code>\l__spintent_spsiglo_print_era_str</code> ...	1224
<code>\l__spintent_spnun_read_terse_bool</code> .	34 , 269	<code>__spintent_spsiglo_print_math:n</code>	1308
<code>__spintent_spnun_render_dec:</code>	327	<code>__spintent_spsiglo_print_text:n</code>	1308
<code>__spintent_spnun_render_decimal_sep:</code> .	381	<code>__spintent_spsiglo_render_simple:nn</code> ..	1259
<code>__spintent_spnun_render_int:</code>	327	<code>__spintent_spsiglo_render_with_-</code> <code>era:nnn</code>	1259
<code>__spintent_spnun_render_only_period:</code> .	344	<code>__spintent_spsiglo_tag_span:nn</code>	1295
<code>__spintent_spnun_render_part:n</code>	327	<code>__spintent_sptime_build_seconds:nn</code> ...	1388
<code>__spintent_spnun_render_period_sep:</code> ..	344	<code>__spintent_sptime_build_tree:</code>	58 , 1406
<code>__spintent_spnz_process:nn</code>	2297	<code>\l__spintent_sptime_luaset_base_str</code> ...	1374
<code>__spintent_spnz_render_parens:</code>	2285	<code>\l__spintent_sptime_luaset_error_str</code> ..	1374
<code>\l__spintent_spPfig_u_print_sym_str</code>	3556	<code>\l__spintent_sptime_luaset_has_seconds_-</code> <code>str</code>	1374
<code>\l__spintent_spPfig_mode_str</code>	3556	<code>\l__spintent_sptime_luaset_is_fraction_-</code> <code>str</code>	1374
<code>\l__spintent_spPfig_prefix_tl</code>	3556	<code>\l__spintent_sptime_luaset_seconds_str</code>	1374
<code>\l__spintent_spPfig_read_arg_tl</code>	3556	<code>\l__spintent_spunit_exp_seq</code>	530
<code>__spintent_spPoint_exec:nn</code>	2622	<code>\l__spintent_spunit_exponent_tl</code>	530
<code>\l__spintent_spPoint_mode_str</code> 83 , 86 , 88 , 91 , 95 , 97 , 2532		<code>__spintent_spunit_format_canonical:</code> ..	593
<code>\l__spintent_spPoint_prefix_tl</code> 83 , 86 , 88 , 91 , 95 , 97 , 2532		<code>\l__spintent_spunit_is_mult_bool</code>	530
<code>__spintent_spPoint_print:n</code>	2570	<code>\l__spintent_spunit_luaset_canonical_-</code> <code>str</code>	523
<code>__spintent_spPoly_exec:nn</code>	3541	<code>\l__spintent_spunit_luaset_compact_exp_-</code> <code>str</code>	523
<code>\l__spintent_spPoly_mode_str</code>	3417	<code>\l__spintent_spunit_luaset_is_compact_-</code> <code>str</code>	523
<code>\l__spintent_spPoly_prefix_tl</code>	3417	<code>\l__spintent_spunit_luaset_is_-</code> <code>sexagesimal_str</code>	523
<code>__spintent_spPoly_print:n</code>	3464	<code>\l__spintent_spunit_luaset_lookup_status_-</code> <code>str</code>	523
<code>__spintent_spPoly_sym:</code>	3456	<code>\l__spintent_spunit_luaset_read_str</code> ...	523
<code>__spintent_sprayo_exec:nn</code>	2891	<code>\l__spintent_spunit_luaset_status_str</code> .	523
<code>\l__spintent_sprayo_mode_str</code>	2778	<code>\l__spintent_spunit_mult_seq</code>	530
<code>\l__spintent_sprayo_prefix_tl</code>	2778	<code>__spintent_spunit_process_base_exp:n</code> .	593
<code>__spintent_sprayo_print:n</code>	2830	<code>__spintent_spunit_process_mult:n</code>	653
<code>\l__spintent_sprayo_read_sty_str</code> ..	87 , 2778	<code>\l__spintent_spunit_read_cdot_bool</code> ...	530
<code>__spintent_spread_exec:nn</code>	1614	<code>\l__spintent_spunit_read_slash_str</code> ...	530
<code>\l__spintent_spread_intent_tl</code>	1613	<code>__spintent_spunit_safe_exec:n</code>	672
<code>__spintent_sprecta_exec:nn</code>	2763	<code>\l__spintent_spunit_unit_name_tl</code>	530
<code>\l__spintent_sprecta_mode_str</code>	2637	<code>\l__spintent_spusep_arg_tl</code>	59 , 60 , 1499
<code>\l__spintent_sprecta_prefix_tl</code>	2637	<code>__spintent_spusep_convert:n</code>	1561
<code>__spintent_sprecta_print:n</code>	2674	<code>\l__spintent_spusep_left_tl</code>	59 , 60 , 1499
<code>__spintent_spsshort_execute_layout:</code> ...	1167	<code>\l__spintent_spusep_right_tl</code>	59 , 60 , 1499
<code>\l__spintent_spsshort_luaset_base_str</code> ..	1084	<code>__spintent_spusep_set_parens:nn</code>	1550
<code>\l__spintent_spsshort_luaset_expanded_-</code> <code>str</code>	1084	<code>\l__spintent_spusep_silent_bool</code>	60
<code>\l__spintent_spsshort_luaset_layout_str</code>	1084	<code>__spintent_unknown_keys:n</code>	221
<code>\l__spintent_spsshort_luaset_output_str</code>	1084	<code>__spintent_unknown_keys:nn</code>	221
<code>\l__spintent_spsshort_luaset_status_str</code>	1084	<code>\spmangle</code>	20 , 91 , 3244
<code>\l__spintent_spsshort_luaset_suffix_str</code>	1084	<code>spmangle-invalid-arg</code>	3091
<code>__spintent_spsshort_print:nn</code>	1120	<code>\spmarc</code>	21 , 94 , 3397
<code>__spintent_spsshort_process_and_-</code> <code>render:n</code>	1132	<code>spmarc-invalid-points</code>	3269
<code>__spintent_spsshort_render_bypass:n</code> ...	1143		
<code>__spintent_spsshort_render_error:n</code> ...	1143		
<code>__spintent_spsshort_render_fallback:</code> ..	1143		

<code>\spmc</code>	15, 68, <u>2050</u>	<code>spside-invalid-points</code>	<u>2913</u>
<code>\spmc</code>	15, 68, <u>2050</u>	<code>\spsiglo</code>	11, 53, <u>1361</u>
<code>\spmoney</code>	6, 44, <u>1073</u>	<code>\sptime</code>	11, 56, <u>1438</u>
<code>\spmQ</code>	16, 76, <u>2510</u>	<code>\spunit</code>	4, 39, 40, 43, <u>718</u>
<code>spmQ-not-integer</code>	<u>2329</u>	<code>spunit-invalid-unit</code>	<u>537</u>
<code>spmQ-zero-denominator</code>	<u>2329</u>	<code>spunit-no-units</code>	<u>537</u>
<code>\spmside</code>	19, 88, <u>3066</u>	<code>\spunitalias</code>	6, 43, <u>765</u>
<code>spmside-invalid-points</code>	<u>2913</u>	<code>spunitalias-duplicate</code>	<u>730</u>
<code>\spmsym</code>	12, 63, <u>1756</u>	<code>spunitalias-invalid-expr</code>	<u>730</u>
<code>\spmul</code>	13, 66, <u>1913</u>	<code>\spusep</code>	12, 59, <u>1599</u>
<code>spmul-empty-argument</code>	<u>1872</u>	<code>spusep-empty-arg</code>	<u>1492</u>
<code>\spnD</code>	14, 70, <u>2190</u>	<code>spusep-invalid-arg</code>	<u>1492</u>
<code>\spnewunit</code>	6, 43, <u>751</u>	str commands:	
<code>spnewunit-duplicate</code>	<u>730</u>	<code>\str_if_eq:nnTF</code>	<u>57</u>
<code>\spnM</code>	14, 70, <u>2190</u>	<code>\str_lowercase:n</code>	<u>57</u>
<code>\spnum</code>	3, 34, 35, <u>505</u>	<code>\str_uppercase:n</code>	<u>57</u>
<code>spnum-has-units</code>	<u>491</u>	sub-index	<u>3895</u>
<code>spnum-no-digits</code>	<u>491</u>	sub-units	<u>832</u>
<code>\spnZ</code>	16, 74, <u>2313</u>	suffix	<u>1637</u> , <u>1707</u>
<code>spnz-not-integer</code>	<u>2280</u>		
<code>\spPfig</code>	23, 100, <u>3851</u>	T	
<code>spPfig-invalid-points</code>	<u>3567</u>	token commands:	
<code>\spPoint</code>	17, 81, <u>2627</u>	<code>\c_math_subscript_token</code>	<u>102</u>
<code>spPoint-invalid-point</code>	<u>2534</u>	<code>\triangle</code>	98, <u>102</u>
<code>\spPoly</code>	22, 97, <u>3546</u>		
<code>spPoly-invalid-points</code>	<u>3419</u>	U	
<code>\sprayo</code>	18, 86, <u>2896</u>	unknown	<u>1502</u> , <u>1637</u> , <u>1707</u> , <u>2339</u> , <u>2542</u> , <u>2646</u> , <u>2789</u> , <u>2924</u> , <u>3103</u> , <u>3280</u> , <u>3427</u> , <u>3600</u> , <u>3895</u>
<code>sprayo-invalid-points</code>	<u>2781</u>	unknown-abbreviation	<u>1090</u>
<code>\spread</code>	11, 61, <u>1623</u>	unknown-choice	<u>194</u>
<code>spread-empty-args</code>	<u>1610</u>	unknown-command-setup	<u>145</u>
<code>\sprecta</code>	18, 83, <u>2768</u>	upper-case	<u>1941</u>
<code>sprecta-invalid-points</code>	<u>2639</u>	use-spnZ	<u>2339</u>
<code>\spshort</code>	7, 50, <u>1211</u>		
<code>\spside</code>	18, 88, <u>3066</u>	W	
		wrap-parens	<u>1781</u> , <u>1852</u> , <u>2264</u> , <u>2339</u>